

tcb/box tcb/boxSect
tcb/box/title tcb/box/titleCaption

RADBOD UNIVERSITY NIJMEGEN



FACULTY OF SCIENCE

Introducing Generalization in AdjointPDR[↓] for Markov Decision Processes

MSC THESIS SOFTWARE SCIENCE

Author:

Ivo Jacob MELSE

Supervisor:

dr. Sebastian JUNGES

Second reader:

dr. Kevin BATZ

September 14, 2026

Abstract

Markov decision processes (MDPs) are discrete-state models where the transitions between states are probabilistic, and partially determined by a set of nondeterministic choices. Symbolic probabilistic model checking algorithms aim to verify properties on MDPs without representing the underlying models and data structures explicitly, since the state space may be too large in practice. Property Directed Reachability (PDR) has been a major breakthrough in symbolic model checking for the simpler transition systems (TSs). Recently, two ways of extending PDR to the probabilistic setting have been proposed that solve the classic problem of maximal quantitative reachability: **PrIC3** and **AdjointPDR[↓]**. In this master's thesis, we investigate **AdjointPDR[↓]** for MDPs. First, we give a detailed treatment of the algorithm. Second, we discuss how we can symbolically represent underlying data structures in order to support large state spaces. Third, we show that it is possible to adopt *generalization* from **PrIC3** into **AdjointPDR[↓]**. Fourth, we provide an implementation of **AdjointPDR[↓]** with generalization, along with benchmark results. Our results show that **AdjointPDR[↓]** can outperform Storm on simple models with very large state spaces. We conclude that generalization can greatly improve the performance of **AdjointPDR[↓]**, and that **AdjointPDR[↓]** has a number of important advantages over **PrIC3**.

Contents

1	Introduction	3
2	Background	5
2.1	Markov models	5
2.2	Frames and lattices	7
2.2.1	Least fixpoints	9
2.3	Operators on MDPs	10
2.3.1	The Bellman operator	11
2.3.2	The reversed Bellman operator	12
3	AdjointPDR[↓]	15
3.1	The Configuration	15
3.2	The Algorithm	18
3.3	Soundness and Negative Termination	21
3.4	Heuristics in AdjointPDR [↓]	22
3.4.1	Representing lf-sets	22
3.4.2	Investigate	22
3.4.3	Search	22
3.4.4	Refine	25
3.5	Positive (non-)termination	27
4	Symbolic Representation	28
4.1	Probabilistic programs	28
4.2	Frames and lf-sets	31
4.3	Bellman operators	32
4.3.1	The Bellman operator	32
4.3.2	The reversed Bellman operator	33
4.4	Symbolic heuristics	35
5	Generalization	36
5.1	Generalization as a Refine heuristic	37
5.2	Generalization framework	37
5.3	State generalizations	38
5.3.1	Linear generalization	39
5.3.2	Binary search linear generalization	41
5.3.3	Exponential Generalization	42
6	Implementation & Experiments	45
7	Related Work	49
7.1	Value Iteration	49
7.2	PrIC3	50
7.3	Sparse frames	53

8	Conclusions & Future work	55
A	Proofs	57
A.1	Background	57
A.2	AdjointPDR [↓]	61
A.3	Symbolic representation	67
A.4	Generalization	70
B	Computation of $\bigwedge \mathcal{Z}_k$	71
C	Data usage statement	74

Chapter 1

Introduction

Property Directed Reachability (PDR), also known as IC3¹, is a symbolic model checking algorithm for Transition systems (TSs) and safety properties [6][12]. It solves problems such as “given a set of bad states, are they unreachable?”, and works by interleaving the construction of an *inductive invariant* (a *positive chain*) with the construction of a *counterexample* (a *negative sequence*), and using the information in either to reduce the search space for the other.

PDR has been shown to be very effective for TSs, has been extended [21][20], and has been successfully applied to software verification [9][26]. The algorithm mostly owes this success to the fact that (i) instead of unrolling the transition relation, it gradually builds the inductive invariant through refinements, and (ii) it uses an aggressive post-processing step called *generalization* [6][12], which heuristically guesses a state that is more refined than the current state of the algorithm.

Markov decision processes (MDPs) are extensions to TSs that have nondeterministic choices and probabilistic transitions. They are used in a wide range of applications in many domains, including biology, economics, mathematics, computer science and artificial intelligence. An important question in the verification of MDPs can be formulated as follows. “Given a set of bad states and a probability threshold λ , is the probability of reaching a bad state greater than λ , if we choose a strategy that maximizes this probability?” This is known as *quantitative maximal reachability*. Various tools for solving these so-called *model checking* queries on MDPs exist, such as PRISM [25], Storm [19], Modest [16] and EPMC [15]. They mostly use variants of the value iteration algorithm [3][13][17][30] with an underlying representation of the model that can be explicit or symbolic [11].

Given the success of PDR on TSs, there have been two recent adaptations for MDPs. First, there is **PrIC3**, proposed by Batz *et al.* [5] in 2020. Second, there is **AdjointPDR[↓]**, proposed by Kori *et al.* [23] in 2023 which is inspired by their earlier LT-PDR [24] algorithm (which does not work on MDPs). **PrIC3** and **AdjointPDR[↓]** work in a similar way since they are both adaptations of PDR, but they have some important differences which we briefly summarize. **AdjointPDR[↓]** is *sound* “out of the box”, in the sense that if it terminates, the result is correct. In order to be sound in this sense, **PrIC3** requires an extra outer loop which relies on an *oracle*, that in the worst case requires solving the quantitative reachability problem using a different method. Counterexamples in **AdjointPDR[↓]** can be inferred from a data structure called the *negative sequence*, whereas counterexamples in **PrIC3** are represented by sets of states. **AdjointPDR[↓]** has more heuristic choices than **PrIC3**, which allows the construction of different variants of **AdjointPDR[↓]**. Finally, **PrIC3** has a version of *generalization* that greatly boosts its performance, whereas **AdjointPDR[↓]** does not support this yet.

The main contributions of this thesis are (1) providing a direct presentation of **AdjointPDR[↓]** and its heuristics for MDPs, (2) creating *symbolic* versions of the data structures and operations used in the algorithm, (3) adopting *generalization* into **AdjointPDR[↓]**, and (4) providing an implementation of **AdjointPDR[↓]** with *generalization*, which uses symbolic representations of the data structures to solve very large problems. Our main conclusion is that *generalization* in **AdjointPDR[↓]** is possible, and that it can greatly increase the performance on certain problems. The remaining chapters are outlined as follows.

¹IC3 is short for: Incremental Construction of Inductive Clauses for Indubitable Correctness

2. **Background.** We give an overview of the relevant background knowledge. This includes an introduction to Markov models, some lattice theory, *frames*, *lf-sets*, and the operators Φ and Ψ .
3. **AdjointPDR[↓].** We formally define the AdjointPDR[↓] algorithm and its heuristics. We prove its soundness and negative termination, and provide a heuristic for positive termination under additional assumptions.
4. **Symbolic Representation.** We discuss how the data structures are represented symbolically in our implementation of AdjointPDR[↓]. We define semantics for this representation, and prove that this symbolic representation is correct with respect to these semantics.
5. **Generalization.** We discuss how generalization can be introduced into symbolic AdjointPDR[↓] as one of its **Refine** heuristics, and present three methods of implementing generalization.
6. **Implementation & Benchmarks.** We present our implementation of symbolic AdjointPDR[↓] and compare its performance to Kori’s [23] implementation of AdjointPDR[↓], and Storm.
7. **Related Work.** We briefly discuss value iteration and discuss PrIC3 more extensively. In both cases we emphasize the parallels with AdjointPDR[↓].
8. **Conclusion & Future Work.**

Chapter 2

Background

2.1 Markov models

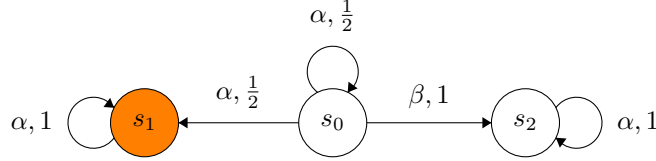


Figure 2.1: MDP with states $S = \{s_0, s_1, s_2\}$ and $\text{Act} = \{\alpha, \beta\}$.

In this thesis, we study Markov models in the form of Markov chains (MCs) and Markov decision processes (MDPs). We mostly follow the presentation of [2], and refer to them for further background and examples. Note that all proofs for this chapter are deferred to Appendix A.1.

Definition 1 (Markov Decision Process). A *Markov decision process* (MDP) $\mathcal{M}$ is a tuple (S, Act, P) , where S is a finite set of states, Act is a finite set of actions, and $P: S \times \text{Act} \times S \rightarrow [0, 1]$ is the *stochastic* transition function. Here *stochastic* means that for all states $s \in S$, there exists a non-empty set of available actions $\text{Av}(s) = \{a \in \text{Act} \mid \sum_{s' \in S} P(s, a, s') = 1\}$, and $\forall a \in \text{Act} \setminus \text{Av}(s). \sum_{s' \in S} P(s, a, s') = 0$. A *Markov chain* $\mathcal{M}$ is an MDP where $|\text{Av}(s)| = 1$ for all $s \in S$. For a Markov chain $\mathcal{M}$, we use the notation $\mathcal{M} = (S, P)$ and $P(s, s') = P(s, a_s, s')$, where a_s is the unique action for s .

Example 1. See the example MDP in Figure 2.1. In state s_0 , if α is chosen, there is $\frac{1}{2}$ probability to stay in s_0 and $\frac{1}{2}$ probability to go to s_1 . If β is chosen, then there is a probability of 1 to go to s_2 . In s_1 and s_2 , the only available action is α , and the model always stays in s_1 and s_2 respectively.

In the rest of this thesis, we fix an MDP $\mathcal{M} = (S, \text{Act}, P)$ with $q = |S| \geq 1$. We commonly refer to the chosen actions in each state as a *policy*. In this thesis, this is a function from the states to the actions. The MDP in Figure 2.1 has two policies: $s_0, s_1, s_2 \mapsto \alpha$ and $s_0 \mapsto \beta; s_1, s_2 \mapsto \alpha$. We are mainly interested in *memoryless deterministic policies*.

Definition 2 (Policy). A *memoryless deterministic policy* (or simply *policy* in this thesis) is a function $\sigma: S \rightarrow \text{Act}$, that chooses an available action for each state, i.e., $\forall s. \sigma(s) \in \text{Av}(s)$. The set of all memoryless deterministic policies for $\mathcal{M}$ is denoted as Π^{MD} . A *history-dependent deterministic policy* is a function $\sigma: S \times \mathbb{N} \rightarrow \text{Act}$ with $\forall s \in S. \forall n \in \mathbb{N}. \sigma(s, n) \in \text{Av}(s)$. The set of all history-dependent deterministic policies is denoted Π^{HD} .

We write $[x_0, x_0, \dots, x_{q-1}]$ for functions $S \rightarrow [0, 1]$ of the form $s_0 \mapsto x_0, s_1 \mapsto x_1, \dots, s_{q-1} \mapsto x_{q-1}$, and $[0, 1]^S = \{F \mid F: S \rightarrow [0, 1]\}$ for the set of all such functions.

We are often interested in knowing the probability of reaching a certain set of states $B \subseteq S$, often referred to as *bad states*, under the policy that maximizes this probability. To this end, we define the *maximal reachability probability* using *finite paths*.

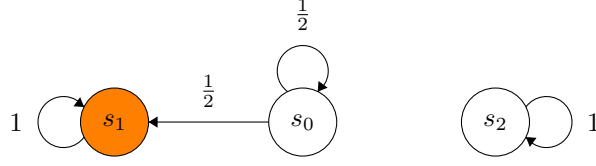


Figure 2.2: Markov chain with $S = \{s_0, s_1, s_2\}$ and $B = \{s_1\}$.

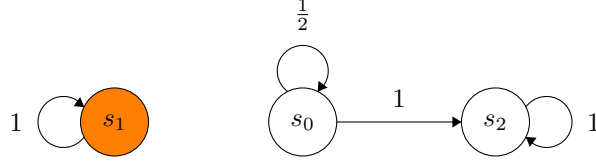


Figure 2.3: Markov chain with $S = \{s_0, s_1, s_2\}$ and $B = \{s_1\}$.

Definition 3 (Paths). Given a history-dependent deterministic policy $\sigma \in \Pi^{\text{HD}}$ and $n \in \mathbb{N}$, a *finite path* in $\mathcal{M}$ is a finite sequence of states $s_0 \dots s_{n-1} \in S^+$, where $P(s_k, \sigma(s_k, k), s_{k+1}) \neq 0$ for $k \in \{0, \dots, n-2\}$. The set of all finite paths of length n for policy σ is denoted as $\text{Path}_{\text{fin}}^n(\mathcal{M}, \sigma)$, and $\text{Path}_{\text{fin}}^{\leq n}(\mathcal{M}, \sigma) = \bigcup_{0 \leq n' \leq n} \text{Path}_{\text{fin}}^{n'}(\mathcal{M}, \sigma)$. Moreover, for a path $s_0 \dots s_{n-1} \in \text{Path}_{\text{fin}}^n(\mathcal{M}, \sigma)$ and a history-dependent policy $\sigma \in \Pi^{\text{HD}}$, the *path probability* $\mathbf{P}_{\sigma}(s_0 \dots s_{n-1}) = \prod_{0 \leq k \leq n-2} P(s_k, \sigma(s_k, k), s_{k+1})$.

Definition 4 (Reachability probability). For $n \in \mathbb{N}$, the *n -step maximum reachability probability* $\mathbb{P}_{\mathcal{M}, \max}^{\leq n}(\diamond B) \in [0, 1]^S$, is the probability that a state $s_0 \in S$ reaches any state $s' \in B$ within n steps under the policy that maximizes this probability,

$$\mathbb{P}_{\mathcal{M}, \max}^{\leq n}(\diamond B)(s_0) = \max_{\sigma \in \Pi^{\text{HD}}} \left\{ \sum_{s_0 \dots s_m \in \text{Path}_{\text{fin}}^{\leq n+1}(\mathcal{M}, \sigma) \cap (S \setminus B)^* B} \mathbf{P}_{\sigma}(s_0 \dots s_m) \right\}.$$

The *eventual reachability probability* $\mathbb{P}_{\mathcal{M}, \max}(\diamond B) \in [0, 1]^S$ is the probability that a state $s_0 \in S$ reaches any state $s' \in B$ *eventually*, i.e. within an unbounded number of steps,

$$\mathbb{P}_{\mathcal{M}, \max}(\diamond B) = \lim_{n \rightarrow \infty} \mathbb{P}_{\mathcal{M}, \max}^{\leq n}(\diamond B).$$

For Markov chains, we may simply write $\mathbb{P}_{\mathcal{M}}^{\leq n}(\diamond B)$ and $\mathbb{P}_{\mathcal{M}}(\diamond B)$.

Example 2. In the Markov chain from Figure 2.2, the probability of reaching s_1 from s_0 within n steps is $1 - (\frac{1}{2})^n$, the probability of reaching s_1 from s_1 is 1 and the probability of reaching s_1 from s_2 is 0. The n -step reachability probability of $\{s_1\}$ is $\mathbb{P}_{\mathcal{M}}^{\leq n}(\diamond \{s_1\}) = [1 - (\frac{1}{2})^n, 1, 0]$. The following table displays $\mathbb{P}_{\mathcal{M}}^{\leq n}(\diamond \{s_1\})$ for increasing n .

n	0	1	2	3	...
$\mathbb{P}_{\mathcal{M}}^{\leq n}(\diamond \{s_1\})$	$[0, 1, 0]$	$[\frac{1}{2}, 1, 0]$	$[\frac{3}{4}, 1, 0]$	$[\frac{7}{8}, 1, 0]$	...

The eventual reachability probability is $\mathbb{P}_{\mathcal{M}}(\diamond \{s_1\}) = [1, 1, 0]$.

Example 3. In the Markov chain from Figure 2.3, the n -step reachability is $\mathbb{P}_{\mathcal{M}}^{\leq n}(\diamond \{s_1\}) = [0, 1, 0]$ for all $n \in \mathbb{N}$, and $\mathbb{P}_{\mathcal{M}}(\diamond \{s_1\}) = [0, 1, 0]$.

When we discuss reachability probabilities in MDPs, it is important to consider the policy. We can see this in Examples 2 and 3. Changing the policy in the MDP from Figure 2.1 results in a different Markov chain which has different reachability probabilities. In this thesis, we are interested in *maximal* reachability

probabilities for MDPs. These are the reachability probabilities where we choose the policy that results in the highest reachability probability.

Example 4. In the MDP from Figure 2.1, the maximal reachability probability $\mathbb{P}_{\mathcal{M},\max}^{\leq n}(\Diamond B) = [1, 1, 0]$. It coincides with the reachability probability of the Markov chain from Figure 2.2.

In practice, we are often not interested in the exact maximal reachability probability. Instead, we would like to verify that the reachability probability is at most a certain bound. This bound is expressed as a function $P: S \rightarrow [0, 1]$, where $P(s)$ represents the bound for state $s \in S$. Furthermore, we usually only care about the reachability probability of B from a single initial state s_0 . The upper bound for s_0 is $\lambda \in [0, 1]$. To reflect this, we fix $s_0 \in S$, $\lambda \in [0, 1]$, and $P = [\lambda, 1_1, \dots, 1_{q-1}]$ (consisting of λ , followed by $q - 1$ ones) throughout the rest of this thesis. We may refer to this P as the *safety property*. This is accompanied by the following problem statement.

Problem statement 1. Decide whether $\mathbb{P}_{\mathcal{M},\max}(\Diamond B) \leq P$.

2.2 Frames and lattices

In this section, we give some background on complete lattices and partial orders. We will also define *frames* and *lower sets of frames* as instances of complete lattices. These will be extensively used in $\text{AdjointPDR}^\downarrow$.

Definition 5 (Partial order). A *partial order* or *poset* is a tuple $(X, \sqsubseteq)$, where X is a set and $\sqsubseteq$ is a relation on X . This relation satisfies the following properties:

1. Reflexivity. $\forall x \in X. x \sqsubseteq x$.
2. Transitivity. $\forall x, y, z \in X. (x \sqsubseteq y \wedge y \sqsubseteq z \implies x \sqsubseteq z)$
3. Anti-symmetry. $\forall x, y \in X. (x \sqsubseteq y \wedge y \sqsubseteq x \implies x = y)$

When reasoning about MDPs, we often use functions of type $S \rightarrow [0, 1]$. We refer to such functions as *frames*. We already saw some examples of frames in the previous section.

Definition 6 (Frame). A function $F: S \rightarrow [0, 1]$ is called a *frame*. We define an order $\leq$ on frames by $F \leq G \iff \forall s \in S. F(s) \leq G(s)$ for $F, G \in [0, 1]^S$, i.e. pointwise comparison. We denote the frames that consist of only ones or zeroes as $\mathbf{1}$ and $\mathbf{0}$, respectively. The set of all frames is denoted $[0, 1]^S$.

Proposition 1. The order $\leq$ on frames is a partial order, i.e. it is reflexive, transitive and anti-symmetric.

For a frame $F \in [0, 1]^S$ and $s \in S$, we may write $F_s = F(s)$, and for $s_n \in S$ we may also write $F_n = F(s_n)$. This will be clear from context. Furthermore, we define two kinds of products on frames: the *dot product* and the *Hadamard product*, along with scalar multiplication and the sum. Finally, we define the *pointwise supremum* and *pointwise infimum*.

Definition 7. Given frames $F, G \in [0, 1]^S$,

1. the *dot product* $(\cdot): [0, 1]^S \times [0, 1]^S \rightarrow \mathbb{R}$ is defined as $F \cdot G = \sum_{s \in S} F_s \cdot G_s$,
2. the *Hadamard product* $(\odot): [0, 1]^S \times [0, 1]^S \rightarrow [0, 1]^S$ is defined as $(F \odot G)(s) = F_s \cdot G_s$,
3. *scalar multiplication* $(\cdot): \mathbb{R} \times [0, 1]^S \rightarrow [0, 1]^S$ is defined as $(r \cdot F)(s) = r \cdot F(s)$, and
4. the *sum* $(+): [0, 1]^S \rightarrow [0, 1]^S$ is defined as $(F + G)(s) = F_s + G_s$.

Example 5. $[0, 1, \frac{1}{2}] \leq [0, 1, \frac{3}{4}]$, but $[0, 1, \frac{3}{4}] \not\leq [0, 1, \frac{1}{2}]$. Furthermore, $[0, 1, \frac{1}{2}] \cdot [0, 1, \frac{3}{4}] = 0 + 1 + \frac{3}{8} = 1\frac{3}{8}$ and $[0, 1, \frac{1}{2}] \odot [0, 1, \frac{3}{4}] = [0, 1, \frac{3}{8}]$.

Definition 8. Let $\mathcal{F} \subseteq [0, 1]^S$ be a set of frames.

1. The *pointwise supremum* $\bigvee \mathcal{F}$, is given by $\bigvee \mathcal{F}(s) = \sup_{F \in \mathcal{F}} F(s)$ for $s \in S$, and $\bigvee \emptyset = \mathbf{0}$. For $F, G \in [0, 1]^S$, we may denote $F \vee G = \bigvee \{F, G\}$.
2. The *pointwise infimum* $\bigwedge \mathcal{F}$, is given by $\bigwedge \mathcal{F}(s) = \inf_{F \in \mathcal{F}} F(s)$ for $s \in S$, and $\bigwedge \emptyset = \mathbf{1}$. For $F, G \in [0, 1]^S$, we may denote $F \wedge G = \bigwedge \{F, G\}$.

Example 6. $[\frac{1}{2}, \frac{1}{3}, 1] \vee [1, 0, \frac{4}{5}] = \bigvee \{[\frac{1}{2}, \frac{1}{3}, 1], [1, 0, \frac{4}{5}]\} = [1, \frac{1}{3}, 1]$. Furthermore, $[\frac{1}{2}, \frac{1}{3}, 1] \wedge [1, 0, \frac{4}{5}] = \bigwedge \{[\frac{1}{2}, \frac{1}{3}, 1], [1, 0, \frac{4}{5}]\} = [\frac{1}{2}, 0, \frac{4}{5}]$.

Apart from frames themselves, the **AdjointPDR**[↓] algorithm also requires us to reason about specific types of sets of frames, called *lower sets of frames* or *lf-sets*. The idea for using lower sets in MDP model checking originates from Kori *et al.*[23].

Definition 9 (Lf-set). A *lower set of frames*, or *lf-set* $\mathcal{F} \subseteq [0, 1]^S$ is a set of frames where, for all $F, G \in [0, 1]^S$, if $F \in \mathcal{F}$ and $G \leq F$, then $G \in \mathcal{F}$. The subset inclusion $\subseteq$ is an order on lf-sets. The set of all lf-sets is denoted $([0, 1]^S)^\downarrow$.

Proposition 2. The order $\subseteq$ on lf-sets is a partial order, i.e. it is reflexive, transitive and anti-symmetric.

Example 7.

1. The empty set $\emptyset$ and the singleton set $\{[0, 0, 0]\}$ are the only finite lf-sets.
2. The set $A = \{[0, 0, 1], [0, 0, \frac{1}{2}]\}$ is *not* an lf-set, because it does not include e.g. $[0, 0, \frac{1}{3}]$, even though $[0, 0, \frac{1}{3}] \leq [0, 0, 1]$ and $[0, 0, 1] \in A$.
3. The set $\{F \in [0, 1]^S : F \leq [0, 0, 1]\}$ is an lf-set.
4. The set $\{F \in [0, 1]^S : F \cdot [0, 0, 1] \leq \frac{1}{2}\}$ is an lf-set. We will see many frame sets of the form $\{F \in [0, 1]^S : F \cdot G \leq r\}$ with $G \in [0, 1]^S$ and $r \in [0, 1]$. We often simply write $\{F : F \cdot G \leq r\}$.

The notions of least upper bound and greatest lower bound are central in lattice theory and model checking.

Definition 10 (Supremum). Let a partial order $(X, \sqsubseteq)$. The *supremum* or *least upper bound* of a subset $Y \subseteq X$ is an element $(\bigsqcup Y) \in X$ that:

1. is an upper bound of Y . $\forall y \in Y. y \sqsubseteq \bigsqcup Y$, and
2. is below all upper bounds of Y . $\forall z \in X. (\forall y \in Y. y \sqsubseteq z) \implies \bigsqcup Y \sqsubseteq z$.

For elements $x, y \in X$, we may write $x \sqcup y$ instead of $\bigsqcup \{x, y\}$. This is often referred to as the *join* of x and y .

Proposition 3. The supremum on $([0, 1]^S, \leq)$ is given by the pointwise supremum $\bigvee$, and the supremum on $(([0, 1]^S)^\downarrow, \subseteq)$ is given by the set union $\bigcup$.

Example 8. Let the set $Y = \{[0, 0, 0], [0, \frac{4}{5}, 0], [1, \frac{1}{3}, 1]\}$. The frame $[1, \frac{4}{5}, 1]$ is the supremum/least upper bound of the set Y , i.e. $\bigvee Y = [1, \frac{4}{5}, 1]$, because:

1. All frames in Y are below $[1, \frac{4}{5}, 1]$: $\forall F \in Y. F \leq [1, \frac{4}{5}, 1]$.
2. $[1, \frac{4}{5}, 1]$ is below all upper bounds of Y : $\forall z \in X. (\forall y \in Y. y \leq z) \implies [1, \frac{4}{5}, 1] \leq z$.

The greatest lower bound or infimum can be thought of as the opposite of supremum.

Definition 11 (Infimum). Let a partial order $(X, \sqsubseteq)$. The *infimum* or *greatest lower bound* of a subset $Y \subseteq X$ is an element $(\bigsqcap Y) \in X$ that:

1. is a lower bound of Y . $\forall y \in Y. \bigsqcap Y \sqsubseteq y$, and
2. is above all lower bounds of Y . $\forall z \in X. (\forall y \in Y. z \sqsubseteq y) \implies z \sqsubseteq \bigsqcap Y$.

For elements $x, y \in X$, we may write $x \sqcap y$ instead of $\bigsqcap \{x, y\}$. This is often referred to as the *meet* of x and y .

Proposition 4. The infimum on $([0, 1]^S, \leq)$ is given by the pointwise infimum $\bigwedge$, and the infimum on $(([0, 1]^S)^\downarrow, \subseteq)$ is given by the set intersection $\bigcap$ (with $\bigcap \emptyset = ([0, 1]^S)^\downarrow$).

Example 9. Let $Y = \{[0, 0, 0], [0, \frac{4}{5}, 0], [1, \frac{1}{3}, 1]\}$. The frame $[0, 0, 0]$ is the infimum/greatest lower bound of the set Y , i.e. $\bigwedge Y = [0, 0, 0]$, because:

1. $[0, 0, 0]$ is below all frames in Y : $\forall F \in Y. [0, 0, 0] \leq F$. It is a lower bound of Y .
2. All lower bounds of Y are below $[0, 0, 0]$: $\forall z \in X. ((\forall y \in Y. z \leq y) \implies z \leq [0, 0, 0])$. (In this case, $[0, 0, 0]$ is the only lower bound of Y .)

The map $(-)^{\downarrow}$ converts a frame to an lf-set containing only frames that are below it in the partial order.

Definition 12. $(F)^{\downarrow} = \{G : G \leq F\}$.

Example 10. $([0, 0, 1])^{\downarrow} = \{F : F \leq [0, 0, 1]\}$.

A complete lattice is a poset that satisfies an extra property. Some of the most important theorems in model checking can be phrased more generally in terms of complete lattices. Both frames and lf-sets form complete lattices.

Definition 13. A poset $(X, \sqsubseteq)$ is a *complete lattice*, if for every subset $Y \subseteq X$, both the supremum $\bigsqcup Y$ and the infimum $\bigsqcap Y$ exist. In particular, each complete lattice has a bottom element $\perp = \bigsqcup \emptyset$ and a top element $\top = \bigsqcap \emptyset$.

Proposition 5.

1. $([0, 1]^S, \leq)$ forms a complete lattice with $\perp = \bigvee \emptyset = \mathbf{0}$ and $\top = \bigwedge \emptyset = \mathbf{1}$.
2. $(([0, 1]^S)^{\downarrow}, \subseteq)$ forms a complete lattice with $\perp = \bigcup \emptyset = \emptyset$ and $\top = \bigcap \emptyset = \mathbf{1}^{\downarrow}$.

A sequence of (non-strictly) increasing elements is called a *chain*. Usually, chains are infinite by definition, but we consider finite chains as well.

Definition 14 (Chain). A sequence S_i of elements of X where $\forall j. S_j \sqsubseteq S_{j+1}$ is called a *chain*, often denoted as:

$$\{S_n\}_n \text{ or } S_0 \sqsubseteq S_1 \sqsubseteq S_2 \sqsubseteq \dots$$

The supremum and infimum of chains are given by the supremum and infimum of the sets that contain every element of the chain. We denote the supremum of $\{S_n\}_n$ as $\bigsqcup_n \{S_n\}$ and the infimum of $\{S_n\}_n$ as $\bigsqcap_n \{S_n\}$.

Example 11. $[0, 0, 0] \leq [\frac{1}{2}, \frac{1}{3}, 1] \leq [1, \frac{1}{3}, 1] \leq [1, 1, 1]$ is a finite chain of length 4. All the sub-chains of this chain are chains as well (of length 1, 2, 3 and 4).

$\{[1 - (\frac{1}{2})^n, 0, 0]\}_n$ is an infinite chain. Here the supremum $\bigvee \{[1 - (\frac{1}{2})^n, 0, 0]\}_n = [1, 0, 0]$.

Example 12 (Frame sets). $\emptyset \subseteq \{[0, 0, 0]\} \subseteq \{F : F \cdot [0, 0, 1] \leq \frac{1}{2}\}$ is a finite chain.

$\{([1 - (\frac{1}{2})^n, 0, 0])^{\downarrow}\}_n$ is an infinite chain. Here the supremum $\bigcup \{([1 - (\frac{1}{2})^n, 0, 0])^{\downarrow}\}_n = [1, 0, 0]^{\downarrow}$.

2.2.1 Least fixpoints

We need to be able to reason about functions on complete lattices. In particular, we need to reason about their *fixpoints*. These are values of the domain where the result of the function equals the input. Note that not all functions have a fixpoint. We will be quite brief as we are mainly interested in the applications of this theory, and not the theory itself. We mostly follow the presentation of [5, Section 2.1], and we refer to them for a more detailed overview of the theory behind fixpoints.

Definition 15 (Fixpoints). Let $(X, \sqsubseteq)$ be a complete lattice, and let $\Phi : X \rightarrow X$.

1. An element $x \in X$ is a *pre-fixpoint* of Φ , if $\Phi(x) \sqsubseteq x$.
2. An element $x \in X$ is a *fixpoint* of Φ , if $\Phi(x) = x$.
3. The (unique) *least fixpoint* is a fixpoint $x \in X$ where for all fixpoints x' , $x \sqsubseteq x'$. We denote the least fixpoint of Φ as $\mu\Phi$.

Example 13. Let the function f on the complete lattice $([0, 10], \leq)$, where

$$f(x) = \begin{cases} 5 & \text{if } x = 5 \\ 10 & \text{if } x = 10 \\ x + 1 & \text{otherwise.} \end{cases}$$

The fixpoints of f are 5 and 10. The least fixpoint $\mu f = 5$.

Monotone functions can be thought of as functions that are "non-decreasing".

Definition 16. Let $(X, \sqsubseteq)$ be a complete lattice. $\Phi: X \rightarrow X$ is *monotone* if for $x, y \in X$,

$$x \sqsubseteq y \Rightarrow \Phi(x) \sqsubseteq \Phi(y).$$

Tarski's fixpoint Theorem allows us to express the least fixed point of a monotone function as the infimum of the set of pre-fixpoints [31].

Theorem 1 (Tarski's Fixpoint Theorem, Theorem 2.1 in [4]). Let $(X, \sqsubseteq)$ be a complete lattice. Every monotone function $\Phi: X \rightarrow X$ has a least fixpoint given by

$$\mu\Phi = \bigcap \{x \in X : \Phi(x) \sqsubseteq x\}.$$

As a direct consequence of Tarski's fixpoint Theorem, we obtain a useful Lemma called Park Induction [29]. It allows us to reason about the fixpoint of a monotone function without knowing exactly what the least fixpoint is.

Lemma 1 (Park Induction, Lemma 2.4 in [4]). Let $(X, \sqsubseteq)$ be a complete lattice and let $\Phi: X \rightarrow X$ be monotone. Then

$$\forall x \in X. \Phi(x) \sqsubseteq x \implies \mu\Phi \sqsubseteq x.$$

Scott-continuous functions are functions that preserve the suprema for directed sets. Such functions are also monotone.

Definition 17. Given a complete lattice $(X, \sqsubseteq)$, a subset $Y \subseteq X$ is *directed* if it is non-empty, and

$$\text{for each } x, y \in Y, \exists z. x, y \sqsubseteq z \text{ and } z \in Y.$$

Definition 18. Let $(X, \sqsubseteq)$ be a complete lattice and let $\Phi: X \rightarrow X$. We say that Φ is *Scott-continuous*, if for all directed subsets $Y \subseteq X$, we have $\Phi(\bigsqcup Y) = \bigsqcup \{\Phi(y) : y \in Y\}$.

Proposition 6. Let $(X, \sqsubseteq)$ be a complete lattice and let $\Phi: X \rightarrow X$. If Φ is Scott-continuous, then it is monotone.

Kleene's Theorem allows us to express the least fixpoint as the supremum of the chain that is obtained by repeatedly applying a continuous function [33].

Theorem 2 (Kleene's Theorem, Theorem 2.1.19 in [1]). Let $(X, \sqsubseteq)$ be a complete lattice, and let $\Phi: X \rightarrow X$. If Φ is Scott-continuous, then the least fixpoint of Φ exists and is given by

$$\mu\Phi = \bigsqcup \{\Phi^n(\perp)\}_n.$$

2.3 Operators on MDPs

In the previous section, we introduced theory about complete lattices and fixpoints. In this section, we will show how this theory can be applied to probabilistic model checking. We introduce two operators on MDPs. Firstly, we introduce the *Bellman operator for maximal reachability probability*. We then use the Bellman operator to formulate a way of proving that the safety property is satisfied. Secondly, we introduce the *reversed Bellman operator for maximal reachability probability*. This operator provides a proof principle to show that the safety property is *not* satisfied.

2.3.1 The Bellman operator

Recall that we fixed an MDP $\mathcal{M} = (S, \text{Act}, P)$ and a set of bad states $B \subseteq S$. We extend the definition of B to frames by having $B \in [0, 1]^S$ where $B(s) = 1$ whenever $s \in B$ and $B(s) = 0$ otherwise. It will be clear from context whether we are referring to a set or to a frame. Intuitively speaking, the Bellman operator works from B to determine the maximal probability of reaching a bad state. For example, $\Phi(B)$ is the probability of reaching B within one step, and $\Phi(\Phi(B))$ is the probability of reaching B within two steps.

Definition 19. In this thesis, the *Bellman operator for maximal reachability* or simply *Bellman operator* $\Phi: [0, 1]^S \rightarrow [0, 1]^S$ is defined as

$$\Phi(F)(s) = \begin{cases} 1, & \text{if } s \in B \\ \max_{a \in \text{Act}} \{ \sum_{s' \in S} F(s') \cdot P(s, a, s') \}, & \text{otherwise.} \end{cases}$$

For a specific policy $\sigma \in \Pi^{\text{MD}}$, we define

$$\Phi_\sigma(F) = \begin{cases} 1, & \text{if } s \in B \\ \sum_{s' \in S} F(s') \cdot P(s, \sigma(s), s'), & \text{otherwise.} \end{cases}$$

Note that the following standard results about Φ also apply to Φ_σ .

Proposition 7.

1. Φ is monotone and Scott-continuous w.r.t. $([0, 1]^S, \leq)$ [17].
2. The least fixpoint of Φ exists, and is denoted $\mu\Phi$.
3. For all $n \in \mathbb{N}$, $\Phi^n(\mathbf{0}) \leq \Phi^{n+1}(\mathbf{0})$.

The chain that is obtained by repeated applications of Φ , starting at $\mathbf{0}$, is called the initial chain.

Definition 20. The *initial chain* is defined as $\{\Phi^n(\mathbf{0})\}_n$, or $\mathbf{0} \leq \Phi(\mathbf{0}) \leq \Phi^2(\mathbf{0}) \leq \dots$

The following Proposition explains why we are interested in the initial chain and fixpoints of the Bellman operator. Namely, the least fixpoint of Φ and the least upper bound of the initial chain are both equal to the maximal reachability probability. This fixed-point characterization is well-known in the literature, see for example [2, Theorem 10.5].

Proposition 8.

1. $\Phi^{n+1}(\mathbf{0}) = \mathbb{P}_{\mathcal{M}, \max}^{\leq n}(\Diamond B)$.
2. $\mu\Phi = \bigvee \{\Phi^n(\mathbf{0})\}_n = \mathbb{P}_{\mathcal{M}, \max}(\Diamond B)$.

Example 14 (Bellman operator). For the MDP in Figure 2.1, the Bellman operator for maximal reachability with $B = \{s_1\}$ is $\Phi([x_0, x_1, x_2]) = [\frac{x_0 + x_1}{2}, 1, 0]$. The fixpoints are $[1, 1, 1]$ and $\mu\Phi = [1, 1, 0]$, the latter of which is the least fixpoint. The initial chain is $\{[1 - (\frac{1}{2})^n, 1, 0]\}_n$:

$$[0, 0, 0] \leq [0, 1, 0] \leq [\frac{1}{2}, 1, 0] \leq [\frac{3}{4}, 1, 0] \leq \dots$$

Here $\mu\Phi = \bigvee \{\Phi^n(\mathbf{0})\}_n = \mathbb{P}_{\mathcal{M}, \max}(\Diamond B) = [1, 1, 0]$.

We can apply Park induction (Lemma 1) to our Bellman operator. This provides us with a way to prove that the reachability probability is below the safety property P , without knowing the exact reachability probability. We obtain the following proof principle that is extensively used in both $\text{AdjointPDR}^\downarrow$ and PrIC3 .

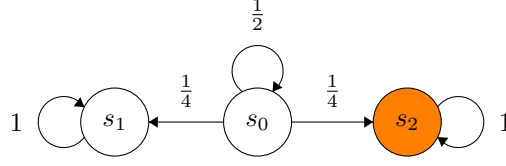


Figure 2.4: Markov chain with $S = \{s_0, s_1, s_2\}$ and $B = \{s_2\}$.

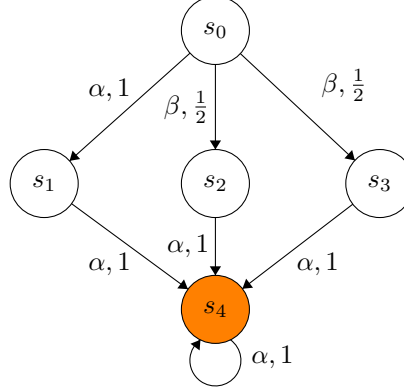


Figure 2.5: MDP with $S = \{s_0, s_1, s_2, s_3, s_4\}$ and $B = \{s_4\}$.

Proof Principle 1 (Applied park induction). Find an $F \in [0, 1]^S$, such that:

1. $\Phi(F) \leq F$, and
2. $F \leq P$.

Then it follows that $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B) \leq P$. We call such an F an *inductive invariant*.

Example 15. In the Markov chain of Figure 2.4, $F = [\frac{1}{2}, 0, 1]$ is an inductive invariant for safety property $P = [\frac{2}{3}, 1, 1]$, because:

1. $\Phi([\frac{1}{2}, 0, 1]) \leq [\frac{1}{2}, 0, 1]$, and
2. $[\frac{1}{2}, 0, 1] \leq [\frac{2}{3}, 1, 1]$.

2.3.2 The reversed Bellman operator

We also need an operator that reverses an application of the Bellman operator, called Ψ . Whereas Φ works *backwards* (w.r.t. the direction of the transitions in $\mathcal{M}$), Ψ works *forwards*. This operator is defined on lf-sets. Example 16 gives an intuitive idea for why it is necessary to define Ψ on lf-sets, and not simply frames.

Definition 21. The *reversed Bellman operator for maximal reachability* or *reversed Bellman operator* $\Psi: ([0, 1]^S)^\downarrow \rightarrow ([0, 1]^S)^\downarrow$, is defined as

$$\Psi(\mathcal{F}) = \{F : \Phi(F) \in \mathcal{F}\}.$$

For a specific policy $\sigma \in \Pi^{\text{MD}}$, we define

$$\Psi_\sigma(\mathcal{F}) = \{F : \Phi_\sigma(F) \in \mathcal{F}\}.$$

Example 16 (Reversed Bellman operator and negative chain). In the MDP from Figure 2.5, we have policies $\sigma_\alpha = s \mapsto \alpha$ and $\sigma_\beta = s_0 \mapsto \beta, s_1, s_2, s_3 \mapsto \alpha$. There are multiple frames such that $\Phi(F) = [\frac{1}{2}, 1, 1, 1, 1]$. For example, $[0, \frac{1}{2}, 0, 0, 1]$, $[0, 0, \frac{1}{2}, \frac{1}{2}, 1]$, or $[0, 0, \frac{1}{4}, \frac{3}{4}, 1]$. The value of s_1 has to be at most $\frac{1}{2}$ (policy σ_α), and the sum over s_2 and s_3 has to be at most $\frac{1}{2}$ (policy σ_β).

$$\begin{aligned}\Psi_{\sigma_\alpha}([\tfrac{1}{2}, 1, 1, 1, 1]^\downarrow) &= \{ [x_0, x_1, x_2, x_3, 1] : x_1 \leq \tfrac{1}{2} \} \\ \Psi_{\sigma_\beta}([\tfrac{1}{2}, 1, 1, 1, 1]^\downarrow) &= \{ [x_0, x_1, x_2, x_3, 1] : x_2 + x_3 \leq \tfrac{1}{2} \} \\ \Psi([\tfrac{1}{2}, 1, 1, 1, 1]^\downarrow) &= \Psi_{\sigma_\alpha}([\tfrac{1}{2}, 1, 1, 1, 1]^\downarrow) \cap \Psi_{\sigma_\beta}([\tfrac{1}{2}, 1, 1, 1, 1]^\downarrow)\end{aligned}$$

Intuitively speaking, $\Psi^n(P^\downarrow)$ contains all the frames, where after n steps, we have not violated the safety property P . We are interested in the sequence of repeated applications of Ψ to $P^\downarrow$, note that it is *not* a chain.

Definition 22. The *safe sequence* for safety property P is $\dots, \Psi^2(P^\downarrow), \Psi(P^\downarrow), P^\downarrow$.

Once again, the results about Ψ are also applicable to Ψ_σ .

Proposition 9.

1. Ψ is monotone w.r.t. $(([0, 1]^S)^\downarrow, \subseteq)$.
2. $\Psi(\mathcal{G}) = \bigcap_{\sigma \in \Pi^{\text{MD}}} \Psi_\sigma(\mathcal{G})$.

The relation between Φ and Ψ as described in Proposition 10, is of great interest to us. Applying it yields Proposition 11 and Proof Principle 2, which is used to show that $\mathcal{M}$ *does not* satisfy the safety property without knowing the exact reachability probability.

Proposition 10. For all $n \in \mathbb{N}$:

1. $\Psi^n(\mathcal{G}) = \{F : \Phi^n(F) \in \mathcal{G}\}$.
2. $\Phi^n(F) \in \mathcal{G} \iff F \in \Psi^n(\mathcal{G})$.
3. $\Phi^n(F) \leq G \iff F \in \Psi^n(G^\downarrow)$.

Proposition 11. $\Psi^{n+1}(P^\downarrow) = \emptyset$, if and only if the safety property is violated after n steps, that is

$$\Psi^{n+1}(P^\downarrow) = \emptyset \iff \Phi^{n+1}(\mathbf{0}) \not\leq P \iff \mathbb{P}_{\mathcal{M}, \max}^{\leq n}(\Diamond B) \not\leq P.$$

Proof Principle 2. Find a finite sequence $\mathcal{G}_i$ of lf-sets, such that:

1. $\mathcal{G}_i \supseteq \Psi^i(P^\downarrow)$ for each i , and
2. $\mathcal{G}_j = \emptyset$, where $\mathcal{G}_j$ is the final item in the sequence.

Then it follows that $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B) \not\leq P$. We call such a sequence a *counterexample*.

The usage of this second proof principle is one of the main innovations in **AdjointPDR**[↓] [23], see Example 17. **PrIC3** does not directly use this proof principle, but uses a similar idea in the *obligation queue* [5], see Section 7.2.

Example 17. In the Markov chain of Figure 2.4 with $P = [\frac{1}{3}, 1, 1]$, we have the following sequence $\mathcal{G}_i$ (read from right to left).

$$\begin{array}{cccc}
\mathcal{G}_3 : & \mathcal{G}_2 : & \mathcal{G}_1 : & \mathcal{G}_0 : \\
\emptyset & , \quad \{v : v \cdot [\frac{1}{4}, \frac{3}{8}, \frac{1}{8}] \leq \frac{1}{12}\} & , \quad \{v : v \cdot [\frac{1}{2}, \frac{1}{4}, \frac{1}{4}] \leq \frac{1}{3}\} & , \quad \{v : v \cdot [1, 0, 0] \leq \frac{1}{3}\} \\
\cup & \cup & \cup & \cup \\
\dots & , \quad \Psi^3(P^\downarrow) & , \quad \Psi^2(P^\downarrow) & , \quad \Psi(P^\downarrow) & , \quad P^\downarrow
\end{array}$$

Now from Proof principle 2, it follows that $\mathbb{P}_{\mathcal{M}, \max} \not\leq [\frac{1}{3}, 1, 1]$, which is indeed the case. The sequence $\mathcal{G}_i$ is a counterexample.

Chapter 3

AdjointPDR[↓]

In the previous chapter, we introduced the foundational principles and operators for **AdjointPDR[↓]**. In this chapter, we first introduce the most important data structure of the algorithm, the *configuration*. After this, we introduce the algorithm itself, prove its correctness and negative termination, and present solutions for its heuristic choices. We then discuss an example run of **AdjointPDR[↓]** that does not terminate, along with a possible solution for non-termination. Most results in this chapter can be attributed to Kori *et al.* [23]. We deviate from their presentation. Most notably, we present the **AdjointPDR[↓]** algorithm for MDPs instead of general complete lattices, and we rename the main steps of the algorithm. Note that (nearly) all proofs for this chapter are deferred to Appendix A.2.

3.1 The Configuration

In **AdjointPDR[↓]**, we build an over-approximation of the initial chain of frames (called the *positive chain* F), and an over-approximation of the safe sequence of lf-sets (called the *negative sequence* $\mathcal{G}$). We store these sequences in the *configuration*. In the positive chain, we use an additional bottom element $\perp \notin [0, 1]^S$. This is a convenient way to deal with a potential edge case during one of the steps of the algorithm.

Definition 23. Let $\perp \notin [0, 1]^S$. The definitions of $\leq$, Φ and $\wedge$ are extended from $[0, 1]^S$ to $[0, 1]^S \cup \{\perp\}$ as follows. $\perp \leq F$, $\Phi(\perp) = \mathbf{0}$, and $F \wedge \perp = \perp$ for all frames $F \in [0, 1]^S$. In particular, note that $\perp \neq \mathbf{0}$.

Reference	Equation	Explanation
P0	$F_0 = \perp \wedge F_1 = \mathbf{0}$	-
P1	$1 \leq k \leq n$ and $n \geq 3$	-
P2	$F_j \leq F_{j+1}$	F is a chain.
P3	$F_j \leq P$	All elements up to the penultimate element of F satisfy the safety property.
P4	$\Phi(F_j) \leq F_{j+1}$	F is relatively inductive w.r.t. Φ .
P5	$\Phi^i(\mathbf{0}) \leq F_{i+1}$	F is an over-approximation of the initial chain. (consequence of P0, P4)
N1	$P \in \mathcal{G}_{n-1}$	The property is contained in the last element of $\mathcal{G}$.
N2	$G_l \supseteq \Psi(\mathcal{G}_{l+1})$	$\mathcal{G}$ is relatively inductive w.r.t. Ψ .
N3	$\Psi^{n-1-m}(P^\downarrow) \subseteq \mathcal{G}_m$	$\mathcal{G}$ is an over-approximation of the safe sequence. (consequence of N1, N2)
PN	$F_m \notin \mathcal{G}_m$	If $\mathcal{G}_m$ exists, then F_m is extra-inductive (see Proposition 12).

Table 3.1: For $j \in \{1 \dots n-2\}$, $i \in \{0 \dots n-2\}$, $l \in \{k \dots n-2\}$, $m \in \{k \dots n-1\}$ we have the invariants of **AdjointPDR[↓]**. See Figure 3.1.

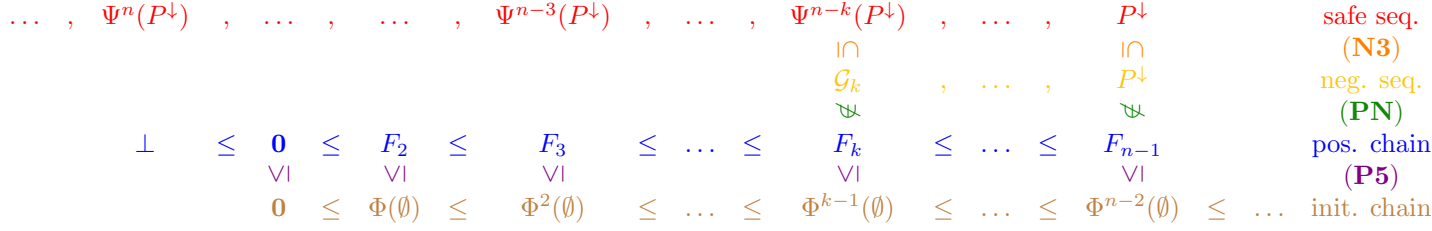


Figure 3.1: Illustration of the relations between the the **initial chain**, **positive chain**, **negative sequence** and **safe sequence**. The **initial chain** is infinite, and the **positive chain** F its finite over-approximation (P5). Similarly, the **safe sequence** is infinite, and the **negative sequence** $\mathcal{G}$ is its finite over-approximation (N3). An element of the **positive chain** F is never included in its corresponding element of the **negative sequence** $\mathcal{G}$ (PN).

Definition 24 (Configuration). A *configuration* is a tuple $(F\|\mathcal{G})_{n,k}$ where F is a sequence of length n consisting of $F_0 = \perp$, followed by frames. $\mathcal{G}$ is a sequence of lower sets of frames of length $n - k$. They satisfy the invariants in Table 3.1, and are indexed as follows:

$$(F\|\mathcal{G})_{n,k} = \begin{pmatrix} F_0 F_1 \dots F_{k-1} F_k, & F_{k+1} \dots F_{n-2} F_{n-1} \\ \mathcal{G}_k, & \mathcal{G}_{k+1} \dots \mathcal{G}_{n-2} \mathcal{G}_{n-1} \end{pmatrix}$$

A configuration is called *positively conclusive* if $F_j = F_{j+1}$ for some j , and *negatively conclusive* if $\mathcal{G}_k = \emptyset$. Note that empty sequences are denoted as ϵ . The set of all configurations is denoted $\mathbf{Conf}$.

Furthermore, we introduce some auxiliary terminology that is useful for discussing the steps of $\mathbf{AdjointPDR}^\downarrow$, in the form of *extra-inductiveness* and *root causes*.

Definition 25. A frame $F \in [0, 1]^S$ is *extra-inductive* (in n steps) if there exists some n , such that $\Phi^n(F) \not\leq P$.

Proposition 12 clarifies the relation between the positive chain and the negative sequence (PN). Element $\mathcal{G}_m$ from the negative sequence may be thought of as *evidence that F_m from the positive sequence is extra-inductive*.

Proposition 12. If $\mathcal{G}_m$ (with $F_m \notin \mathcal{G}_m$) exists in the configuration (i.e. $k \leq m \leq n - 1$), then F_m is extra-inductive.

Proof. If $\mathcal{G}_m$ exists, then $F_m \notin \mathcal{G}_m \supseteq \Psi^{n-m-1}(P^\downarrow)$. Then from Proposition 10(3), $\Phi(F_m)^{n-m-1} \not\leq P$. $\square$

If F_j is the first frame in the positive chain that is extra-inductive, then by P2, so are all frames after F_j . The algorithm continually attempts to refine the positive chain so that there are no extra-inductive frames, and it needs to refine F_j in order to make this happen. We refer to this F_j as the *root cause* of extra-inductiveness.

Definition 26. For a configuration, the *root cause* F_j of extra-inductiveness is the first extra-inductive element of the positive chain.

Finally, the algorithm always begins in the *initial configuration*.

Definition 27. The *initial configuration* is $c_0 = \begin{pmatrix} \perp & 0 & 1 \\ \epsilon & & \end{pmatrix}_{3,3}$.

Lemma 2. The initial configuration satisfies the invariants from Table 3.1.

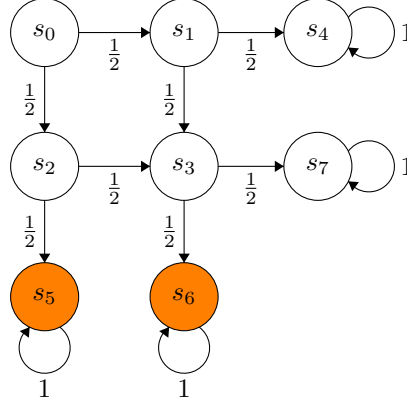


Figure 3.2: Markov chain with $S = \{s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7\}$ and $B = \{s_5, s_6\}$.

Example 18. We give some example configurations for the Markov chain in Figure 3.2. We start with the initial configuration in Table 3.2. Any run of $\text{AdjointPDR}^\downarrow$ starts with this configuration (modulo the size of the state space).

	F	$\mathcal{G}$
0	$\perp$	—
1	$[0, 0, 0, 0, 0, 0, 0, 0]$	—
2	$[1, 1, 1, 1, 1, 1, 1, 1]$	—

Table 3.2: Example configuration with $n = 3, k = 3$.

Now we show a *positively conclusive* configuration in Table 3.3. Here, $F_4 = F_5 = [\frac{7}{10}, \frac{1}{4}, \frac{3}{4}, \frac{1}{2}, 0, 1, 1, 0]$ is an inductive invariant for $\lambda = \frac{7}{10}$. Indeed $\Phi(F_4) \leq F_4$, and $F_4 \leq [\frac{7}{10}, 1, 1, 1, 1, 1, 1, 1]$. Then by proof principle 1, this inductive invariant shows that the probability of reaching B from the initial states is at most $\frac{7}{10}$.

	F	$\mathcal{G}$
0	$\perp$	—
1	$[0, 0, 0, 0, 0, 0, 0, 0]$	—
2	$[\frac{7}{10}, 0, 0, 0, 0, 1, 1, 0]$	—
3	$[\frac{7}{10}, 0, \frac{1}{2}, \frac{1}{2}, 0, 1, 1, 0]$	—
4	$[\frac{7}{10}, \frac{1}{4}, \frac{3}{4}, \frac{1}{2}, 0, 1, 1, 0]$	—
5	$[\frac{7}{10}, \frac{1}{4}, \frac{3}{4}, \frac{1}{2}, 0, 1, 1, 0]$	—

Table 3.3: Example configuration with $n = 6, k = 6$.

Now we show a *negatively conclusive* configuration in Table 3.4, where a counterexample is found. In this case, our property is $[\frac{2}{5}, 1, 1, 1, 1, 1, 1, 1]$. Note that $\mathcal{G}_1 = \emptyset$ and that $\mathcal{G}$ is an over-approximation of the safe sequence. Then by proof principle 2, the probability of reaching a bad state from s_0 exceeds $\frac{2}{5}$.

	F	$\mathcal{G}$
0	$\perp$	—
1	$[0, 0, 0, 0, 0, 0, 0, 0]$	$\emptyset$
2	$[\frac{2}{5}, 0, 0, 0, 0, 1, 1, 0]$	$\{v : [0, 0, 0, 0, \frac{1}{4}, 0, \frac{1}{4}, \frac{1}{4}] \cdot v \leq \frac{3}{20}\}$
3	$[\frac{2}{5}, 0, \frac{1}{2}, \frac{1}{2}, 0, 1, 1, 0]$	$\{v : [0, 0, 0, \frac{1}{2}, \frac{1}{4}, \frac{1}{4}, 0, 0] \cdot v \leq \frac{2}{5}\}$
4	$[\frac{2}{5}, \frac{1}{4}, \frac{3}{4}, \frac{1}{2}, 0, 1, 1, 0]$	$\{v : [0, \frac{1}{2}, \frac{1}{2}, 0, 0, 0, 0, 0] \cdot v \leq \frac{2}{5}\}$
5	$[1, 1, 1, 1, 1, 1, 1, 1]$	$\{v : [1, 0, 0, 0, 0, 0, 0, 0] \cdot v \leq \frac{2}{5}\}$

Table 3.4: Example configuration with $n = 6, k = 1$.

3.2 The Algorithm

The algorithm consists of a single repeating loop which on each iteration applies one of 6 steps, depending on the current configuration. At three of these steps, **Investigate**, **Search** and **Refine**, there is a non-deterministic heuristic choice.

Intuitively speaking, the following process is repeated. We extend the positive chain with **1** (**Extend** step). After each extension, we start the *investigation phase* consisting of the **Investigate** step, **Search** steps and **Refine** steps. During this investigation phase, we may find an inductive invariant (Positive conclusion), or find enough evidence to show that an inductive invariant does not exist (Negative conclusion). We might also find neither, in which case we re-apply **Extend** and start a bigger "investigation".

See Algorithm 1. Below, we explain each of the steps in more detail. We will use $\lambda = \frac{7}{10}$ and the configuration from Table 3.5 (on Figure 3.2) as an example to illustrate the **Extend**, **Investigate** and **Refine** steps.

Algorithm 1 AdjointPDR[↓]

Parameters: An MDP $\mathcal{M}$

Result: true if $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B) \leq P$, false otherwise

```

1:  $\begin{pmatrix} F \\ \mathcal{G} \end{pmatrix}_{n,k} \leftarrow \begin{pmatrix} \perp & 0 & 1 \\ & \epsilon & \end{pmatrix}_{3,3}$ 
2: while true do
3:   if  $\exists j. F_j = F_{j+1}$  then ▷ Positive conclusion
4:     return true
5:   else if  $\mathcal{G}_k = \emptyset$  then ▷ Negative conclusion
6:     return false
7:   else if (a)  $\mathcal{G} = \epsilon$  and (b)  $F_{n-1} \leq P$  then ▷ Extend
8:      $\begin{pmatrix} F_0 \dots F_{n-1} \\ \end{pmatrix}_{n,k} \leftarrow \begin{pmatrix} F_0 \dots F_{n-1} & 1 \end{pmatrix}_{n+1, n+1}$ 
9:   else if (a)  $\mathcal{G} = \epsilon$  and (b)  $F_{n-1} \not\leq P$  then ▷ Investigate
10:     $\mathcal{Z} \leftarrow$  such that (1)  $F_{n-1} \notin \mathcal{Z}$  and (2)  $P \in \mathcal{Z}$  ▷ In
11:     $\begin{pmatrix} F_0 \dots F_{n-1} \\ \end{pmatrix}_{n,k} \leftarrow \begin{pmatrix} F_0 \dots F_{n-1} \\ \mathcal{Z} \end{pmatrix}_{n, n-1}$ 
12:   else if (a)  $\mathcal{G} \neq \epsilon$  and (b)  $\Phi(F_{k-1}) \notin \mathcal{G}_k$  then ▷ Search
13:     $\mathcal{Z} \leftarrow$  such that (1)  $F_{k-1} \notin \mathcal{Z}$  and (2)  $\Psi(\mathcal{G}_k) \subseteq \mathcal{Z}$  ▷ Se
14:     $\begin{pmatrix} F_0 \dots F_{k-1} & F_k \dots F_{n-1} \\ \mathcal{G}_k \dots \mathcal{G}_{n-1} \end{pmatrix}_{n,k} \leftarrow \begin{pmatrix} F_0 \dots F_{k-1} & F_k \dots F_{n-1} \\ \mathcal{Z} & \mathcal{G}_k \dots \mathcal{G}_{n-1} \end{pmatrix}_{n, k-1}$ 
15:   else if (a)  $\mathcal{G} \neq \epsilon$  and (b)  $\Phi(F_{k-1}) \in \mathcal{G}_k$  then ▷ Refine
16:     $z \leftarrow$  such that (1)  $z \in \mathcal{G}_k$  and (2)  $\Phi(F_{k-1} \wedge z) \leq z$  ▷ Re
17:    if  $\Phi(F_k \wedge z) \leq F_k \wedge z$  and  $z \wedge F_k \leq P$  then return true ▷ (Positive conclusion)
18:    else
19:       $\begin{pmatrix} F_0 \dots F_{k-1} & F_k & F_{k+1} \dots F_{n-1} \\ \mathcal{G}_k & \mathcal{G}_{k+1} \dots \mathcal{G}_{n-1} \end{pmatrix}_{n,k} \leftarrow \begin{pmatrix} F_0 \wedge z \dots F_{k-1} \wedge z & F_k \wedge z & F_{k+1} \dots F_{n-1} \\ \mathcal{G}_{k+1} \dots \mathcal{G}_{n-1} \end{pmatrix}_{n, k+1}$ 

```

	F	$\mathcal{G}$
0	$\perp$	-
1	$[0, 0, 0, 0, 0, 0, 0]$	-
2	$[\frac{7}{10}, 0, 0, 1, 0, 0, 1, 0]$	-
3	$[\frac{7}{10}, \frac{1}{2}, 0, 1, \frac{1}{2}, 0, 1, 0]$	-
4	$[\frac{7}{10}, \frac{3}{4}, \frac{1}{4}, 1, \frac{1}{2}, 0, 1, 0]$	-

Table 3.5: Example configuration where the **Extend** step will be applied, with $n = 5, k = 5$.

Extend ($\mathcal{G} = \epsilon$ and $F_{n-1} \leq P$)

Extend (**Unfold** in [23]) is applied when $\mathcal{G}$ is empty, and the last element of F satisfies the property. Intuitively speaking, we have yet to find an inductive invariant or a counterexample, and we want to start a bigger "investigation" to find either. This is accomplished by adding a **1** to the end of the positive chain. An **Extend** step is always immediately followed by a **Investigate** step.

$$\left(F_0 \dots F_{n-1} \right) \text{ becomes } \left(F_0 \dots F_{n-1} \mathbf{1} \right)$$

	F	$\mathcal{G}$
0	$\emptyset$	-
1	$[0, 0, 0, 0, 0, 0, 0, 0]$	-
2	$[\frac{7}{10}, 0, 0, 1, 0, 0, 1, 0]$	-
3	$[\frac{7}{10}, \frac{1}{2}, 0, 1, \frac{1}{2}, 0, 1, 0]$	-
4	$[\frac{7}{10}, \frac{3}{4}, \frac{1}{4}, 1, \frac{1}{2}, 0, 1, 0]$	-
5	$[1, 1, 1, 1, 1, 1, 1, 1]$	-

Table 3.6: Configuration after applying the **Extend** step to Table 3.5, with $n = 6, k = 6$.

Investigate ($\mathcal{G} = \epsilon$ and $F_{n-1} \not\leq P$)

Investigate (**Candidate** in [23]) is applied when $\mathcal{G}$ is empty, and F_{n-1} *does not* satisfy the property P . Intuitively speaking, F_{n-1} is extra-inductive but we do not know where the root cause is. We start an "investigation" to find this root cause by starting a negative sequence using an lf-set $\mathcal{Z}$.

$$\left(F_0 \dots F_{n-1} \right) \text{ becomes } \left(F_0 \dots F_{n-1} \right. \\ \left. \mathcal{Z} \right)$$

For this $\mathcal{Z}$, we require that

1. $F_{n-1} \notin \mathcal{Z}$. F_{n-1} is not included in $\mathcal{Z}$ (as required for **PN**).
2. $P \in \mathcal{Z}$. $\mathcal{Z}$ over-approximates $P^\downarrow$ (as required for **N1**, **N3**).

Together, these ensure that $\mathcal{G}_{n-1}$ is evidence that F_{n-1} is extra-inductive, see Proposition 12.

	F	$\mathcal{G}$
0	$\perp$	-
1	$[0, 0, 0, 0, 0, 0, 0, 0]$	-
2	$[\frac{7}{10}, 0, 0, 1, 0, 0, 1, 0]$	-
3	$[\frac{7}{10}, \frac{1}{2}, 0, 1, \frac{1}{2}, 0, 1, 0]$	-
4	$[\frac{7}{10}, \frac{3}{4}, \frac{1}{4}, 1, \frac{1}{2}, 0, 1, 0]$	-
5	$[1, 1, 1, 1, 1, 1, 1, 1]$	$\{ v : [1, 0, 0, 0, 0, 0, 0, 0] \cdot v \leq \frac{7}{10} \}$

Table 3.7: Example configuration after applying the **Investigate** step to Table 3.6, with $n = 6, k = 5$. $\Phi(F_4) = [\frac{1}{2}, \frac{3}{4}, \frac{1}{4}, 1, \frac{1}{2}, 0, 1, 0] \in \mathcal{G}_5$ and heuristic choice $\mathcal{Z} = \mathcal{G}_5$.

Refine ($\mathcal{G} \neq \emptyset$ and $\Phi(F_{k-1}) \in \mathcal{G}_k$)

Refine (**Conflict** in [23]) is applied when $\mathcal{G}$ is not empty and $\Phi(F_{k-1}) \in \mathcal{G}_k$. Intuitively speaking, F_k is extra-inductive (**PN**), but we have no reason to believe that F_{k-1} is extra-inductive as well ($\Phi(F_{k-1}) \in \mathcal{G}_k$). This means that F_k may be the root cause, so we refine it (and the frames leading up to it) using a frame z .

$$\left(F_0 \dots F_{k-1} \quad F_k \quad F_{k+1} \dots F_{n-1} \right) \text{ becomes } \left(F_0 \wedge z \dots F_{k-1} \wedge z \quad F_k \wedge z \quad F_{k+1} \dots F_{n-1} \right) \\ \left(\mathcal{G}_k \quad \mathcal{G}_{k+1} \dots \mathcal{G}_{n-1} \right) \quad \left(\mathcal{G}_{k+1} \dots \mathcal{G}_{n-1} \right)$$

For this z , we require that

1. $z \in \mathcal{G}_k$. Recall that $\mathcal{G}$ is an over-approximation of the safe sequence. Then $z \in \mathcal{G}_k \supseteq \Psi^{n-k-1}(P^\downarrow)$ may be thought of as "it is *likely* that z satisfies the property after $n-1-k$ applications of Φ ".
2. $\Phi(F_{k-1} \wedge z) \leq z$. This preserves the inductive chain structure of F (as required for **P4**).

	F	$\mathcal{G}$
0	$\perp$	-
1	$[0, 0, 0, 0, 0, 0, 0, 0]$	-
2	$[\frac{7}{10}, 0, 0, 1, 0, 0, 1, 0]$	-
3	$[\frac{7}{10}, \frac{1}{2}, 0, 1, \frac{1}{2}, 0, 1, 0]$	-
4	$[\frac{7}{10}, \frac{3}{4}, \frac{1}{4}, 1, \frac{1}{2}, 0, 1, 0]$	-
5	$[\frac{7}{10}, \frac{3}{4}, \frac{1}{4}, 1, \frac{1}{2}, 0, 1, 0]$	-

Table 3.8: Example configuration after applying the **Refine** step to Table 3.7, with $n=6, k=6$. This configuration is also positively conclusive. Heuristic choice $z = [\frac{7}{10}, \frac{3}{4}, \frac{1}{4}, 1, \frac{1}{2}, 0, 1, 0]$.

Search ($\mathcal{G} \neq \epsilon$ and $\Phi(F_{k-1}) \notin \mathcal{G}_k$)

Search (**Decide** in [23]) is applied when $\mathcal{G}$ is not empty and $\Phi(F_{k-1}) \notin \mathcal{G}_k$. Intuitively, both F_k and F_{k-1} are extra-inductive. We need to "search" for a potential root cause at an earlier index of the positive chain. This is accomplished by expanding the negative sequence using an lf-set $\mathcal{Z}$.

$$\begin{pmatrix} F_0 \dots F_{k-1} & F_k \dots F_{n-1} \\ \mathcal{G}_k \dots \mathcal{G}_{n-1} \end{pmatrix} \text{ becomes } \begin{pmatrix} F_0 \dots F_{k-1} & F_k \dots F_{n-1} \\ \mathcal{Z} & \mathcal{G}_k \dots \mathcal{G}_{n-1} \end{pmatrix}$$

For this $\mathcal{Z}$, we require that

1. $F_{k-1} \notin \mathcal{Z}$. F_{k-1} is not included in $\mathcal{Z}$ (as required for **PN**).
2. $\Psi(\mathcal{G}_k) \subseteq \mathcal{Z}$. $\mathcal{G}$ stays an over-approximation of the safe sequence (as required for **N2**, **N3**).

Similarly to **Investigate**, together (a) and (b) show that F_{k-1} is extra-inductive, see Proposition 12.

Example 19. We take the Markov Chain from Figure 3.2 with $\lambda = \frac{1}{4}$. The configuration of Table 3.9 triggers a decide step and the result is the configuration of Table 3.10. Here the heuristic choice $\mathcal{Z} = \{v : [0, 0, 0, \frac{1}{4}, \frac{1}{2}, \frac{1}{4}, 0, 0] \cdot v \leq \frac{1}{4}\}$.

	F	$\mathcal{G}$
0	$\perp$	-
1	$[0, 0, 0, 0, 0, 0, 0, 0]$	-
2	$[\frac{1}{4}, 0, 0, 1, 0, 0, 1, 0]$	-
3	$[\frac{1}{4}, \frac{1}{2}, 0, 1, \frac{1}{2}, 0, 1, 0]$	-
4	$[\frac{1}{4}, \frac{3}{4}, \frac{1}{4}, 1, \frac{1}{2}, 0, 1, 0]$	$\{v : [0, \frac{1}{2}, \frac{1}{2}, 0, 0, 0, 0, 0] \cdot v \leq \frac{1}{4}\}$
5	$[1, 1, 1, 1, 1, 1, 1, 1]$	$\{v : [1, 0, 0, 0, 0, 0, 0, 0] \cdot v \leq \frac{1}{4}\}$

Table 3.9: Example configuration to which the search step will be applied.
 $\Phi(F_3) = [\frac{1}{4}, \frac{3}{4}, \frac{1}{4}, 1, \frac{1}{2}, 0, 1, 0] \notin \mathcal{G}_4$.

	F	$\mathcal{G}$
0	$\perp$	-
1	$[0, 0, 0, 0, 0, 0, 0, 0]$	-
2	$[\frac{1}{4}, 0, 0, 1, 0, 0, 1, 0]$	-
3	$[\frac{1}{4}, \frac{1}{2}, 0, 1, \frac{1}{2}, 0, 1, 0]$	$\{v : [0, 0, 0, \frac{1}{4}, \frac{1}{2}, \frac{1}{4}, 0, 0] \cdot v \leq \frac{1}{4}\}$
4	$[\frac{1}{4}, \frac{3}{4}, \frac{1}{4}, 1, \frac{1}{2}, 0, 1, 0]$	$\{v : [0, \frac{1}{2}, \frac{1}{2}, 0, 0, 0, 0, 0] \cdot v \leq \frac{1}{4}\}$
5	$[1, 1, 1, 1, 1, 1, 1, 1]$	$\{v : [1, 0, 0, 0, 0, 0, 0, 0] \cdot v \leq \frac{1}{4}\}$

Table 3.10: Resulting configuration after applying the search step to Table 3.9.

Positive conclusion

When j is found such that $F_j = F_{j+1}$, it follows from Proof Principle 1 (along with **P2**, **P3** and **P4**), that $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B) \leq P$. The algorithm returns true. Table 3.3 contains a positively conclusive configuration.

Negative conclusion

When $\mathcal{G}_j = \emptyset$, it follows from Proof Principle 2 (along with **N3**), that $\mathcal{G}$ forms a counterexample. Then, $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B) \not\leq P$, so the algorithm returns false. Table 3.4 contains a negatively conclusive configuration.

3.3 Soundness and Negative Termination

In this section, we discuss the soundness and termination of $\text{AdjointPDR}^\downarrow$. We start by introducing the following notation regarding configurations and steps of the algorithm. The results in this section have already been shown by Kori *et al.* but not written down explicitly for $\text{AdjointPDR}^\downarrow$.

Definition 28. Let c and c' be configurations. If executing one loop iteration of $\text{AdjointPDR}^\downarrow$ on c results in configuration c' , we write $c \rightarrow c'$. We write $c \rightarrow^+ c'$ if $c \rightarrow \dots \rightarrow c'$, and $c \rightarrow^* c'$ if either $c \rightarrow^+ c'$ or $c = c'$.

Definition 29. We use $c \xRightarrow{Ex}, c \xRightarrow{In}, c \xRightarrow{Se}$ and $c \xRightarrow{Re}$ to denote that c triggers a **Extend Investigate**, **Search** or **Refine** step respectively.

$\text{AdjointPDR}^\downarrow$ is *sound* in the sense that if it terminates, the result is correct.

Theorem 3 (Soundness, similar to Theorem 5 in [23]). If $\text{AdjointPDR}^\downarrow$ terminates, then the result is true iff $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B) \leq P$.

As noted before, the $\text{AdjointPDR}^\downarrow$ makes non-deterministic choices at **In**, **Se** and **Re**. We refer to functions that make these choices as *heuristics*. It is, of course, important to prove that for each choice, there is actually a heuristic that satisfies its requirements. We call such a heuristic *legit*.

Definition 30. A heuristic is a function $h: \text{Conf} \rightarrow [0, 1]^S \cup ([0, 1]^S)^\downarrow$. h is *legit*

1. for **In** if (1) $F_{n-1} \notin h(c)$ and (2) $P \in h(c)$ for all c where $c_0 \rightarrow^* c$ and $c \xRightarrow{In}$.
2. for **Se** if (1) $F_{k-1} \notin h(c)$ and (2) $\Psi(\mathcal{G}_k) \subseteq h(c)$ for all c where $c_0 \rightarrow^* c$ and $c \xRightarrow{Se}$.
3. for **Re** if (1) $h(c) \in \mathcal{G}_k$ and (2) $\Phi(F_{k-1} \wedge h(c)) \leq h(c)$ for all c where $c_0 \rightarrow^* c$ and $c \xRightarrow{Re}$.

Corollary 1. There exist legit heuristics for **In**, **Se** and **Re**.

$\text{AdjointPDR}^\downarrow$ never loops (see Proposition 13), but does not always terminate. We can only prove that it terminates in negative cases, when the chosen heuristics are *finite*, i.e. they satisfy the conditions of Definition 31. Importantly, termination can be guaranteed in negative cases because we have heuristics that are both finite and legit, see Corollary 2.

Proposition 13 (Similar to Proposition 7 in [23]). Looping configurations are not possible in $\text{AdjointPDR}^\downarrow$. That is, for configurations c and c' , if $c_0 \rightarrow^* c \rightarrow^+ c'$, then $c \neq c'$.

Definition 31. A set of heuristics H is *finite* if the total amount of unique choices that the heuristics in H make in any run of $\text{AdjointPDR}^\downarrow$ with a finite positive chain, is finite. That is, for any $l \in \mathbb{N}$, the set $\text{choices}(H) = \{Z : h(c) = Z \mid c_0 \rightarrow^* c, n^c = l, c \xRightarrow{\text{step}} \text{ and } h \in H \text{ is a heuristic for step}\}$ is finite.

Theorem 4 (Negative Termination, similar to Corollary 20 in [23]). If $P \not\leq \mathbb{P}_{\mathcal{M}, \max}(\Diamond B)$, all heuristics are legit, and the heuristics for **In** and **Se** from a finite set of heuristics, then $\text{AdjointPDR}^\downarrow$ terminates.

■ **Corollary 2.** There exist heuristics for In and Se that are legit, and form a finite set of heuristics.

3.4 Heuristics in $\text{AdjointPDR}^\downarrow$

In this section, we discuss solutions for the heuristic choices in $\text{AdjointPDR}^\downarrow$. The majority of these have been adapted from Kori *et al.* [23], with the exception of hRm. We will *define* our heuristics using the symbol "=", and *define how they are computed* using "=". The correctness of the definition and computation are given by Proposition 14 and Proposition 15 respectively. Proposition 16 states that heuristics hIn and hSe are finite, which is necessary for negative termination.

■ **Proposition 14.** All heuristics presented in Section 3.4 are *legit* by definition (=).

■ **Proposition 15.** The computations (:=) of all heuristics presented in Section 3.4 are correct.

■ **Proposition 16.** $\{\text{hIn}, \text{hSe}\}$ is a finite set of heuristics.

3.4.1 Representing lf-sets

Lf-sets are represented by a series of inequalities that its members need to satisfy. This allows finite representation of sets with infinite cardinality. For example, for $|S| = 2$ we write

$$\{v : ([\frac{1}{2}, 1] \cdot v \leq 1 \wedge [0, 1] \cdot v \leq \frac{1}{4})\}$$

to represent the set of frames where $(\frac{1}{2}v_0 + 1v_1 \leq 1) \wedge (v_1 \leq \frac{1}{4})$ is satisfied.

Depending on our choices of heuristics, we might end up with an exponential amount of inequalities for the lf-sets in the negative sequence. To prevent this, we present heuristics hIn and hSe that always create an lf-set with only one equation, see Proposition 17. This justifies that throughout the rest of this section, we fix

$$\mathcal{G}_k = \{v : v \cdot w \leq r\} \text{ for some } w \in [0, 1]^S \text{ and } r \in [0, 1].$$

■ **Proposition 17.** Given a configuration $c = (F \parallel \mathcal{G})_{n,k}$ where $\mathcal{G}_k$ can be represented by one equation, $\text{hIn}(c)$ and $\text{hSe}(c)$ can be represented by one equation.

3.4.2 Investigate

Provided that $\mathcal{G} = \epsilon$ and $F_{n-1} \not\leq P$, we need to pick $\mathcal{Z}$ such that $F_{n-1} \notin \mathcal{Z}$ and $P \in \mathcal{Z}$. We propose the heuristic

$$\text{hIn}((F \parallel \mathcal{G})_{n,k}) = P^\downarrow := \{v : [1, 0, \dots, 0] \cdot v \leq \lambda\}.$$

That is, we pick the lf-set that contains exactly all frames F that satisfy the safety property, i.e. $F \leq P$. Since P is of the form $[\lambda, 1, 1, \dots]$, $P^\downarrow$ can always be represented as $\{v : [1, 0, 0, \dots] \cdot v \leq \lambda\}$. For example, if the property is $P = [\frac{1}{3}, 1]$, our heuristic picks $[\frac{1}{3}, 1]^\downarrow = \{v : [1, 0] \cdot v \leq \frac{1}{3}\}$.

3.4.3 Search

Provided that $\mathcal{G} \neq \epsilon$ and $\Phi(F_{k-1}) \notin \mathcal{G}_k$, we need to pick $\mathcal{Z}$ such that $F_{k-1} \notin \mathcal{Z}$ and $\Psi(\mathcal{G}_k) \subseteq \mathcal{Z}$. We present the following two heuristics. In the case of Markov chains, they coincide.

Naive heuristic hSe_Ψ

We may simply use $\Psi(\mathcal{G}_k)$ as a Search heuristic. This means that $\mathcal{G}$ is a prefix of the safe sequence, $\mathcal{G}_m = \Psi^{n-m-1}(P^\downarrow)$ for $m \in \{k \dots n-1\}$.

$$\text{hSe}_\Psi((F \parallel \mathcal{G})_{n,k}) = \Psi(\mathcal{G}_k) = \bigcap_{\sigma \in \Pi^{\text{MD}}} \{\Psi_\sigma(\mathcal{G}_k)\} \quad (\text{for computation, see hSe})$$

When hSe_Ψ is used in $\text{AdjointPDR}^\downarrow$ on MDPs, we may end up with an exponential number of inequalities to represent the elements of the negative sequence. This is problematic in practice.

Concise heuristic hSe

Unlike hSe_Ψ , the result of hSe is an lf-set that can be represented using a single equality. The main insight here is that for the policy σ^* , where $\forall s \in S. \sigma^*(s) = \operatorname{argmax}_{a \in \text{Act}} \{ \sum_{s' \in S} P(s, a, s') \cdot F_{k-1}(s') \}$, $\Psi_{\sigma^*}(\mathcal{G}_k) \supseteq \Psi(\mathcal{G}_k)$ (cf. Proposition 9(2)), but $\Psi_{\sigma^*}(\mathcal{G}_k)$ can be represented by a single inequality, unlike $\Psi(\mathcal{G}_k)$. hSe is defined and computed as follows for $\mathcal{G}_k = \{ v : w \cdot v \leq r \}$.

$$\text{hSe}((F \parallel \mathcal{G})_{n,k}) = \Psi_{\sigma^*}(\mathcal{G}_k) := \{ v : \Theta_{\sigma^*}(u) \cdot v \leq t \}, \text{ where} \quad (3.1)$$

$$\sigma^*(s) = \operatorname{argmax}_{a \in \text{Act}} \{ \sum_{s' \in S} P(s, a, s') \cdot F_{k-1}(s') \} \quad (\text{maximal policy}) \quad (3.2)$$

$$u(s) := \begin{cases} w(s) & \text{if } s \notin B \\ 0 & \text{if } s \in B \end{cases} \quad (3.3)$$

$$t := r - \sum_{s \in B} w(s) \quad (\text{bad state tolerance}) \quad (3.4)$$

$$\Theta_\sigma(F)(s') := \sum_{s \in S} P(s, \sigma(s), s') \cdot F(s) \quad (\text{next step operator}) \quad (3.5)$$

Intuitively speaking, the equation $\{ v : \Theta_\sigma(u) \cdot v \leq t \}$ is obtained by first removing the values of bad states (equation 3.3), and lowering the tolerance for reaching bad states in the future accordingly (equation 3.4). Then we take the next step in the MDP and build $\Psi_{\sigma^*}(\mathcal{G}_k)$ from this (equation 3.1). See Lemma 16 in Appendix A.2 for more details. Examples 20 and 21 show what computing $\Psi_{\sigma^*}(\mathcal{G}_k)$ looks like in practice.

Example 20. Recall the configuration in Table 3.10. We will compute another search step. Recall that $B = \{s_5, s_6\}$. Since it is a Markov chain, the policy is not relevant here.

$$\begin{aligned} \mathcal{G}_k &= \{ v : w \cdot v \leq r \} \\ r &= \frac{1}{4} \\ w &= [0, 0, 0, \frac{1}{4}, \frac{1}{2}, \frac{1}{4}, 0, 0] \\ u &= [0, 0, 0, \frac{1}{4}, \frac{1}{2}, \mathbf{0}, 0, 0] && (\text{because } s_5 \in B) \\ \Theta(u) &= [0, 0, 0, 0, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}] \\ t &= r - \sum_{s \in B} w(s) = \frac{1}{4} - \frac{1}{4} = 0 && (\text{because } u_{s_5} = \frac{1}{4} \text{ and } s_5 \in B, \text{ we have reached } \frac{1}{4} \text{ badness already}) \\ \Psi(\mathcal{G}_k) &= \{ v : v \cdot [0, 0, 0, 0, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}] \leq 0 \} \end{aligned}$$

If we compute another search step, we will see that $t = 0 - (\frac{1}{4} + \frac{1}{4}) = -\frac{1}{2}$, hence

$$\Psi(\Psi(\mathcal{G}_k)) = \{ v : [\dots] \cdot v \leq -\frac{1}{2} \} = \emptyset$$

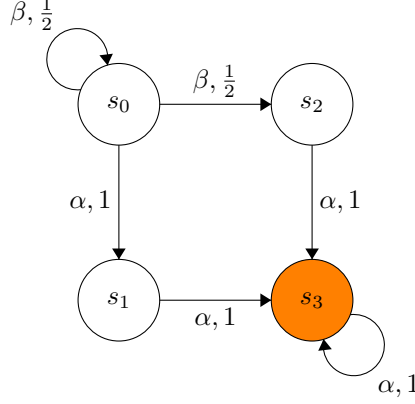


Figure 3.3: MDP with $S = s_0, s_1, s_2, s_3$ and $B = \{s_3\}$.

Example 21. In this example, we show that the policy needs to be taken into consideration for computing hSe. In the MDP from Figure 3.3, we set $\lambda = \frac{1}{2}$. The MDP has two policies: $\sigma_\alpha = s \mapsto \alpha$ and $\sigma_\beta = s_0 \mapsto \beta, s_1, s_2, s_3 \mapsto \alpha$. After 7 steps of running the algorithm, we end up with the configuration in Table 3.11. Now for each F_j , one of the policies σ_α or σ_β is equal to σ^* .

j	F_j	$\mathcal{G}_j$	$\Phi_{\sigma_\alpha}(F_j)$	$\Phi_{\sigma_\beta}(F_j)$	$\Phi(F_j) = \Phi_{\dots}(F_j)$
0	$\perp$	-	$[0, 0, 0, 0]$	$[0, 0, 0, 0]$	σ_α or σ_β
1	$[0, 0, 0, 0]$	-	$[0, 0, 0, 1]$	$[0, 0, 0, 1]$	σ_α or σ_β
2	$[1/2, 0, 0, 1]$	-	$[0, 1, 1, 1]$	$[1/4, 1, 1, 1]$	σ_β
3	$[1/2, 1, 1, 1]$	-	$[1, 1, 1, 1]$	$[3/4, 1, 1, 1]$	σ_α
4	$[1, 1, 1, 1]$	$\{v : [1, 0, 0, 0] \cdot v \leq \frac{1}{2}\}$	$[1, 1, 1, 1]$	$[1, 1, 1, 1]$	σ_α or σ_β

Table 3.11: Configuration after running $\text{AdjointPDR}^\downarrow$ with hIn and hRb after 7 steps. The maximal policy is displayed in the rightmost column. The policies that will be used to compute three subsequent **Search** steps are **highlighted**.

The remaining 3 steps are **Search** steps, after which the algorithm returns false. In these steps, the policies which are highlighted in Table 3.11 are used to compute the next element of the negative sequence in order $j = 3, 2, 1$. In Table 3.12, we see that the result of these **Search** steps is an over-approximation of the safe sequence, recall Figure 3.1.

j	safe sequence	negative sequence $\mathcal{G}_j$	policy used for computing $\mathcal{G}_j$
0	$\perp$	-	-
1	$\emptyset$	$\emptyset$	σ_α or σ_β
2	$\{v : [0, 0, 0, 1] \cdot v \leq \frac{1}{2}\} \cap \{v : [0, 1/2, 0, 1/2] \cdot v \leq \frac{1}{2}\} \cap \{v : [1/4, 0, 1/4, 1/2] \cdot v \leq \frac{1}{2}\}$	$\{v : [0, 0, 0, 1] \leq \frac{1}{2}\}$	σ_β
3	$\{v : [0, 1, 0, 0] \cdot v \leq \frac{1}{2}\} \cap \{v : [1/2, 0, 1/2, 0] \cdot v \leq \frac{1}{2}\}$	$\{v : [0, 1, 0, 0] \leq \frac{1}{2}\}$	σ_α
4	$\{v : [1, 0, 0, 0] \cdot v \leq \frac{1}{2}\}$	$\{v : [1, 0, 0, 0] \cdot v \leq \frac{1}{2}\}$	-

Table 3.12: Configuration obtained by applying three **Search** steps to the configuration in Table 3.11, using heuristic hSe.

3.4.4 Refine

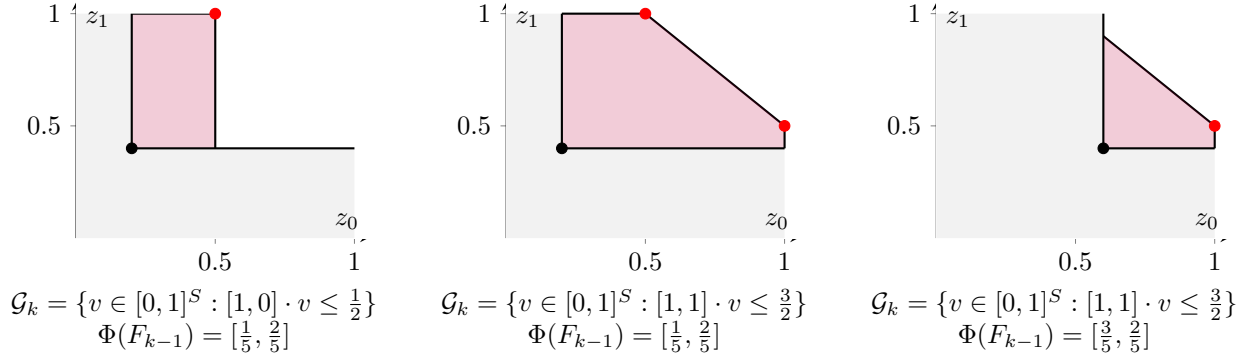


Figure 3.4: Example solution spaces for the **Refine** heuristic with $|S| = 2$, and various values of $\mathcal{G}_k$ and $\Phi(F_{k-1})$. The pink zone represents the points $z \in [0, 1]^S$ where (1) $z \in \mathcal{G}_k$ and (2a) $\Phi(F_{k-1}) \geq z$. The black dot represents $\Phi(F_{k-1})$ and the red dots represent the members of $\mathcal{Z}_k$ for heuristic hRb/hR₀₁.

Provided that $\mathcal{G} \neq \epsilon$ and $\Phi(F_{k-1}) \in \mathcal{G}_k$, we need to pick a z such that (1) $z \in \mathcal{G}_k$ and (2) $\Phi(F_{k-1} \wedge z) \leq z$. In particular, the **Refine** heuristics that we present also satisfy a slightly stronger version of (2), namely (2a) $\Phi(F_{k-1}) \leq z$ (note that $\Phi(F_{k-1}) \leq z \implies \Phi(F_{k-1} \wedge z) \leq z$ by monotonicity). We can interpret picking z such that (1) and (2a) hold, as picking a point in a space, see Example 22. We will now discuss four **Refine** heuristics.

Example 22. In Figure 3.4, we have the possible space of solutions of a **Refine** heuristic, where $|S| = 2$ with different options for $\Phi(F_{k-1})$ and $\mathcal{G}_k$. In each case, the pink zone represents all $z \in [0, 1]^S$ that satisfy (1) and (2a). The gray zone represents values z where $\Phi(F_{k-1}) \not\leq z$. In Table 3.13, we show the values that are chosen by each of the heuristics.

Figure 3.4	left	middle	right
hRs($(F \parallel \mathcal{G})_{n,k}$)	$[1/5, 2/5]$	$[1/5, 2/5]$	$[3/5, 2/5]$
hRm($(F \parallel \mathcal{G})_{n,k}$)	$[1/2, 1/5]$	$[3/4, 3/4]^*$	$[3/4, 3/4]^+$
hRb($(F \parallel \mathcal{G})_{n,k}$)	$[1/2, 1/5]$	$[1/2, 1/2]$	$[1, 1/2]$
hR ₀₁ ($(F \parallel \mathcal{G})_{n,k}$)	$[1/2, 1]$	$[1/2, 1/2]$	$[1, 1/2]$

Table 3.13: Values that are chosen by the **Refine** heuristics, here $\Phi(F_{k-1}) = [\frac{1}{5}, \frac{2}{5}]$.

(*) hRm is not deterministic in this case, it may choose any point on the line between $[\frac{1}{2}, 1]$ and $[1, \frac{1}{2}]$.
 (+) similar, but here it may choose any point on the line between $[\frac{3}{5}, \frac{19}{20}]$ and $[1, \frac{1}{2}]$.

Simple heuristic hRs

hRs is the simplest **Refine** heuristic, and essentially turns **AdjointPDR**[↓] into value iteration (see Section 7.1) with extra steps. In terms of Figure 3.4, hRs always chooses the bottom left corner of the pink polygon. In order for **AdjointPDR**[↓] to be different from value iteration, we need to choose a "better" value. The next **Refine** heuristics each attempt to pick a value that hopefully leads to faster termination than hRs.

$$\text{hRs}((F \parallel \mathcal{G})_{n,k}) = \Phi(F_{k-1}).$$

Maximizing heuristic hRm

In terms of Figure 3.4, heuristic hRm finds a point $u \in [0, 1]^S$ in the pink polygon that is as far away from $\Phi(F_{k-1})$ as possible. We then use this point for $s \in S$ where $w_s \neq 0$, but if $w_s = 0$, we refer back to $\Phi(F_{k-1})$. The specification of u is non-deterministic. In practice, we let an SMT solver resolve this non-determinism.

$$\begin{aligned} \text{hRm}((F\|\mathcal{G})_{n,k})(s) &= \begin{cases} u_s & \text{if } w_s \neq 0 \\ \Phi(F_{k-1})(s) & \text{if } w_s = 0 \end{cases}, \text{ where } u \in [0, 1]^S \text{ satisfies} \\ &\text{maximize } \sum_{w_s \neq 0} u_s, \\ &\text{subject to } \sum_{w_s \neq 0} u_s \cdot w_s \leq r \text{ and } \Phi(F_{k-1}) \leq u. \end{aligned}$$

Meet of generators hRb

In terms of Figure 3.4, heuristic hRb uses the meet/piecewise minimum of a set of relevant corner points of the pink polygon, called $\mathcal{Z}_k$, to come up with a reasonable value for z within the pink polygon.

Definition 32. A frame $F \in [0, 1]^S$ is *almost 0-1-valued* if it consists of only 0 or 1, except for at most one coordinate/state $s \in S$.

Definition 33. Let $w \in [0, 1]^S$, $r \in [0, 1]$ and $\mathcal{G}_k = \{v : w \cdot v \leq r\}$. Then the set of relevant corner points,

$$\mathcal{Z}_k = \{d \in \mathcal{G}_k : d \text{ is almost 0-1-valued} \wedge (\sum_{s \in S} d_s w_s) = r \wedge \Phi(F_{k-1}) \leq d\}.$$

Example 23. In Figure 3.4 (middle), all frames $d \in [0, 1]^S$ on the line between $[\frac{1}{2}, 1]$ and $[1, \frac{1}{2}]$ satisfy $(\sum_{s \in S} d_s w_s) = r \wedge \Phi(F_{k-1}) \leq d$. Of these coordinates, only $[\frac{1}{2}, 1]$ and $[1, \frac{1}{2}]$ are almost 0-1-valued, so $\mathcal{Z}_k = \{[\frac{1}{2}, 1], [1, \frac{1}{2}]\}$. In Figure 3.4 (right), we additionally exclude $[\frac{1}{2}, 1]$ since $\Phi(F_{k-1}) \not\leq [\frac{1}{2}, 1]$, so $\mathcal{Z}_k = \{[1, \frac{1}{2}]\}$.

If $\mathcal{Z}_k$ is non-empty, we use its meet for computing the value of states $s \in S$ where $w_s \neq 0$. Otherwise we fall back to $\Phi(F_{k-1})$.

$$\text{hRb}((F\|\mathcal{G})_{n,k})(s) = \begin{cases} (\bigwedge \mathcal{Z}_k)(s), & \text{if } w_s \neq 0 \text{ and } \mathcal{Z}_k \neq \emptyset \\ \Phi(F_{k-1})(s), & \text{otherwise} \end{cases}$$

Computing $\bigwedge \mathcal{Z}_k$ naively by first enumerating $\mathcal{Z}_k$ is not viable in practice because $|\mathcal{Z}_k|$ may grow exponentially with $|S|$. Kori *et al.* provide an algorithm for efficiently computing $\bigwedge \mathcal{Z}_k$, called **ComputeMeet**, see Appendix B.

We compute $(\mathcal{Z}_k = \emptyset) := b$ and $\bigwedge \mathcal{Z}_k := c$, where $b, c = \mathbf{CM}(\Phi(F_{k-1}), \mathcal{G}_k)$ (see Appendix B).

Binary meet of generators hR01

Here, we take the previous heuristic, but for state $s \in S$, whenever $w_s = 0$ and $\Phi(F_{k-1})(s) \neq 0$, we set z_s to 1 instead of $\Phi(F_{k-1})(s)$.

$$\text{hR01}((F\|\mathcal{G})_{n,k})(s) = \begin{cases} (\bigwedge \mathcal{Z}_k)(s), & \text{if } w_s \neq 0 \text{ and } \mathcal{Z}_k \neq \emptyset \\ \lceil \Phi(F_{k-1})(s) \rceil, & \text{if } w_s = 0 \text{ and } \mathcal{Z}_k \neq \emptyset \\ \Phi(F_{k-1})(s), & \text{if } \mathcal{Z}_k = \emptyset \end{cases}$$

where $\lceil k \rceil = 0$ if $k = 0$ else 1

Example 24. See the column "left" in Table 3.13 (here, $\mathcal{G}_k = \{v \in [0, 1]^S : [1, 0] \cdot v \leq \frac{1}{2}\}$ and $\Phi(F_{k-1}) = [\frac{1}{5}, \frac{2}{5}]$). Then $w_0 = 0$, and $\text{hRb}((F\|\mathcal{G})_{n,k})(s_0) = \Phi(F_{k-1})(s_0) = \frac{1}{5}$. $\text{hR01}((F\|\mathcal{G})_{n,k})(s) = \lceil \Phi(F_{k-1})(s_0) \rceil = 1$.

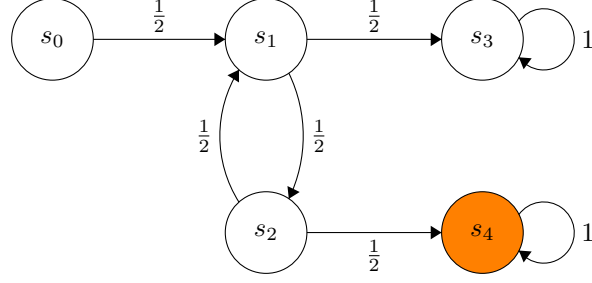


Figure 3.5: A Markov chain with $S = \{s_0, s_1, s_2, s_3, s_4\}$.

3.5 Positive (non-)termination

As already discussed, we can prove negative termination with the right heuristics. Positive termination is not guaranteed with any of the heuristics from Section 3.4. See Example 25.

Example 25. Let us run $\text{AdjointPDR}^\downarrow$ on the Markov chain of Figure 3.5, with heuristics hIn , hSe and hRb , and $\lambda = \frac{3}{4}$. An inductive invariant for this problem is $[\frac{1}{3}, \frac{1}{3}, \frac{2}{3}, 0, 1]$, but $\text{AdjointPDR}^\downarrow$ does not find this and never terminates. This is more or less the same problem that we encounter in value iteration without a proper stopping criterion. The problem is not exclusive to hRs either. Similar things can be observed for all the **Refine** heuristics that we presented. Let $\xRightarrow{3}$ be short for $\xRightarrow{\text{Ex}} \xRightarrow{\text{In}} \xRightarrow{\text{Re}}$.

$\begin{array}{c c} F & \mathcal{G} \\ \hline \perp & - \\ \mathbf{0} & - \\ \mathbf{1} & - \end{array} \xRightarrow{\text{In}} \begin{array}{c c} F & \mathcal{G} \\ \hline \perp & - \\ \mathbf{0} & - \\ \mathbf{1} & \{v : v_0 \leq \frac{1}{2}\} \end{array} \xRightarrow{\text{Re}} \begin{array}{c c} F & \mathcal{G} \\ \hline \perp & - \\ \mathbf{0} & - \\ [0, 0, 0, 0, 1] & - \end{array} \xRightarrow{3} \begin{array}{c c} F & \mathcal{G} \\ \hline \perp & - \\ \mathbf{0} & - \\ [0, 0, 0, 0, 1] & - \\ [0, 0, \frac{1}{2}, 0, 1] & - \end{array}$	
$\xRightarrow{3} \begin{array}{c c} F & \mathcal{G} \\ \hline \perp & - \\ \mathbf{0} & - \\ [0, 0, 0, 1, 0, 0] & - \\ [0, 0, \frac{1}{2}, 0, 1] & - \\ [0, \frac{1}{4}, \frac{1}{2}, 0, 1] & - \end{array} \xRightarrow{3} \begin{array}{c c} F & \mathcal{G} \\ \hline \perp & - \\ \mathbf{0} & - \\ [0, 0, 0, 1, 0, 0] & - \\ [0, 0, \frac{1}{2}, 0, 1] & - \\ [0, \frac{1}{4}, \frac{1}{2}, 0, 1] & - \\ [\frac{1}{4}, \frac{1}{4}, \frac{5}{8}, 0, 1] & - \end{array} \xRightarrow{3} \begin{array}{c c} F & \mathcal{G} \\ \hline \perp & - \\ \mathbf{0} & - \\ [0, 0, 0, 1, 0, 0] & - \\ [0, 0, \frac{1}{2}, 0, 1] & - \\ [0, \frac{1}{4}, \frac{1}{2}, 0, 1] & - \\ [\frac{1}{4}, \frac{1}{4}, \frac{5}{8}, 0, 1] & - \\ [\frac{1}{4}, \frac{5}{16}, \frac{5}{8}, 0, 1] & - \end{array} \xRightarrow{3} \dots$	

A simple way of guaranteeing positive termination in $\text{AdjointPDR}^\downarrow$, is to work under the assumption that the inductive invariant can be represented using only coordinates with a denominator of some maximum size $c \in \mathbb{N}$. This is similar to adding a simple stopping criterion to value iteration to guarantee termination.

Definition 34. Let $x \in \mathbb{R}$. The *closest c -fraction* of x is the fraction $\text{round}_c(x) = \frac{q}{r}$ with $r \leq c$ and $x \leq \text{round}_c(x)$ (where $c, q, r \in \mathbb{N}$), such that $\text{round}_c(x)$ is closest to x . For a frame $F \in [0, 1]^S$, $\text{round}_c(F)(s) = \text{round}_c(F(s))$.

Definition 35. For $c \in \mathbb{N}$, let

$$\text{hRs}_c(F \parallel \mathcal{G})_{n,k} = \begin{cases} z & \text{if } \Phi(z) \leq z \text{ and } z \in \mathcal{G}_k \\ \Phi(F_{k-1}) & \text{otherwise} \end{cases}$$

where $z = \text{round}_c(\Phi(F_{k-1}))$

Proposition 18. hRs_c is legit.

Theorem 5. For $c \in \mathbb{N}$, if $\mu\Phi = \text{round}_c(\mu\Phi)$, then $\text{AdjointPDR}^\downarrow$ with hRs_c terminates.

Chapter 4

Symbolic Representation

```
module chain
  x : [0..N];
  y : [0..1];

  [] x < N & y = 0    ->    (1/10) : (x'=x) & (y'=y+1)
                        +    (9/10) : (x'=x+1) & (y'=y);
  [] x < N & y = 1    ->    1 : (x'=x+1) & (y'=y);
  [] x = N            ->    1 : (x'=x) & (y'=y);
endmodule

label "bad" = y=1;
```

Figure 4.1: Example of a *PRISM* program with a single module.

Any implementation of the $\text{AdjointPDR}^\downarrow$ algorithm needs a way to represent the model, frames and lf-sets. The simplest way to represent frames, is to have explicit array-like data structures that are the size of the state space. In practice however, explicit representation is infeasible for sufficiently large finite state spaces. To overcome this problem, we need a *symbolic* representation of frames and related functionality. This means that we do not explicitly store the probability values for every state. Instead, we use a representation of frames that allows us to store information about many states using relatively little space, while still being able to represent every finite frame.

In this chapter, we discuss our symbolic representation of models, frames and lf-sets in the setting of *probabilistic programs*. We specify the basic operations that we need, and then define equivalents to the operators Φ and Ψ that work with this representation. We also briefly discuss how we can adopt the existing heuristics for $\text{AdjointPDR}^\downarrow$ to work using our symbolic frames and lf-sets. For simplicity, we restrict ourselves to finite deterministic models, i.e. finite Markov chains as opposed to general MDPs. Note that all proofs for this chapter are deferred to Appendix A.3.

4.1 Probabilistic programs

We accept programs in a syntax that is essentially a subset of a *PRISM* [25] programs with a single module, see Example 26.

Example 26. We present example syntax of a supported *PRISM* module in Figure 4.1. This simple program has two variables x and y with their respective ranges. If $x < N$ and $y = 0$, then there is a $\frac{1}{10}$ chance to keep x constant and assign $y + 1$ to y , and a $\frac{9}{10}$ chance to assign $x + 1$ to x and keep y constant. If $x = N$, then x and y stay constant. The label "bad" is used to note which states are "bad" in our reachability query, similar to the set B in previous chapters.

Instead of precisely defining how exactly we should interpret *PRISM* syntax, we will assume that our

programs are given in an AST-like data structure that we will define throughout the rest of this section. We start by fixing a finite ordered set $\mathbf{Vars} = \{x_0, x_1, \dots, x_{v-1}\}$ with $v = |\mathbf{Vars}|$ of program variables. For each variable $x_i \in \mathbf{Vars}$, we have a lower bound and upper bound $l_{x_i}, u_{x_i} \in \mathbb{N}$. Furthermore, the *domain* of the variables is given by

$$\text{dom}(x_i) = \{n \in \mathbb{N} : l_{x_i} \leq n \leq u_{x_i}\}.$$

Now we can have *states* s , functions from variables to values inside the variable domain, i.e. $s(x_i) \in \text{dom}(x_i)$. The finite set of all states is

$$S = \{s \in \mathbf{Vars} \rightarrow \mathbb{N} : s(x_i) \in \text{dom}(x_i), \text{ for all } x_i \in \mathbf{Vars}\}.$$

We write $(n_0 \dots, n_{v-1})$ for a state s with $s(x_0) = n_0, \dots, s(x_{v-1}) = n_{v-1}$.

Example 27. If $\mathbf{Vars} = \{x, y\}$ with $\text{dom}(x) = \{0, 1\}$ and $\text{dom}(y) = \{0, \dots, 2\}$, then

$$S = \{(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)\}.$$

This corresponds to the state space of the syntactic program in Example 26 with $N = 2$. Note that $(0, 1)(x) = 0$ and $(0, 1)(y) = 1$.

We define our probabilistic programs from the ground up. We start by defining the *affine variable update*. This is a special kind of affine function.

Definition 36. An *affine variable update* for x_i , $f_i: S \rightarrow \text{dom}(x_i)$ is a function where

$$f_i(s) = k_0 \cdot s(x_0) + k_1 \cdot s(x_1) + \dots + k_{v-1} \cdot s(x_{v-1}) + k_v \text{ (recall } v = |\mathbf{Vars}|)$$

If $k_i > 0$, then we say that f_i is *invertible* on x_i . If $k_j = 0$ for all $j \in \{0 \dots v-1\}$, then we say that f_i is *constant*.

For example, we identify $\mathbf{x}' = \mathbf{x} + 1$ from the syntactic program in Example 26 with the affine variable update $f(s) = s(x) + 1$.

Definition 37. A *state predicate* is a function $\varphi: S \rightarrow \{0, 1\}$. The set of all state predicates is denoted $\mathbf{Pred}$. We may write $s \models \varphi$ instead of $\varphi(s) = 1$ and $s \not\models \varphi$ instead of $\varphi(s) = 0$. Boolean connectives carry over to state predicates in the usual way. Furthermore, $\text{true}, \text{false} \in \mathbf{Pred}$ such that $\forall s \in S, \text{true}(s) = 1$ and $\text{false}(s) = 0$.

Furthermore, we often use the inversion bracket notation $[B]$ for a state predicate with Boolean expression B in the usual way, see Example 28.

Example 28. Let $\mathbf{Vars} = \{x, y\}$. Then $[x = 5](s) = \begin{cases} 1 & \text{if } s(x) = 5 \\ 0 & \text{otherwise} \end{cases}$.

An *update* is a function that maps the current state to the next, updated, state.

Definition 38. An *update* $u: S \rightarrow S$ is a function

$$u(s)(x_i) = f_i(s) \text{ for all } i \text{ where } 0 \leq i < v,$$

where for all i , f_i is a affine variable update for x_i .

If f_i is invertible on x_i for all i , then we say that u is *invertible* as well. If f_i is constant for all i , then we say that u is *constant* as well. We write $u(*)$ to get the constant result of such updates.

We often use the notation $[x_0 \mapsto E_0; \dots; x_{v-1} \mapsto E_{v-1}]$ for an update u with arithmetic expressions $E_0, \dots, E_{v-1}$ in the usual way, see Example 29. We do not allow all updates that meet Definition 38, only updates that are *valid*. This puts a significant restriction on the accepted programs. The reason for this is related to the Θ operator (see Section 2.3.2).

Definition 39. An update is *valid* if it is either constant or invertible. The set of all valid updates is denoted **Update**.

Example 29. If we recall the syntactic program in Example 26, then we can identify $(x'=x+1) \ \& \ (y'=y)$ with the update function $u(s)(x) = s(x) + 1, u(s)(y) = s(y)$, also written as $[x \mapsto x + 1; y \mapsto y]$. This update function is valid, since it is invertible. Another example of a valid (constant) update function is $u(s)(x) = 1, u(s)(y) = 0$, i.e. $[x \mapsto 1; y \mapsto 0]$. However, the function $u(s)(x) = s(x) + 1, u(s)(y) = 0$, i.e. $[x \mapsto x + 1; y \mapsto 0]$ is *not* a valid update function, because it is partially constant, and partially invertible. Furthermore, the update function $u(s)(x) = 2^{s(x)}, u(s)(y) = 2^{s(y)}$ or $[x \mapsto 2^x; y \mapsto 2^y]$ is not valid because it does not consist of affine variable updates, they are exponential.

A branch is a probability distribution with updates corresponding to each probability.

Definition 40. A *branch* $\mathbf{br} \subseteq [0, 1] \times \mathbf{Update}$ is a set of tuples

$$\mathbf{br} = \{(p_0, u_0), \dots, (p_{v-1}, u_{v-1})\}$$

such that $\sum_{0 \leq i < v} p_i = 1$, and u_i is a valid *update* for $0 \leq i < b$ for some $b \in \mathbb{N}$. The set of all branches is denoted **Branch**.

Then finally, our program is a list of branches with *guards*, state predicates that restrict in what states this branch can be taken.

Definition 41. In this thesis, a *probabilistic program* $A \subseteq \mathbf{Pred} \times \mathbf{Branch}$ is a set of tuples

$$A = \{(\varphi_0, \mathbf{br}_0), \dots, (\varphi_{m-1}, \mathbf{br}_{m-1})\},$$

We refer to φ_i as a *guard*. For each guard φ_i , there is at least one state $s_i \in S$ s.t. $\varphi_i(s_i) = 1$. Any two guards φ_i and φ_j with $i \neq j$ are *disjoint*, meaning that there exists no state $s \in S$ where $\varphi_i(s) = \varphi_j(s) = 1$. The set of all programs is denoted **Prog**.

Example 30. The syntactic program in Example 26 can be represented as a probabilistic program

$$A = \left\{ \left([x < N \wedge y = 0], \left\{ \left(\frac{1}{10}, [x \mapsto x; \quad y \mapsto y + 1] \right), \right. \right. \right. \\ \left. \left. \left(\frac{9}{10}, [x \mapsto x + 1; \quad y \mapsto y] \right) \right\} \right) \right\} \\ \left([x < N \wedge y = 1], \left\{ \left(1, [x \mapsto x + 1; \quad y \mapsto y + 1] \right) \right\} \right) \\ \left([x = N], \left\{ \left(1, [x \mapsto x; \quad y \mapsto y] \right) \right\} \right) \Big\}.$$

Here $\left\{ \left(\frac{1}{10}, [x \mapsto x + 1; y \mapsto y + 1] \right), \left(\frac{9}{10}, [x \mapsto x + 1; y \mapsto y] \right) \right\}$ is an example of a branch.

Finally, we define the semantics of our programs as a Markov chain, and the semantics of predicates as subsets of S .

Definition 42. The semantics of probabilistic program $A \in \mathbf{Prog}$, where

$$A = \{(\varphi_0, \mathbf{br}_0), \dots, (\varphi_{m-1}, \mathbf{br}_{m-1})\} \text{ and} \\ \mathbf{br}_i = \{(p_{i,0}, u_{i,0}), \dots, (p_{i,b_i-1}, u_{i,b_i-1})\} \text{ for } 0 \leq i < m$$

is a Markov chain $\text{sem}(A) = (S, P)$, where

$$P(s, s') = \sum_{0 \leq i < m} \sum_{0 \leq j < b_i} p_{i,j} \cdot [u_{i,j}(s) = s'] \cdot \varphi_i(s)$$

Definition 43. The semantics of predicate $\varphi \in \mathbf{Pred}$ is a set $\text{sem}(\varphi) = \{s \in S : \varphi(s) = 1\} \subseteq S$.

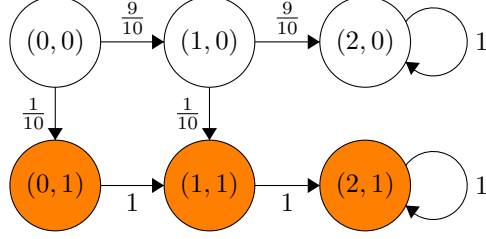


Figure 4.2: Markov chain with $S = \{(0, 0), (1, 0), (2, 0), (0, 1), (1, 1), (2, 1)\}$ and $B = \{(0, 1), (1, 1), (2, 1)\}$.

Example 31. The semantics of the program described in Example 30 with $N = 2$ is the Markov chain in Figure 4.2, and the semantics of $\eta = [y = 1]$ is $B = \{(0, 1), (1, 1), (2, 1)\}$.

name	symbol	usage	signature	specification
partial order	$\leq$	$F \leq G$	$[0, 1]_{\text{sym}}^S \times [0, 1]_{\text{sym}}^S \rightarrow \{0, 1\}$	$(F \leq G) = 1 \iff \text{sem}(F) \leq \text{sem}(G)$
meet	$\wedge$	$F \wedge G$	$[0, 1]_{\text{sym}}^S \times [0, 1]_{\text{sym}}^S \rightarrow [0, 1]_{\text{sym}}^S$	$\text{sem}(F \wedge G) = \text{sem}(F) \wedge \text{sem}(G)$
dot product	$\cdot$	$F \cdot G$	$[0, 1]_{\text{sym}}^S \times [0, 1]_{\text{sym}}^S \rightarrow \mathbb{R}$	$F \cdot G = \text{sem}(F) \cdot \text{sem}(G)$
Hadamard	$\odot$	$F \odot G$	$[0, 1]_{\text{sym}}^S \times [0, 1]_{\text{sym}}^S \rightarrow [0, 1]_{\text{sym}}^S$	$\text{sem}(F \odot G) = \text{sem}(F) \odot \text{sem}(G)$
composition	$\circ$	$F \circ u$	$[0, 1]_{\text{sym}}^S \times \mathbf{Update} \rightarrow [0, 1]_{\text{sym}}^S$	$\text{sem}(F \circ u) = \text{sem}(F) \circ u$
region sum	sum	$\text{sum}(F, \varphi)$	$[0, 1]_{\text{sym}}^S \times \mathbf{Pred} \rightarrow \mathbb{R}$	$\text{sum}(F, \varphi) = \sum_{s \in \text{sem}(\varphi)} (\text{sem}(F))(s)$
Frame sum	$\sum$	$\sum(\mathcal{F})$	$\mathcal{P}([0, 1]_{\text{sym}}^S) \rightarrow [0, 1]_{\text{sym}}^S$	$\text{sem}(\sum(\mathcal{F})) = \sum_{F \in \mathcal{F}} \text{sem}(F)$
Evaluation	$()$	$F(s)$	$[0, 1]_{\text{sym}}^S \times S \rightarrow [0, 1]$	$F(s) = (\text{sem}(F))(s)$

Table 4.1: Required basic operations on $F, G \in [0, 1]_{\text{sym}}^S$, $\mathcal{F} \subseteq [0, 1]_{\text{sym}}^S$ and $u \in \mathbf{Update}$, $s \in S$ and $\varphi \in \mathbf{Pred}$.

4.2 Frames and lf-sets

Recall that frames are functions $S \rightarrow [0, 1]$ that assign a probability value to each state. We can represent any frame as a piecewise affine function. We call such a representation a *symbolic frame*.

Definition 44. A *real-valued affine function* $f : S \rightarrow \mathbb{R}$ is a function where

$$f(s) = k_0 \cdot s(x_0) + \dots + k_{v-1} \cdot s(x_{v-1}) + k_v,$$

where $k_0, k_1, \dots, k_v \in \mathbb{R}$. The set of all real-valued-affine functions is $\mathbf{Raf}$. For $p \in [0, 1]$, we may write $[p] \in \mathbf{Raf}$ where $[p](s) = p$ for all $s \in S$.

Definition 45. A *symbolic frame* F is a set of tuples $\{(f_0, \varphi_0), \dots, (f_{k-1}, \varphi_{k-1})\} \subseteq \mathbf{Raf} \times \mathbf{Pred}$ (for some $k \in \mathbb{N}$), where φ_i and φ_j are disjoint for $i \neq j$, i.e. there is no state s where $\varphi_i(s) = \varphi_j(s) = 1$, and for all i and $s \in S$, $\varphi_i(s) = 1$ implies $f_i(s) \in [0, 1]$. The set of all symbolic frames is $[0, 1]_{\text{sym}}^S$. For a state predicate φ , we may write $\text{frame}(\varphi) = \{([1], \varphi), ([0], \neg\varphi)\}$. We may write $[p] = \{([p], \text{true})\}$ for $p \in [0, 1]$.

Definition 46. The semantics of a symbolic frame $F = \{(f_0, \varphi_0), \dots, (f_{k-1}, \varphi_{k-1})\} \in [0, 1]_{\text{sym}}^S$ is a frame

$$\text{sem}(F) \in [0, 1]^S, \text{ where } \text{sem}(F)(s) = \begin{cases} f_0(s) & \text{if } s \models \varphi_0 \\ \dots & \dots \\ f_{k-1}(s) & \text{if } s \models \varphi_{k-1} \end{cases}.$$

In practice, symbolic frames provide a reasonable balance between space usage and expressivity. In Table 4.1 we provide an overview of all required basic operation of symbolic frames, with a specification according to the semantics. In our implementation, we use a library that efficiently computes all of these, even for large state spaces. Note that we overload the symbols $\leq, \wedge, \cdot, \odot, \circ, \sum, ()$ here.

In a similar way to symbolic frames, we define our symbolic representation of lf-sets. By convention, we use the symbol W instead of F for symbolic frames in the context of symbolic lf-sets.

name	symbol	usage	signature	specification
Checking emptiness	emp	$\text{emp}(\mathcal{G})$	$([0, 1]_{\text{sym}}^S \rightarrow \{0, 1\})$	$\text{emp}(\mathcal{G}) = 1 \iff \text{sym}(\mathcal{G}) = \emptyset$
Checking membership	$\in$	$F \in \mathcal{G}$	$[0, 1]_{\text{sym}}^S \times ([0, 1]_{\text{sym}}^S \rightarrow \{0, 1\})$	$(F \in \mathcal{G}) = 1 \iff \text{sym}(F) \in \text{sym}(\mathcal{G})$

Table 4.2: Required basic operations on $\mathcal{G} \in ([0, 1]_{\text{sym}}^S)_{\text{sym}}^{\downarrow}$ and $F \in [0, 1]_{\text{sym}}^S$.

Definition 47. A *symbolic lf-set* $\mathcal{G}$ is a set of tuples $\{(W_0, r_0), \dots, (W_{k-1}, r_{k-1})\} \subseteq [0, 1]_{\text{sym}}^S \times [0, 1]$. The set of all symbolic lf-sets is $([0, 1]_{\text{sym}}^S)_{\text{sym}}^{\downarrow}$.

Definition 48. The semantics of a symbolic lf-set $\mathcal{G} = \{(W_0, r_0), \dots, (W_{k-1}, r_{k-1})\} \in ([0, 1]_{\text{sym}}^S)_{\text{sym}}^{\downarrow}$ is an lf-set $\text{sem}(\mathcal{G})$ where

$$\text{sem}(\mathcal{G}) = \{V \in [0, 1]^S : \text{sem}(W_0) \cdot V \leq r_0\} \cap \dots \cap \{V \in [0, 1]^S : \text{sem}(W_{k-1}) \cdot V \leq r_{k-1}\}.$$

For lf-sets, we need the basic operations in Table 4.2. These are not provided by a library, but they are rather simple to implement using the dot product from Table 4.1, see Proposition 19.

Proposition 19. Let a symbolic lf-set $\mathcal{G} = \{(W_0, r_0), \dots, (W_{k-1}, r_{k-1})\} \in ([0, 1]_{\text{sym}}^S)_{\text{sym}}^{\downarrow}$. Then

1. $\text{sem}(\mathcal{G}) = \emptyset \iff \exists i. r_i < 0$, and
2. For $F \in [0, 1]_{\text{sym}}^S$, $\text{sem}(F) \in \text{sem}(\mathcal{G}) \iff \forall i. F \cdot W_i \leq r_i$. ($\cdot$ is the dot product on symbolic frames.)

4.3 Bellman operators

In this section, we define operators Φ and Ψ , equivalents of Φ , and Ψ on *symbolic frames* and *symbolic lf-sets*. Throughout the rest of this section, we fix a predicate η , a program

$$A = \{(\varphi_0, \mathbf{br}_0), \dots, (\varphi_{m-1}, \mathbf{br}_{m-1})\} \text{ with } \\ \mathbf{br}_i = (\varphi_i, \{(p_{i,0}, u_{i,0}), \dots, (p_{i,v-1}, u_{i,v-1})\}) \text{ for } 0 \leq i < m,$$

a Markov chain $\mathcal{M} = \text{sem}(A) = (S, P)$ and a set of bad states $B = \text{sem}(\eta) = \{s \in S : \eta(s) = 1\}$. Operators Φ , Ψ and Θ are defined w.r.t. this transition function P and set of bad states B .

4.3.1 The Bellman operator

We define the symbolic Bellman operator Φ . In Proposition 20 we prove that it is equivalent to the Bellman operator Φ , according to our semantics. $\phi(F, i, j)$ is the symbolic frame that holds the probability mass from a single update $u_{i,j}$. It is computed by taking the probability mass from update F with $u_{i,j}$ ($F \circ u_{i,j}$) on the domain $\varphi_i \wedge \eta$, and multiplying it by $p_{i,j}$. $\{\text{frame}(\eta)\}$ represents the probability mass from states satisfying η (bad states), and $\{\phi(F, i, j) : 0 \leq i < m \wedge 0 \leq j < v\}$ represents the probability mass from other states. We take the sum over these symbolic frames (see $\sum$ in Table 4.1). See Example 32.

Definition 49. The *symbolic Bellman operator* Φ : $[0, 1]_{\text{sym}}^S \rightarrow [0, 1]_{\text{sym}}^S$ is defined as

$$\Phi(F) = \sum \left(\{\text{frame}(\eta)\} \cup \{\phi(F, i, j) : 0 \leq i < m \wedge 0 \leq j < v\} \right)$$

where $\phi(F, i, j) = [p_{i,j}] \odot (F \circ u_{i,j}) \odot \text{frame}(\varphi_i \wedge \neg \eta)$.

Proposition 20. For $F \in [0, 1]_{\text{sym}}^S$, $\Phi(\text{sem}(F)) = \text{sem}(\Phi(F))$.

Example 32. Let $\mathbf{Vars} = \{x, y\}$ with $\text{dom}(x) = \text{dom}(y) = \{0, 1, 2\}$, a simple program $A = \{(\text{true}, \{(1, u)\})\}$ for update $u = [x \mapsto x + 1, y \mapsto y + 1]$, and $\eta = [x = 2 \wedge y = 2]$. Let a symbolic frame

$$F = \{([\frac{1}{2}], [x = 1 \wedge y = 1]), ([\frac{1}{8}], [x = 2 \wedge y = 1]), ([0], [x = 0 \vee y \neq 1])\}, \text{ then } \text{sem}(F) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & \frac{1}{2} & \frac{1}{8} \\ 0 & 0 & 0 \end{bmatrix},$$

$$\text{and } \Phi(\text{sem}(F)) = \begin{bmatrix} \frac{1}{2} & \frac{1}{8} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}. \text{ Furthermore,}$$

$$\begin{aligned} \Phi(F) &= \sum \left(\{\text{frame}(\eta)\} \cup \{\phi(F, i, j) : 0 \leq i < m \wedge 0 \leq j < v\} \right) = \text{frame}(\eta) + \phi(0, 0) \\ &= \text{frame}(\eta) + [1] \odot [F \circ u] \odot \text{frame}(\text{true} \wedge \neg \eta) \\ &= \text{frame}(\eta) + [F \circ u] \odot \text{frame}(\neg \eta) \\ &= \text{frame}(\eta) + \{([\frac{1}{2}], [x = 0 \wedge y = 0 \wedge \neg \eta]), ([\frac{1}{8}], [x = 1 \wedge y = 0 \wedge \neg \eta]), ([0], [x = 2 \vee y > 0])\} \\ &= \{(1, \eta), (0, \neg \eta)\} + \{([\frac{1}{2}], [x = 0 \wedge y = 0]), ([\frac{1}{8}], [x = 1 \wedge y = 0]), ([0], [x = 2 \vee y > 0])\} \\ &= \{([\frac{1}{2}], [x = 0 \wedge y = 0]), ([\frac{1}{8}], [x = 1 \wedge y = 0]), ([0], [x = 2 \vee y > 0 \wedge \neg \eta]), ([1], \eta)\} \end{aligned}$$

and indeed $\text{sem}(\Phi(F)) = \Phi(\text{sem}(F))$.

4.3.2 The reversed Bellman operator

Recall that in Section 3.4.3, we already briefly defined the next step operator Θ . We formally define Θ here for Markov chains, and then define its equivalent Θ on symbolic frames. This is the first step towards defining Ψ on symbolic lf-sets.

Definition 50. The *next step operator* $\Theta: [0, 1]^S \rightarrow [0, 1]^S$ is defined as

$$\Theta(F)(s') = \sum_{s \in S} P(s, s') \cdot F(s).$$

Recall that we only consider two kinds of valid updates: they are either *invertible* or *constant*. For an invertible update, we can define the inverse. We prove its relevant property in Lemma 3.

Definition 51 (Inverse). Let an invertible update for variable x_i , $f_i(s) = k_i \cdot s(x_i) + g(s)$, where $g(s) = \sum_{0 \leq j < v, j \neq i} k_j \cdot s(x_j) + k_v$. The *inverse* of f_i , is $f_i^{-1}(s) = \frac{s(x_i) - g_i(s)}{k_i}$. For invertible update u , the *inverse* of u is $u^{-1}(s)(x_i) = f_i^{-1}(s)$.

Example 33. For $f_i(s) = 2 \cdot s(x_i) + 5$, $k_i = 2$ and $g(s) = 5$, $f_i^{-1}(s) = \frac{s(x_i) - 5}{2} = \frac{1}{2}s(x_i) - 2\frac{1}{2}$.

Lemma 3. If u is an invertible update, then for $s, s' \in S$, $u(s) = s' \iff u^{-1}(s') = s$.

Example 34. Let $\mathbf{Vars} = \{x, y\}$, and update $u = [x \mapsto 2x + 5; y \mapsto y]$, then $u^{-1} = [x \mapsto \frac{1}{2}x - 2\frac{1}{2}, y]$. $u((2, 2)) = (2 \cdot 2 + 5) = (9, 2)$ and $u^{-1}((9, 2)) = (\frac{1}{2} \cdot 9 - 5, 2) = (2, 2)$.

Now we define Θ and prove its correctness according to our semantics in Proposition 21. $\theta(F, i, j)$ is the symbolic frame that holds the probability mass from a single update $u_{i,j}$. In case of an invertible update, it is computed by taking the probability mass from update F with the inverted update $(F \circ u_{i,j}^{-1})$ on the updated domain $\varphi \circ u_{i,j}^{-1}$, and multiplying it by $p_{i,j}$. In case of a constant update, it is computed by taking the sum of F over the domain φ ($\text{sum}(F, \varphi)$) multiplied by $p_{i,j}$ for the single state s' that results from applying $u_{i,j}$, and 0 for all other states. We take the sum over these symbolic frames (see $\sum$ in Table 4.1). See Example 35. Furthermore, recall that all updates in program A are valid by definition, so we do not need to account for cases where they are neither invertible nor constant.

Definition 52. The *symbolic next step operator* $\Theta: [0, 1]_{\text{sym}}^S \rightarrow [0, 1]_{\text{sym}}^S$ is defined as

$$\Theta(F) = \sum \left(\{ \theta(F, i, j) : 0 \leq i < m \wedge 0 \leq j < v \} \right)$$

$$\text{where } \theta(F, i, j) = \begin{cases} [p_{i,j}] \odot (F \circ u_{i,j}^{-1}) \odot \text{frame}(\varphi \circ u_{i,j}^{-1}) & \text{if } u_{i,j} \text{ is invertible,} \\ [p_{i,j} \cdot \text{sum}(F, \varphi_i)] \odot \text{frame}(\lambda s'. [u_{i,j}(*) = s']) & \text{if } u_{i,j} \text{ is constant.} \end{cases}$$

Proposition 21. For $F \in [0, 1]_{\text{sym}}^S$, $\Theta(\text{sem}(F)) = \text{sem}(\Theta(F))$.

Example 35. Let $\text{Vars} = \{x, y\}$ with $\text{dom}(x) = \text{dom}(y) = \{0, 1, 2\}$, a simple program $A = \{(\text{true}, \{(1, u)\})\}$ for some update u . Let a symbolic frame

$$F = \{([\frac{1}{2}], [x = 0 \wedge y = 0]), ([\frac{1}{8}], [x = 1 \wedge y = 0]), ([0], [x = 2 \vee y > 0])\}, \text{ then } \text{sem}(F) = \begin{bmatrix} \frac{1}{2} & \frac{1}{8} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

1. If we have a constant update $u(s) = s[x \mapsto 0; y \mapsto 0]$, then $\Theta(F) = \begin{bmatrix} \frac{5}{8} & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$. Then

$$\begin{aligned} \Theta(F) &= \sum (\{ \theta(i, j) : 0 \leq i < m \wedge 0 \leq j < v \}) = \theta(0, 0) \\ &= [1 \cdot \text{sum}(F, \varphi)] \odot \text{frame}([\lambda s'. [u(*) = s']]) \\ &= \{ ([1 \cdot (1/2 + 1/8)], \lambda s'. [s' = (0, 0)]), ([0], \lambda s'. [s' \neq (0, 0)]) \} \\ &= \{ ([5/8], \lambda s'. [s' = (0, 0)]), ([0], \lambda s'. [s' \neq (0, 0)]) \}, \end{aligned}$$

and indeed $\text{sem}(\Theta(F)) = \Theta(\text{sem}(F))$.

2. If we have an invertible update $u(s) = s[x \mapsto x + 1; y \mapsto y + 1]$, then $\Theta(F) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & \frac{1}{2} & \frac{1}{8} \\ 0 & 0 & 0 \end{bmatrix}$. In this case, note that $u^{-1} = [x \mapsto x - 1; y \mapsto y - 1]$. Then

$$\begin{aligned} \Theta(F) &= \sum (\{ \theta(i, j) : 0 \leq i < m \wedge 0 \leq j < v \}) = \theta(0, 0) \\ &= [1] \odot ((F \circ u^{-1}) \odot \text{frame}(\text{true} \circ u^{-1})) \\ &= [1] \odot ((F \circ u^{-1}) \odot [1]) \\ &= F \circ u^{-1} \\ &= F \circ [x \mapsto x - 1; y \mapsto y - 1] \\ &= \{ ([1/2], [x = 1 \wedge y = 1]), ([1/8], [x = 2 \wedge y = 1]), ([0], [x = 0 \vee y = 0 \vee y = 2]) \}, \end{aligned}$$

and indeed $\text{sem}(\Theta(F)) = \Theta(\text{sem}(F))$.

Now that we have defined Θ , we can finally define Ψ . Note that this is very similar to what we previously discussed in Section 3.4.3. We refer back to Example 20.

Definition 53. For $\mathcal{G} = \{(W, r)\} \in ([0, 1]_{\text{sym}}^S)_{\text{sym}}^{\downarrow}$, the reversed Bellman operator $\Psi: ([0, 1]_{\text{sym}}^S)_{\text{sym}}^{\downarrow} \rightarrow ([0, 1]_{\text{sym}}^S)_{\text{sym}}^{\downarrow}$ is defined as

$$\begin{aligned} \Psi(\{(W, r)\}) &= \{(\Theta(U), t)\}, \text{ where} \\ U &= (\neg \eta) \odot W, \\ t &= r - \sum_{s \in S} \eta(s) \cdot w(s). \end{aligned}$$

■ **Proposition 22.** For a symbolic lf-set $\mathcal{G} \in ([0, 1]^S)_{\text{sym}}^\downarrow$ with $|\mathcal{G}| = 1$, $\Psi(\text{sem}(\mathcal{G})) = \text{sem}(\Psi(\mathcal{G}))$.

4.4 Symbolic heuristics

Now that we have defined symbolic operators Φ , Θ and Ψ , the heuristics presented in Section 3.4 can be easily adopted to work using symbolic frames and lf-sets. For the hIn, hSe, hSe $^\Psi$ and hRs, the computational complexity for the symbolic variants does not scale with $|S|$.

With the **Refine** heuristics hRb, hR₀₁ and hRm, we run into a problem however. We have not found a way to compute these that does not computationally scale with $|\{s_i \in S : W(i) \neq 0\}|$, and we doubt that this is possible. This may be problematic, since this means that the time to compute them could scale with $|S|$ in the worst case. In practice, this means that we may face a similar problem that **PrIC3** in Example 46. We believe that it is possible to come up with good **Refine** heuristics that do not have this problem, but we leave this for future work.

Chapter 5

Generalization

Generalization is a post-processing step used inside the original PDR algorithm on TSs that uses a heuristic (called the generalization) to facilitate solving the reachability problem [6][12]. In the PDR algoirthm for TSs, there is a remarkably simple and effective generalization which allows PDR to find an inductive invariant much quicker.

Batz *et al.* have introduced generalization into the probabilistic setting in **PrIC3** [5]. We can broadly define probabilistic generalization as a heuristic step that tries to *guess* a frame that brings the algorithm closer to an inductive invariant. After making such a guess, the algorithm checks whether this guess was good enough to be potentially useful. See Example 36 for a situation in which a simple generalization can save many computation steps. In this chapter, we will discuss how generalization can be adopted in **AdjointPDR**[↓], and introduce two new generalization approaches. Note that all proofs for this chapter are deferred to Appendix A.4.

Example 36. In the Markov chain in Figure 5.1, with $\lambda \geq \frac{3439}{1000}$, there is an inductive invariant

$$I = [1 - (\frac{9}{10})^4, 1 - (\frac{9}{10})^3, 1 - (\frac{9}{10})^2, 1 - \frac{9}{10}, 0, 1] = [\frac{3439}{1000}, \frac{271}{1000}, \frac{19}{100}, \frac{1}{10}, 0, 1].$$

Because of the chain structure of this Markov chain, we can quickly compute an inductive variant by using the formula $1 - (\frac{9}{10})^{3-n}$ where n is the number of the state. However, **AdjointPDR**[↓] is not able to take advantage of this in its current form. In the following table, we run **AdjointPDR**[↓] with $\lambda = \frac{4}{10}$ and keep track of hRb at the conflict steps.

Step	z
1	$[\frac{4}{10}, 0, 0, 0, 0, 1]$
4	$[\frac{4}{10}, \frac{1}{10}, \frac{1}{10}, \frac{1}{10}, 0, 1]$
7	$[\frac{4}{10}, \frac{19}{100}, \frac{19}{100}, \frac{1}{10}, 0, 1]$
10	$[\frac{4}{10}, \frac{271}{1000}, \frac{19}{100}, \frac{1}{10}, 0, 1]$

As we can see, **AdjointPDR**[↓] arrives at the inductive invariant, but the amount of steps that this takes is linearly proportional to the amount of states in the Markov chain. In essence, we are performing value iteration with extra steps. This means that **AdjointPDR**[↓] with hRb (or hRs, HRm, or hR₀₁ for that matter) is infeasible for large state spaces for Markov chains with this structure, even though the problem is, in a sense, trivial.

If we use generalization, we may immediately try the frame $z = [\frac{4}{10}, \frac{3}{10}, \frac{2}{10}, \frac{1}{10}, 0, 1]$ for example. This frame is an inductive invariant since $z \leq P$ and $\Phi(z) \leq z$ (see Proof Principle 1). Instead of going through all these steps of inefficient value iteration, generalization has the potential to come up with an inductive invariant quickly.

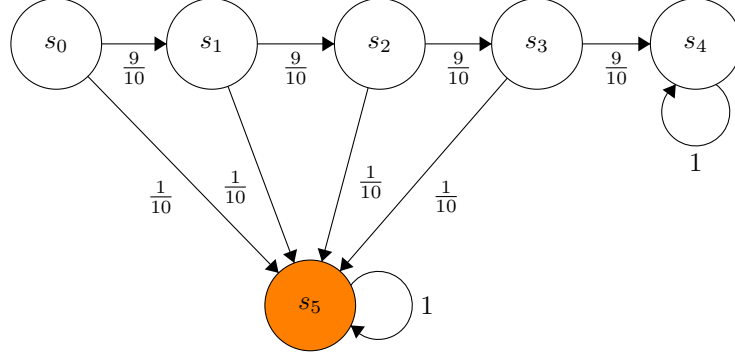


Figure 5.1: Markov chain with $S = \{s_0, s_1, s_2, s_3, s_4, s_5\}$ and $B = \{s_5\}$.

5.1 Generalization as a Refine heuristic

If we directly translated how generalization is used in **PrIC3** to the setting of **AdjointPDR**[↓], we would first apply the **Refine** heuristic, and then attempt to lower the probability values in the frame z additionally through generalization. We present what applying this idea to **AdjointPDR**[↓] would look like in Algorithm 2. Note that the new frame z' that is obtained after generalization has essentially the same constraints as the **Refine** heuristic itself.

Algorithm 2 Idea for the **Refine** step of **AdjointPDR**[↓] with generalization

- 1: **if** (a) $\mathcal{G} \neq \epsilon$ and (b) $\Phi(F_{k-1}) \in \mathcal{G}_k$ **then** ▷ **Refine**
 - 2: Let w , such that $\mathcal{G}_k = \{v : v \cdot w \leq r\}$.
 - 3: $z \leftarrow$ such that (1) $z \in \mathcal{G}_k$, and (2) $\Phi(F_{k-1} \wedge z) \leq z$. ▷ **Re**
 - 4: $z' \leftarrow$ such that (1a) $z' \in \mathcal{G}_k$, and (2a) $\Phi(F_{k-1} \wedge z') \leq z'$ and (3a) $z' \leq z$. ▷ **Generalization**
 - 5: **if** $\Phi(z') \leq z$ and $z' \leq P$ **then return true**
 - 6: **else**
 - 7:
$$\begin{pmatrix} F_0 \dots F_{k-1} & F_k & F_{k+1} \dots F_{n-1} \\ \mathcal{G}_k & \mathcal{G}_{k+1} \dots \mathcal{G}_{n-1} \end{pmatrix} \leftarrow \begin{pmatrix} F_0 \wedge z' \dots F_{k-1} \wedge z' & F_k \wedge z' & F_{k+1} \dots F_{n-1} \\ \mathcal{G}_{k+1} \dots \mathcal{G}_{n-1} \end{pmatrix}$$
-

This approach to generalization as a nonessential (in terms of termination and correctness) post-processing step that happens after the essential **Refine** heuristic, is line with the generalization in the original **PDR** algorithm, and generalization in **PrIC3**. However, this approach is not suitable for **AdjointPDR**[↓]. The main reason for this is that the heuristic in line 3 might give a frame z that "too low" in the partial order, such that any frame $z' \leq z$ cannot be useful, see Example 37. To avoid this problem, we instead interpret generalization as an instance of a **Refine** heuristic.

Example 37. When using **AdjointPDR**[↓] with the generalization as presented in Algorithm 2 on Example 36, generalization could not come up with $z' = [\frac{4}{10}, \frac{3}{10}, \frac{2}{10}, \frac{1}{10}, 0, 1]$ at the first conflict step, because heuristic **hRb** gives $z = [\frac{4}{10}, 0, 0, 0, 0, 1]$ and then $z' \not\leq z$.

5.2 Generalization framework

In this section, we introduce a simple framework for generalization in that lifts generalization techniques that consider only a single state to work as a **Refine** heuristic in **AdjointPDR**[↓], see Proposition 24. This enables us to adopt the generalization techniques from **PrIC3**. We start by considering that the element $\mathcal{G}_k = \{V : W \cdot V \leq r\}$ in the negative chain gives us constraints on states $s \in S$ where $W(s) \neq 0$, that any inductive invariant has to satisfy. These are useful hints for generalization.

Example 38. Suppose that we run $\text{AdjointPDR}^\downarrow$ on an MDP with $S = \{s_0, s_1, s_2\}$ and we obtain a negative sequence with $\mathcal{G}_k = ([\frac{1}{2}, 0, 1], \frac{1}{2})$. From this $\mathcal{G}_k$, we can derive that any inductive invariant F needs to satisfy $\frac{1}{2}F(s_0) + 1F(s_2) \leq \frac{1}{2}$. In this case, s_0 and s_2 are the non-zero states.

We present our generalization framework in Algorithm 3. We use variables $z \in [0, 1]^S$, which starts at **1** and later stores the result of generalization, and $r' \in [0, 1]$. We then loop over the set of non-zero states of $W \in [0, 1]^S$ (where $\mathcal{G}_k = \{F : F \cdot W \leq r\}$), and execute the generalization technique that works on the state level, **GeneralizeState** in line 3. This operation finds a frame z_g and $\delta \in [0, 1]$, where $z_g(s) \leq \delta$, and δ is as low as possible, see Definition 54. We keep updating z and r' at lines 4 and 5, and check if $r' < 0$ at line 6. If $r' < 0$, then we know that this generalization cannot provide us with a z such that $\sum_{s \in S} W(s) \cdot z(s) \leq r$ (i.e. $z \notin \mathcal{G}_k$), hence we give up on generalization and use a fallback conflict heuristic (which is provided as a parameter). If this does not happen, we complete the for-loop. We perform a last check to see whether $\Phi(F_k \wedge z) \leq z$ as required, and use the fallback heuristic if this is not the case.

Algorithm 3 Generalize

Parameters: A configuration $(F \parallel \mathcal{G})$ with $W \in [0, 1]^S, r \in [0, 1]$, where $\mathcal{G}_k = \{F : F \cdot W \leq r\}$, a heuristic $\text{hR}_{\text{fallback}}$, and a method for state generalization **GeneralizeState**

Result: A frame z , such that $z \in \mathcal{G}_k$ and $\Phi(F_{k-1} \wedge z) \leq z$

```

1:  $z \leftarrow \mathbf{1}, r' \leftarrow r.$ 
2: for  $s$  where  $W(s) \neq 0$  do
3:    $\delta, z_g \leftarrow \text{GeneralizeState}(F_{k-1}, s).$ 
4:    $z \leftarrow z \wedge z_g$ 
5:    $r' \leftarrow r' - W(s) \cdot \delta$ 
6:   if  $r' < 0$  then
7:     return  $\text{hR}_{\text{fallback}}(F \parallel \mathcal{G})$  ▷ Generalization failed
8: if  $\Phi(F_k \wedge z) \not\leq z$  then
9:   return  $\text{hR}_{\text{fallback}}(F \parallel \mathcal{G})$  ▷ Generalization failed
10: return  $z$  ▷ Generalization successful

```

We call a state generalization *valid* if the resulting value for δ is "correct" relative to the value of z_g . That is, δ is an upper bound for $z_g(s)$.

Definition 54. A function $\text{sg}: [0, 1]^S \times S \rightarrow [0, 1] \times [0, 1]^S$ is a *valid state generalization* if for all $s \in S$ and $F \in [0, 1]^S$, $(\delta, z_g) = \text{sg}(F, s)$ implies $z_g(s) \leq \delta$.

Proposition 23. All state generalization presented in Section 5.3 are valid state generalizations.

Proposition 24. Let $hR_{\text{gen}}^{\text{sg}, h}((F \parallel \mathcal{G}) := \text{Generalize}((F \parallel \mathcal{G}), h, \text{sg})$ for a **Refine** heuristic h and sg a valid state generalization. If h is a legit **Refine** heuristic, then so is $hR_{\text{gen}}^{\text{sg}, h}$.

A key difference between generalization in **PrIC3** and $\text{AdjointPDR}^\downarrow$ is as follows. In **PrIC3**, the value $\delta \in [0, 1]$, to which the generalization will attempt to lower the probability for a state $s \in S$ is determined beforehand using an oracle. In $\text{AdjointPDR}^\downarrow$ on the other hand, the state generalization tries to find the lowest possible value of δ during the process of generalization (in the worst case, $\delta = 1$). This effectively allows us to sidestep the need for an oracle.

5.3 State generalizations

In this section, we discuss three different possibilities for the subroutine **GeneralizeState**. Firstly, we discuss *linear generalization*. It is directly adopted from **PrIC3**, and needs an oracle to determine δ externally. Secondly, we discuss *linear binary generalization*. This uses a binary search to determine how low the value of δ can be. Thirdly, we discuss *exponential generalization*, which attempts to fit an exponential formula and takes a piecewise affine over-approximation. Throughout the rest of this chapter, we use states that consist

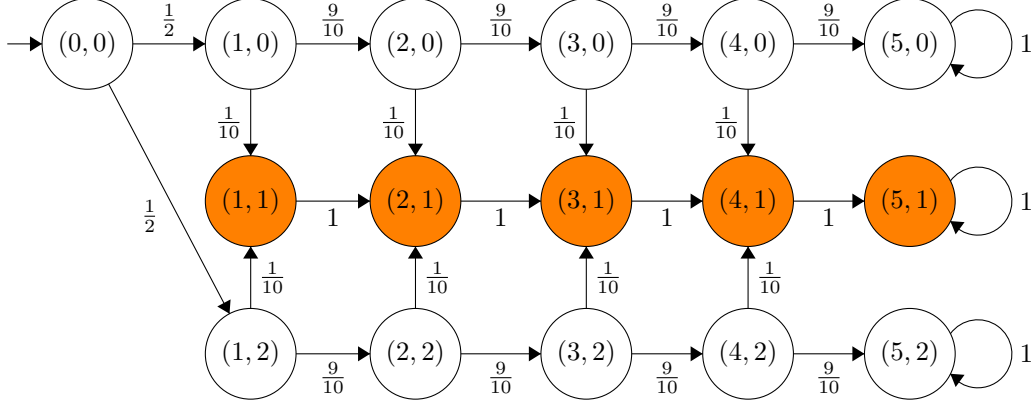


Figure 5.2: Markov chain with $\mathbf{Vars} = \{x, y\}$, $\text{dom}(x) = \{0 \dots 5\}$, $\text{dom}(y) = \{0, 1, 2\}$, $\eta = [y = 1]$. The states $(0,1)$ and $(0,2)$ are not shown.

of assignments of variables like in Chapter 4 (but we do not use the symbolic representation of frames). Furthermore, we use the Markov chain in Figure 5.2 as a running example.

5.3.1 Linear generalization

We first present linear generalization, which is directly based on Batz *et al.* [5]. The outer loop (Algorithm 4) asks the oracle what δ should be, loops over all variables, and then uses the subroutine **LinearGeneralizeVariable** to generalize state s over this variable x .

In **LinearGeneralizeVariable** (Algorithm 5) we start by obtaining the predicate ϑ , which is true on all all states s' where (1) $s(x) \leq s'(x) \leq u_x$, and (2) for all $y \in \mathbf{Vars}$ where $x \neq y$, $s'(y) = s(y)$. This is the domain on which values will actually be updated. We then *interpolate* a straight line between $(s(x), \delta)$ and (u_x, m_x) , where $m_x = \Phi^2(F)(s[x \mapsto u_x])$. Finally, we check whether $\Phi^2(F) \leq z$. Intuitively speaking, this rules out cases where we have a line that is "too low to work". If this check passes, we return true and a frame z where $z(s) = \delta$. Otherwise, we return false and **1**.

Finding a good oracle for linear generalization is a difficult problem in practice. In the **PrIC3** algorithm, determining the oracle may even require falling back on a different model checking technique in the worst case. The conflict heuristics that were introduced in previous chapters are not well-suited as oracles either. In Example 39, we show a run of **AdjointPDR**[↓] with linear generalization. Example 40 shows how to compute one step and detail, and also shows that the **Refine** heuristics from Chapter 3 are not good oracles in practice.

Algorithm 4 LinearGeneralization

Parameters: F, s, oracle

Result: A frame z

- 1: $\delta \leftarrow \text{oracle}(s)$
 - 2: $z \leftarrow \mathbf{1}$
 - 3: **for** $x \in \mathbf{Vars}$ **do**
 - 4: $z' \leftarrow \text{LinearGeneralizeVariable}(F, s, \delta, x)$
 - 5: $z \leftarrow z \wedge z'$
 - 6: **return** δ, z
-

Algorithm 5 LinearGeneralizeVariable**Parameters:** F, s, δ, x **Result:** A boolean **success**, and a frame z , where $\Phi^2(F) \leq z$

```

1:  $\vartheta \leftarrow s(x) \leq x \leq u_x \wedge \bigwedge_{y \in \text{Vars} \setminus \{x\}} y = s(x)$ 
2:  $m_x \leftarrow \Phi^2(F)(s[x \mapsto u_x])$ 
3:  $e \leftarrow \text{Interpolate}((s(x), \delta), (u_x, m_x))$ 
4:  $z \leftarrow [\vartheta] \cdot e + [\neg\vartheta] \cdot 1$ 
5: if  $\Phi^2(F) \leq z$  then
6:   true,  $z$ 
7: return false, 1

```

Example 39. When we run $\text{AdjointPDR}^\downarrow$ on the Markov chain from Figure 5.2 with $\lambda = \frac{5}{10}$, we start by applying an **Investigate** step. Then the next step is a **Refine** step. We use linear generalization with an oracle such that $\text{oracle}((0,0)) = \frac{5}{10}$. Generalization is applied successfully, even though the result it is still clearly not an inductive invariant.

	F	$\mathcal{G}$							F	$\mathcal{G}$	
0	$\emptyset$	-	$\xRightarrow{\text{In}}$	0	$\emptyset$	-			0	$\emptyset$	-
1	$\mathbf{0}$	-		1	$\mathbf{0}$	-			1	$\mathbf{0}$	-
2	$\mathbf{1}$	-		2	$\mathbf{1}$	$\{V : W_{(0,0)} \leq \frac{1}{2}\}$	$\xRightarrow{\text{Re}}$		2	$\begin{bmatrix} \frac{5}{10} & \frac{4}{10} & \frac{3}{10} & \frac{2}{10} & \frac{1}{10} & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$	-

Then after applying one **Extend** step, one **Investigate** step, and two **Search** steps, we get the following.

	F	$\mathcal{G}$
0	$\emptyset$	-
1	0	-
2	$\begin{bmatrix} \frac{5}{10} & \frac{4}{10} & \frac{3}{10} & \frac{2}{10} & \frac{1}{10} & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$	$\{V : \frac{1}{2}W_{(1,0)} + \frac{1}{2}W_{(1,2)} \leq \frac{1}{2}\}$
3	1	$\{V : W_{(0,0)} \leq \frac{1}{2}\}$

$\xRightarrow{\text{Ex}} \xRightarrow{\text{In}} \xRightarrow{\text{Se}} \xRightarrow{\text{Se}}$

Now we apply a final **Refine** step, with $\text{oracle}((1,0)) = \text{oracle}((1,2)) = \frac{1}{2}$ and obtain an inductive invariant, hence $\text{AdjointPDR}^\downarrow$ terminates positively. In Example 40 we show how the linear generalization on $(1,2)$ for variable x is computed in detail.

	F	$\mathcal{G}$
0	$\emptyset$	-
1	0	-
2	$\begin{bmatrix} \frac{5}{10} & \frac{4}{10} & \frac{3}{10} & \frac{2}{10} & \frac{1}{10} & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \frac{4}{8} & \frac{3}{8} & \frac{2}{8} & \frac{1}{8} & 0 \end{bmatrix}$	-
3	1	$\{V : W_{(0,0)} \leq \frac{1}{2}\}$

$\xRightarrow{\text{Re}}$

Example 40. In Example 39, the linear generalization on $(1, 2)$ for variable x is computed as follows.

$$\begin{aligned}
s(x) &= 1 \\
\delta &= \frac{4}{8} \\
u_x &= 5 \\
\vartheta &= \{(1, 2), (2, 2), (3, 2), (4, 2), (5, 2)\} \\
m_x &= 0 \\
e = s \mapsto \frac{4}{8} - \frac{1}{8}(s(x) - 1) & \quad \text{(A line through } (1, \frac{4}{8}) \text{ and } (5, 0)) \\
z &= \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \frac{4}{8} & \frac{3}{8} & \frac{2}{8} & \frac{1}{8} & 1 \end{bmatrix} \\
F_{k-1} \wedge z &= \begin{bmatrix} \frac{5}{10} & \frac{4}{10} & \frac{3}{10} & \frac{2}{10} & \frac{1}{10} & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \frac{4}{8} & \frac{3}{8} & \frac{2}{8} & \frac{1}{8} & 0 \end{bmatrix} \\
\Phi(F_{k-1} \wedge k) &\leq z \checkmark
\end{aligned}$$

Recall that $\text{oracle}((1, 2)) = \frac{1}{2}$. If we try to use hRb, hR₀₁, hRm, or hRs as oracles instead, we obtain 0, 1, 1, $\frac{1}{10}$ respectively. In none of these cases, generalization is successful. This shows that even in a simple case, the **Refine** heuristics from Chapter 3 may be unsuitable as oracles.

5.3.2 Binary search linear generalization

Binary search linear generalization aims to solve the problem of needing an oracle. Instead of relying on an external oracle to determine δ , we try to find the lowest possible δ using a binary search, see Algorithm 6 and Example 41.

Algorithm 6 BinaryGeneralization

Parameters: F, s

Result: A frame z

```

1: for  $x \in \text{Vars}$  do
2:    $\text{lb}, \text{ub} \leftarrow 0, 1$ 
3:    $n \leftarrow N$ 
4:    $z \leftarrow \mathbf{1}$ 
5:   while  $n > 0$  do
6:      $\text{mid} = \text{ub} - \frac{(\text{ub} - \text{lb})}{2}$ 
7:      $\text{success}, z' \leftarrow \text{LinearGeneralizeVariable}(F, s, \text{mid}, x)$ 
8:     if  $\text{success}$  then
9:        $z \leftarrow z \wedge z'$ 
10:       $\text{ub} = \text{mid}$ 
11:    else
12:       $\text{lb} = \text{mid}$ 
13:       $n = n - 1$ 
14: return  $z(s), z$ 

```

Example 41. We start with the following configuration of $\text{AdjointPDR}^\downarrow$ on the Markov chain of Figure 5.2. Note that this configuration would never be reached by an actual run of $\text{AdjointPDR}^\downarrow$.

	F	$\mathcal{G}$
0	$\emptyset$	-
1	$\mathbf{0}$	-
2	$\mathbf{1}$	$\{V : \frac{1}{2}W_{(1,0)} + \frac{1}{2}W_{(1,2)} \leq \frac{1}{2}\}$
3	$\mathbf{1}$	$\{V : W_{(0,0)} \leq \frac{1}{2}\}$

Now we compute binary search linear generalization over $F_{k-1} = \mathbf{1}$ on the state $s = (1, 2)$ for variable x , see Table 5.1. We use $N = 5$.

n	mid	z' (for $y = 2$)	success
5	$\frac{1}{2}$	$[1, \frac{4}{8}, \frac{3}{8}, \frac{2}{8}, \frac{1}{8}, 0]$	true
4	$\frac{1}{4}$	$[1, \frac{4}{16}, \frac{3}{16}, \frac{2}{16}, \frac{1}{16}, 0]$	false
3	$\frac{3}{8}$	$[1, \frac{12}{32}, \frac{9}{32}, \frac{6}{32}, \frac{3}{32}, 0]$	false
2	$\frac{7}{16}$	$[1, \frac{28}{64}, \frac{21}{64}, \frac{14}{64}, \frac{7}{64}, 0]$	true
1	$\frac{13}{32}$	$[1, \frac{52}{128}, \frac{39}{128}, \frac{26}{128}, \frac{13}{128}, 0]$	false

Table 5.1: Iterations of binary search over $F_{k-1} = \mathbf{1}$ on the state $s = (1, 2)$ for variable x . For z' we only show the values where $y = 2$ since the values where $y < 2$ remain unchanged.

The result of binary search linear generalization is

$$z = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & \frac{28}{64} & \frac{21}{64} & \frac{14}{64} & \frac{7}{64} & 0 \end{bmatrix}.$$

5.3.3 Exponential Generalization

The Markov chains in Figures 5.1 and 5.2 have a structure that allows inductive invariants to be succinctly represented by an exponential formula. Linear generalization techniques then merely over-approximate this exponential distribution using a straight line. With exponential generalization, we aim to exploit the potential exponential structure of an inductive invariant directly by deriving an exponential formula and taking a reasonable affine over-approximation of this formula. We still use this affine over-approximation so that this generalization technique may be applied on our symbolic representation from Chapter 4.

We present exponential generalization in Algorithm 7. We start by taking three points p_0, p_1, p_2 that are obtained by taking $\Phi^4(F)^1$, and reading the values at $s[x \mapsto u_x]$, $s[x \mapsto u_x - 1]$ and $s[x \mapsto u_x - 2]$. We try to fit an exponential formula to these points (line 6). If this is successful, we determine ϑ as in linear generalization (line 8). We then compute a piecewise affine over-approximation of e . Here $K \in \mathbb{N}$ is a constant that determines the maximum number of affine pieces in this over-approximation.

The piecewise affine over-approximation of e , a , consists of $K \in \mathbb{N}$ pieces, and must satisfy the following.

1. a must be greater than or equal to e on domain ϑ . That is, $\forall s \in \vartheta. a(s) \geq e(s)$.
2. the slope of a must be greater than or equal to the slope of e on domain ϑ . That is,

$$\forall s, s' \in \vartheta. (s(x) = s'(x) - 1 \implies e(s') - e(s) \geq a(s') - a(s)).$$

¹We apply Φ 4 times because in many of the examples we tested it on, this is enough to give 3 points that are informative for the exponential fit. See the Markov chain of Figure 5.2 for example with variable x . If we take only $\Phi^3(\mathbf{0})$, then we get $p_1 = (u_x - 1, \frac{19}{100})$ and $p_2 = (u_x - 2, \frac{19}{100})$, whereas we get $p_2 = (u_x - 2, \frac{271}{1000})$ with $\Phi^4(\mathbf{0})$.

Algorithm 7 ExponentialGeneralization

Parameters: F, s **Result:** a frame z

```
1:  $z \leftarrow \mathbf{1}$ 
2: for  $x \in \text{Vars}$  do
3:    $p_0 \leftarrow (u_x, \Phi^4(F)(s[x \mapsto u_x]))$ 
4:    $p_1 \leftarrow (u_x - 1, \Phi^4(F)(s[x \mapsto u_x - 1]))$ 
5:    $p_2 \leftarrow (u_x - 2, \Phi^4(F)(s[x \mapsto u_x - 2]))$ 
6:    $\text{success}, e \leftarrow \text{ExponentialFit}(p_2, p_1, p_0)$ 
7:   if  $\text{success}$  then
8:      $\vartheta \leftarrow s(x) \leq x \leq u_x \wedge \bigwedge_{y \in \text{Vars} \setminus \{x\}} y = s(x)$ 
9:      $a \leftarrow \text{AffineApproximation}(e, K)$ 
10:     $z' \leftarrow [\vartheta] \cdot a + [\neg\vartheta] \cdot \mathbf{1}$ 
11:    if  $\Phi^4(F) \leq F''$  then
12:       $z \leftarrow z \wedge z'$ 
13:  else
14:    return false,  $\mathbf{1}$ 
15: return  $z(s), \mathbf{1}$ 
```

Example 42. We start with the following configuration of $\text{AdjointPDR}^\downarrow$ on the Markov chain of Figure 5.2. This configuration would never be reached by an actual run of $\text{AdjointPDR}^\downarrow$, but that not important.

	F	$\mathcal{G}$
0	$\emptyset$	-
1	$\mathbf{0}$	-
2	$\mathbf{1}$	$\{V : \frac{1}{2}W_{(1,0)} + \frac{1}{2}W_{(1,2)} \leq \frac{1}{2}\}$
3	$\mathbf{1}$	$\{V : W_{(0,0)} \leq \frac{1}{2}\}$

Now we compute exponential generalization over $F_{k-1} = \mathbf{1}$ on the state $s = (1, 2)$ for variable x .

$$\begin{aligned}\Phi^4(F) &= \begin{bmatrix} 1 & 1 & \frac{271}{1000} & \frac{19}{100} & \frac{1}{10} & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & \frac{271}{1000} & \frac{19}{100} & \frac{1}{10} & 0 \end{bmatrix} \\ p_0, p_1, p_2 &= (5, 0), (4, \frac{1}{10}), (3, \frac{19}{100}) \\ e &= s \mapsto 1 - (\frac{9}{10})^{(5-s(x))} \\ \vartheta &= \{(1, 0), (2, 0), (3, 0), (4, 0), (5, 0)\} \\ z' &= \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0.36 & 0.28 & 0.19 & 0.10 & 0 \end{bmatrix}\end{aligned}$$

See Figure 5.3 for a plot of the points, e and a .

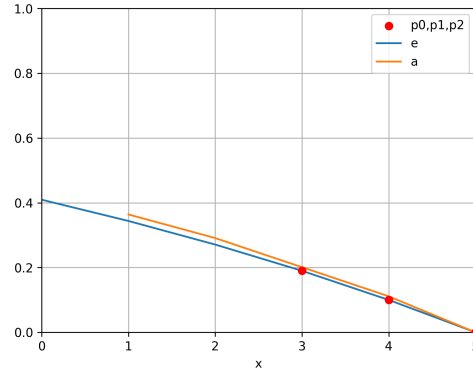


Figure 5.3: Plot of the points p_0 , p_1 and p_2 and exponential e , along with its piecewise affine over-approximation a on domain ϑ , see Example 42.

Chapter 6

Implementation & Experiments

In this chapter, we assess the performance of our implementation of `AdjointPDR↓` with different methods of generalization on a set of benchmarks. We compare this to the performance of the implementation of Kori *et al.* [23], and Storm, which serve as baselines. We then analyze these experimental results to gain insight into the feasibility of our approach, and the effectiveness of generalization. We pose the following research questions.

RQ1: How does our implementation without generalization compare against the implementation of Kori *et al.*?

RQ2: How does the performance of heuristics `hRs` and `hRb` differ in the two implementations of `AdjointPDR↓`?

RQ3: How do the different generalizations affect the performance of our implementation?

RQ4: How does the threshold λ affect the and the effectiveness of generalization?

RQ5: How do the two implementations of `AdjointPDR↓` compare against Storm?

Implementation

We have implemented a prototype of symbolic ¹ of `AdjointPDR↓` with generalization in Python. The implementation takes as input a *PRISM* file describing a Markov chain with the additional restrictions discussed in Chapter 4. We use the *ISL* library with *Barvinok* extensions [32][22] for storing and manipulating our core data structures (symbolic frames, lf-sets, probabilistic programs). Each of the heuristics presented in Chapter 3, and the generalizations presented in Chapter 5 are supported. We briefly use the library *sympy* [27] in the exponential generalization, and *z3* [28] for heuristic `hRm`.

Benchmarks & experimental setup

We mostly use a set of benchmark Markov chains that was originally compiled by Batz *et al.* [4] to evaluate their implementation of *PrIC3* and later used by Kori *et al.* to evaluate their implementation of `AdjointPDR↓`. This includes a combination of Markov chains from the literature (BRP [18][10], ZeroConf [8], Haddad-Monmege [14][17]) and some straightforward variations of grids (Chain, DoubleChain, Grid). Because of the additional restrictions on supported models (see Chapter 4), we have created new encodings for all models. All experiments were executed on a laptop with a 12th Gen Intel Core i5-1235U processor and 16GB DDR4 RAM, with a timeout limit of 240 seconds.

Experiments

In Table 6.1, we present the run-times for our implementation of `AdjointPDR↓`. We give for each model the name, the amount of states and the (estimated) actual probability. For each model and size, we have various values of the probability threshold λ . Then for each threshold, we have the rows `hRs` and `hRb` for the *Refine* heuristics. We chose to include only `hRs` and `hRb` in our measurements because in almost all of Kori *et al.*'s [23] experiments, either of them was fastest. We use `hIn` and `hSe` as *Investigate* and

¹<https://github.com/YouGuessedMyName/master-thesis>

Search heuristics respectively. The column "None" has the run-time of our implementation using hRs or hRb without generalization. Then the columns "Linear" "Binary" and "Exponential" have the run-time of runs of our implementation with the generalizations and hRs and hRb as fallback heuristics (see Algorithm 3). For Linear generalization, the fallback heuristic is also used as the oracle. For binary generalization, we use $N = 5$ (amount of binary search iterations) and for exponential generalization we use $K = 100$ (amount of pieces of the affine over-approximation). In the column "Kori" we have the run-times for Kori's implementation of $\text{AdjointPDR}^\downarrow$ on our machine. The best result for $\text{AdjointPDR}^\downarrow$ in each row (Kori's or ours) is **highlighted**. Finally, we have the columns "dd" and "sparse", which contain the run-times using Storm's [19] dd and sparse engines respectively. For timeouts, we use TO and for memory outs we use MO. We have two experiments where the `sympy` library was unable to handle a very large number, these are marked with *.

Discussion

RQ1: Our *implementation without generalization* (column "None") is much less efficient than Kori's implementation for most smaller models (Grid, Haddad-Monmege, DoubleChain and BRP, ZeroConf with 10^2 states). On these, our implementation often times out while Kori *et al.*'s implementation is able to find a result quickly, despite the fact that they implement the exact same algorithm with the same heuristics. This stark difference in performance can be explained by the difference in representation of the frames between the algorithm, and the more costly computation of Φ and Ψ (see Section 7.3). Furthermore, Hakell is typically a much faster programming language than Python. For bigger models, both our *implementation without generalization* and Kori *et al.*'s implementation reach a time out or memory out most of the time. There is one situation (ZeroConf with $|S| = 10^9$ and $\lambda = 0.45$) where our symbolic representation of frames enables $\text{AdjointPDR}^\downarrow$ to find a negative result very quickly, while Kori *et al.*'s Sparse representation results in a memory out.

RQ2: In Kori's implementation, we observe some differences between the performance of hRs and hRb. hRb is generally faster than hRs, but not always. We also observe something unexpected. Recall that $\text{AdjointPDR}^\downarrow$ with hRs can be seen as simply an instance of value iteration (see Section 3.4.4, Section 7.1). We would then expect the performance of $\text{AdjointPDR}^\downarrow$ with hRs to be the same for any $\lambda > \mathbb{P}_{\mathcal{M}}(\Diamond B)(s_0)$. However, in Kori's implementation, we observe differences between $\lambda = 0.75$ and $\lambda = 0.9$ in ZeroConf with $|S| = 10^2, 10^4$ and Chain with $|S| = 10^3$. We do not know what causes this difference. Our *implementation without generalization* times out on too many benchmarks to provide additional insights about this.

RQ3: Generalization is essential for the performance of our implementation in positive cases (where $\lambda \geq \mathbb{P}_{\mathcal{M}}(\Diamond B)$). In ZeroConf with $|S| = 10^2$ and Grid with $|S| = 10^2$, generalization enables our implementation to prove safety, whereas it times out without generalization. For ZeroConf with $|S| = 10^4$ and Chain with $|S| = 10^3$, generalization allows our implementation to significantly outperform Kori *et al.*'s implementation. In Chain with $|S| = 10^{12}$ for $\lambda = 0.4, 0.9$, generalization allows our implementation to find a result very quickly whereas Kori's implementation always reaches a memory out. Linear generalization appears to be only rarely effective, and exclusively with hRb. Exponential generalization appears to work the best on this set of benchmarks. Binary generalization is sometimes faster than Exponential generalization, but times out more often. For the latter two, the chosen fallback heuristic (hRs or hRb) does not appear to affect the performance significantly.

RQ4: We observe that the size of the probability threshold matters for the performance, with both Kori's implementation and our implementation having differing termination times for different λ , and generally terminating faster when $|\mathbb{P}_{\mathcal{M}}(\Diamond B)(s_0) - \lambda|$ is bigger. The effectiveness of generalizations is also affected by the value of λ . When $\lambda < \mathbb{P}_{\mathcal{M}}(\Diamond B)(s_0)$, enabling generalization always leads to a decrease in performance. However, when $\lambda > \mathbb{P}_{\mathcal{M}}(\Diamond B)(s_0)$, generalization often allows our implementation to find a result, whereas it fails without generalization.

RQ5: The comparison with Storm is interesting. Storm's dd engine appears to offer no benefit to performance on any of the benchmarks, and performs much slower on many benchmarks that the sparse engine can solve almost instantly. Neither our *implementation without generalization* nor Kori *et al.*'s implementation is able to outperform Storm's sparse engine on any of the benchmarks. On the very largest models, our *implementation with generalization* is able to vastly outperform Storm. For Chain with $|S| = 10^{12}$, our implementation is able to prove safety thanks to the Exponential generalization. For ZeroConf

with $|S| = 10^9$, our implementation is able to find a counterexample thanks to our symbolic data structures. Storm is completely unable to provide a result in these cases and reaches either a timeout or a memory out.

Conclusion

In conclusion, our *implementation without generalization* generally performs much worse than Kori’s implementation, while on (very) large models our *implementation with generalization* is sometimes able to greatly outperform Kori’s implementation and even Storm. Generalization is only effective in positive cases (when $\lambda \geq \mathbb{P}_{\mathcal{M}}(\Diamond B)(s)$), and Exponential generalization seems most effective, followed by Binary generalization.

Model	$ S $	$\mathbb{P}_{\mathcal{M}}(\Diamond B)(s_0)$	λ	Heuristic	None	Lin	Bin	Exp	Kori	dd	sparse
ZeroConf	10^2	0.5	0.45	hRs	0.048	0.074	0.188	0.085	0.128	<0.001	<0.001
				hRb	0.051	0.085	0.197	0.094	< 0.001		
			0.52	hRs	TO	TO	65.534	3.155	0.331		
				hRb	TO	TO	TO	3.152	0.533		
			0.75	hRs	TO	TO	1.508	3.136	0.297		
				hRb	TO	TO	3.682	3.209	0.520		
			0.9	hRs	TO	TO	1.514	3.150	0.092		
				hRb	TO	TO	1.536	3.257	0.527		
ZeroConf	10^4	0.5	0.45	hRs	0.044	0.075	0.186	0.082	0.059	49.649	0.004
				hRb	0.055	0.085	0.196	0.091	0.042		
			0.52	hRs	TO	TO	TO	13.218	TO		
				hRb	TO	TO	TO	13.904	MO		
			0.75	hRs	TO	TO	1.498	13.215	5.077		
				hRb	TO	TO	1.521	13.994	MO		
			0.9	hRs	TO	TO	1.519	13.229	2.419		
				hRb	TO	TO	1.529	14.161	MO		
ZeroConf	10^9	0.55	0.45	hRs	0.044	0.076	0.187	0.081	MO	TO	MO
				hRb	0.052	0.085	0.197	0.100	MO		
			0.52	hRs	TO	TO	TO	TO	MO		
				hRb	TO	TO	TO	TO	MO		
			0.75	hRs	TO	TO	TO	TO	MO		
				hRb	TO	TO	TO	*	MO		
			0.9	hRs	TO	TO	TO	TO	MO		
				hRb	TO	TO	TO	*	MO		
Chain	10^3	0.394	0.3	hRs	TO	TO	TO	TO	54.951	0.134	<0.001
				hRb	TO	TO	TO	TO	26.796		
			0.35	hRs	TO	TO	TO	TO	199.380		
				hRb	TO	TO	TO	TO	44.902		
			0.4	hRs	TO	TO	TO	0.532	175.998		
				hRb	TO	TO	TO	0.536	15.309		
			0.9	hRs	TO	TO	TO	0.541	0.211		
				hRb	TO	0.019	TO	0.532	15.354		

Continued on next page

Table 6.1 – continued from previous page

Model	$ S $	$\mathbb{P}_{\mathcal{M}}(\Diamond B)(s_0)$	λ	Heuristic	None	Lin	Bin	Exp	Kori	dd	sparse
Chain	10^{12}	0.394	0.3	hRs	TO	TO	TO	TO	MO	TO	MO
				hRb	TO	TO	TO	TO	MO		
			0.35	hRs	TO	TO	TO	TO	MO		
				hRb	TO	TO	TO	TO	MO		
			0.4	hRs	TO	TO	TO	0.545	MO		
				hRb	TO	TO	TO	0.537	MO		
			0.9	hRs	TO	TO	TO	0.537	MO		
				hRb	TO	0.024	TO	0.545	MO		
Grid	10^2	0.033	0.2	hRs	12.286	TO	TO	16.253	0.368	0.037	<0.001
				hRb	11.071	TO	TO	15.081	0.005		
			0.3	hRs	12.342	TO	TO	16.272	0.377		
				hRb	11.065	TO	TO	15.085	0.005		
Grid	10^3	< 0.001	0.2	hRs	TO	TO	TO	TO	2.543	0.162	<0.001
				hRb	TO	TO	TO	TO	0.441		
			0.3	hRs	TO	TO	TO	TO	2.590		
				hRb	TO	TO	TO	TO	0.450		
HaddadMonmege	41	0.7	0.75	hRs	TO	TO	TO	TO	3.57	80.581	<0.001
				hRb	TO	TO	TO	TO	0.20		
			0.9	hRs	TO	TO	TO	TO	4.002		
				hRb	TO	TO	TO	TO	0.001		
HaddadMonmege	10^3	0.7	0.75	hRs	TO	TO	TO	TO	TO	209.080	<0.001
				hRb	TO	TO	TO	TO	21.494		
			0.9	hRs	TO	TO	TO	TO	TO		
				hRb	TO	TO	TO	TO	21.289		
DoubleChain	10^3	0.215	0.216	hRs	TO	TO	TO	TO	TO	0.102	<0.001
				hRb	TO	TO	TO	TO	5.327		
			0.3	hRs	TO	TO	TO	TO	TO		
				hRb	TO	TO	TO	TO	5.330		
			0.9	hRs	TO	TO	TO	TO	TO		
				hRb	TO	TO	TO	TO	5.322		
BRP	10^3	0.035	0.005	hRs	TO	TO	TO	TO	1.221	0.0676	<0.001
				hRb	TO	TO	TO	TO	0.229		
			0.01	hRs	TO	TO	TO	TO	2.383		
				hRb	TO	TO	TO	TO	0.729		
			0.1	hRs	TO	TO	TO	TO	23.709		
				hRb	TO	TO	TO	TO	5.283		

Table 6.1: Benchmark results for our and Kori’s implementation of $\text{AdjointPDR}^\downarrow$, with a timeout of 240 seconds and a memory limit of around 16 GB.

Chapter 7

Related Work

In this chapter, we first briefly discuss a simple variant of value iteration, and observe that $\text{AdjointPDR}^\downarrow$ with hRs can be interpreted as an instance of value iteration. Then we discuss the similarities between PrIC3 and $\text{AdjointPDR}^\downarrow$ and show an example where $\text{AdjointPDR}^\downarrow$ has a potential advantage. Finally, we briefly discuss the representation of data structures in Kori *et al.*'s implementation of $\text{AdjointPDR}^\downarrow$, and how they compare to our symbolic implementation.

7.1 Value Iteration

Value iteration (VI) is the simplest and most used algorithm to solve quantitative maximal reachability. In modern model checking tools such as PRISM [25], Storm [19], Modest [16] and EPMC [15], it is usually the default algorithm. These tools then specialize in using variants of value iteration with efficient data structures such as Storm's sparse representation.

We describe a simple variant of value iteration in Algorithm 8. The algorithm has two frames F and G where Φ is repeatedly applied on G , and F stores the previous value of G . Then, we have a function **StoppingCriterion** which determines whether G is close enough to $\mu\Phi$ that the algorithm can terminate. There are many options for stopping criteria in the literature which we will not discuss in detail [3][13][17][30]. We call the simplest stopping criterion **Equal**, where $\text{Equal}(F, G)$ is true iff $F = G$. Example 43 illustrates that $\text{AdjointPDR}^\downarrow$ with hIn and hRs can be interpreted as an instance of value iteration.

Algorithm 8 Value iteration

Question: What is $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B)$?

```

1:  $F, G \leftarrow \mathbf{0}$ 
2: while  $\neg \text{StoppingCriterion}(F, G)$  do
3:    $F \leftarrow G$ 
4:    $G \leftarrow \Phi(G)$ 
5: return  $G$ 

```

Example 43. We run value iteration on the Markov chain of Figure 3.5 with stopping criterion **Equal**. We only show the value of G , since F is simply the value of G on the previous iteration. Notice that the value of G always becomes exactly the same as the last element of the positive chain in the run of $\text{AdjointPDR}^\downarrow$ with hIn, hSe and hRs in Example 25.

iteration	0	1	2	3	4	5	...
G	$\mathbf{0}$	$[0, 0, 0, 0, 1]$	$[0, 0, \frac{1}{2}, 0, 1]$	$[0, \frac{1}{4}, \frac{1}{2}, 0, 1]$	$[\frac{1}{4}, \frac{1}{4}, \frac{5}{8}, 0, 1]$	$[\frac{1}{4}, \frac{5}{16}, \frac{5}{8}, 0, 1]$	...

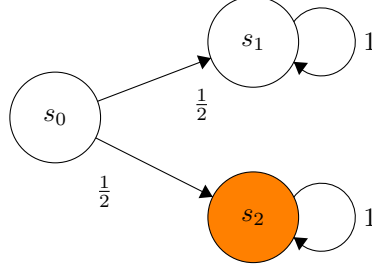


Figure 7.1: Markov chain with $S = \{s_0, s_1, s_2\}$ and $B = \{s_2\}$.

7.2 PrIC3

In this section, we give a very brief overview of **PrIC3**[5], mainly to draw attention to the similarities between **AdjointPDR**[↓] and **PrIC3**. We refer to [4, Chapter 6] for a more detailed treatment of **PrIC3**. Much like **AdjointPDR**[↓], **PrIC3** uses a positive chain.

■ **Definition 55.** A configuration in **PrIC3** is a chain of frames F of length k , written as $(F_0, \dots, F_{k-1})_k$.

We present the main loop of **PrIC3**, which is called **PrIC3** _{$\mathcal{H}$} in Algorithm 9. We can immediately notice the similarities to **AdjointPDR**[↓]. There is a very clear equivalent to the **Extend** step in line 6, and a positive conclusion in line 7. An equivalent to what we sometimes referred to as the *investigation phase* (where the **Investigate**, **Search** and **Refine** steps are applied), is performed in a separate loop called the *strengthening loop* (line 4). (Potential) counterexamples are sets of states that are returned by the strengthening loop and need to be investigated by the additional outer loop to verify if they contain paths that are indeed counterexamples.

PrIC3 _{$\mathcal{H}$} is not sound in the sense of Theorem 3; it is only sound if it returns true. For soundness, **PrIC3** needs an additional outer loop that processes the counterexamples, for which we refer to [4, Section 6.2.6].

Algorithm 9 **PrIC3** _{$\mathcal{H}$}

Question: Is $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B) \leq P$?

```

1:  $F \leftarrow (\mathbf{0}, \mathbf{1})_2$ 
2: oldSubsystem  $\leftarrow \emptyset$ 
3: while true do
4:   success,  $F$ , subsystem  $\leftarrow$  Strengthen $\mathcal{H}$ ( $F$ )                                ▷ “Investigate, Search, Refine”
5:   if  $\neg$ success then return false, subsystem
6:    $F \leftarrow (F_0, \dots, F_{k-1}, \mathbf{1})_{k+1}$                                           ▷ “Extend”
7:   if  $\exists 1 \leq i < k : F_i = F_{i+1}$  then return true                                ▷ “Positively conclusive”
8:   if oldSubsystem = subsystem then return false, subsystem
9:   oldSubsystem  $\leftarrow$  subsystem
  
```

See Algorithm 10 for the strengthening loop. Just like in **AdjointPDR**[↓], the goal here is to lower the frames in the positive chain such that the last element is hopefully close to an inductive invariant, or to find a counterexample. In **PrIC3**, strengthening uses an *obligation queue* that keeps track of individual states, and to what value the probability of these states needs to be lowered. This probability is already decided during the equivalent of the **Search** step, using a heuristic $\mathcal{H}$. This is different from **AdjointPDR**[↓], where this is decided during the **Refine** step using a **Refine** heuristic. This heuristic $\mathcal{H}$ is refined by the outermost **PrIC3** loop when a “false counterexample” is found. See Example 44 and Example 45.

Algorithm 10 Strengthen _{$\mathcal{H}$}

```

1:  $Q \leftarrow \{(k-1, s_0, \lambda)\}$  ▷ "Investigate"
2: while  $Q$  is not empty do
3:    $(i, s, \delta) \leftarrow Q.\text{popMin}()$ 
4:   if  $i = -1 \vee (s \in B \wedge \delta < 1)$  then return false, subsystem ▷ "Negatively conclusive" (not sound)
5:   if  $\exists \sigma \in \Pi^{\text{MD}} : \Phi_\sigma(F_{i-1})(s) > \delta$  then
6:      $\{s'_0, \dots, s'_{n-1}\} \leftarrow \{s' \in S : P(s, \sigma(s), s') \neq 0\}$  ▷ "Search" for one state  $s$ 
7:      $\delta_0, \dots, \delta_{n-1} \leftarrow \mathcal{H}(s, \sigma, \delta)$ 
8:      $Q.\text{push}((i-1, s'_0, \delta_0), \dots, (i-1, s'_{n-1}, \delta_{n-1}))$ 
9:   else
10:     $G(s') \leftarrow (\delta \text{ if } s = s' \text{ else } 1)$  ▷ "Refine" for one state  $s$ 
11:     $F \leftarrow (F_0 \wedge G, \dots, F_{i-1} \wedge G)$ 
12:     $F \leftarrow \text{Generalization}(F)$ 
13: return true,  $F$ ,  $Q.\text{touched}()$ 

```

Example 44. We show a run of the strengthening step in PrIC3 for the Markov chain of Figure 7.1 with $\lambda = \frac{1}{2}$, using a heuristic that is “unhelpful” and a heuristic that is “helpful”. Let us begin with the configuration $(\mathbf{0}, [\frac{1}{2}, 1, 1], \mathbf{1})_3$ (this configuration is obtained after a simple strengthening loop which we do not show here). The progression of the obligation queue is shown in the table below, from left to right. Let the “unhelpful” heuristic $\mathcal{H}(s_1, \sigma, \frac{1}{2}) = \mathcal{H}(s_2, \sigma, \frac{1}{2}) = \frac{1}{2}$. The coordinates where values are lowered in the “Refine” step are **highlighted**.

Step	“Investigate”	“Search”	“Refine”	“Negatively conclusive”
$s \in B \wedge \delta < 1?$	-	$s_0 \notin B$	$s_1 \notin B$	$s_2 \in B$ and $\frac{1}{2} < 1$
$\Phi(F_{i-1})(s) > \delta?$	-	$1 > \frac{1}{2}$	$0 \not\geq \frac{1}{2}$	-
(i, s, δ)	-	$(2, s_0, \frac{1}{2})$	$(1, s_1, \frac{1}{2})$	$(1, s_2, \frac{1}{2})$
Q	$\{(2, s_0, \frac{1}{2})\}$	$\{(1, s_1, \frac{1}{2}), (1, s_2, \frac{1}{2}), (2, s_0, \frac{1}{2})\}$	$\{(1, s_2, \frac{1}{2}), (2, s_0, \frac{1}{2})\}$	$\{(2, s_0, \frac{1}{2})\}$
F_0	$[0, 0, 0]$	$[0, 0, 0]$	$[0, 0, 0]$	$[0, 0, 0]$
F_1	$[\frac{1}{2}, 1, 1]$	$[\frac{1}{2}, 1, 1]$	$[\frac{1}{2}, \frac{1}{2}, 1]$	$[\frac{1}{2}, \frac{1}{2}, 1]$
F_2	$[1, 1, 1]$	$[1, 1, 1]$	$[1, 1, 1]$	$[1, 1, 1]$

Here Strengthen _{$\mathcal{H}$} and subsequently PrIC3 _{$\mathcal{H}$} return false, even though $\mathbb{P}_{\mathcal{M}}(\Diamond B)(s_0) \leq \lambda$. This illustrates that PrIC3 _{$\mathcal{H}$} is not sound. Now, the outermost loop of PrIC3 improves the heuristic to $\mathcal{H}(s_1, \sigma, \frac{1}{2}) = 0$ and $\mathcal{H}(s_2, \sigma, \frac{1}{2}) = 1$. Once again we end up with $(\mathbf{0}, [0, 1, 1], \mathbf{1})_3$ and we show the subsequent strengthening step below.

Step	“Investigate”	“Search”	“Refine”	“Refine”	“Refine”
$s \in B \wedge \delta < 1?$	-	$s_0 \notin B$	$s_1 \notin B$	$s_2 \in B$ but $1 \not< 1$	$s_0 \notin B$
$\Phi(F_{i-1})(s) > \delta?$	-	$1 > \frac{1}{2}$	$0 \not\geq 0$	$0 \not\geq 1$	$0 \not\geq \frac{1}{2}$
(i, s, δ)	-	$(2, s_0, \frac{1}{2})$	$(1, s_1, 0)$	$(1, s_2, 1)$	$(2, s_0, \frac{1}{2})$
Q	$\{(2, s_0, \frac{1}{2})\}$	$\{(1, s_1, 0), (1, s_2, 1), (2, s_0, \frac{1}{2})\}$	$\{(1, s_2, 1), (2, s_0, \frac{1}{2})\}$	$\{(2, s_0, \frac{1}{2})\}$	$\{\}$
F_0	$[0, 0, 0,]$	$[0, 0, 0]$	$[0, 0, 0]$	$[0, 0, 0]$	$[0, 0, 0]$
F_1	$[\frac{1}{2}, 1, 1,]$	$[\frac{1}{2}, 1, 1]$	$[\frac{1}{2}, \mathbf{0}, 1]$	$[\frac{1}{2}, \mathbf{0}, 1]$	$[\frac{1}{2}, 0, 1]$
F_2	$[1, 1, 1,]$	$[1, 1, 1]$	$[1, 1, 1]$	$[1, 1, 1]$	$[\frac{1}{2}, 1, 1]$

After this strengthening step, PrIC3 has to perform one more similar strengthening step to end up in configuration $([0, 0, 0], [\frac{1}{2}, 0, 1], [\frac{1}{2}, 0, 1], [\frac{1}{2}, 1, 1])$ and then returns true.

The difference in handling the obligation queue vs. the negative sequence can result in situations where the obligation queue gets exponentially large, whereas the negative sequence is more manageable, see Example 46. The representation of the negative sequence can still grow exponentially, but this is a problem with the symbolic representation, and not with AdjPDR[↓]. See Example 47.

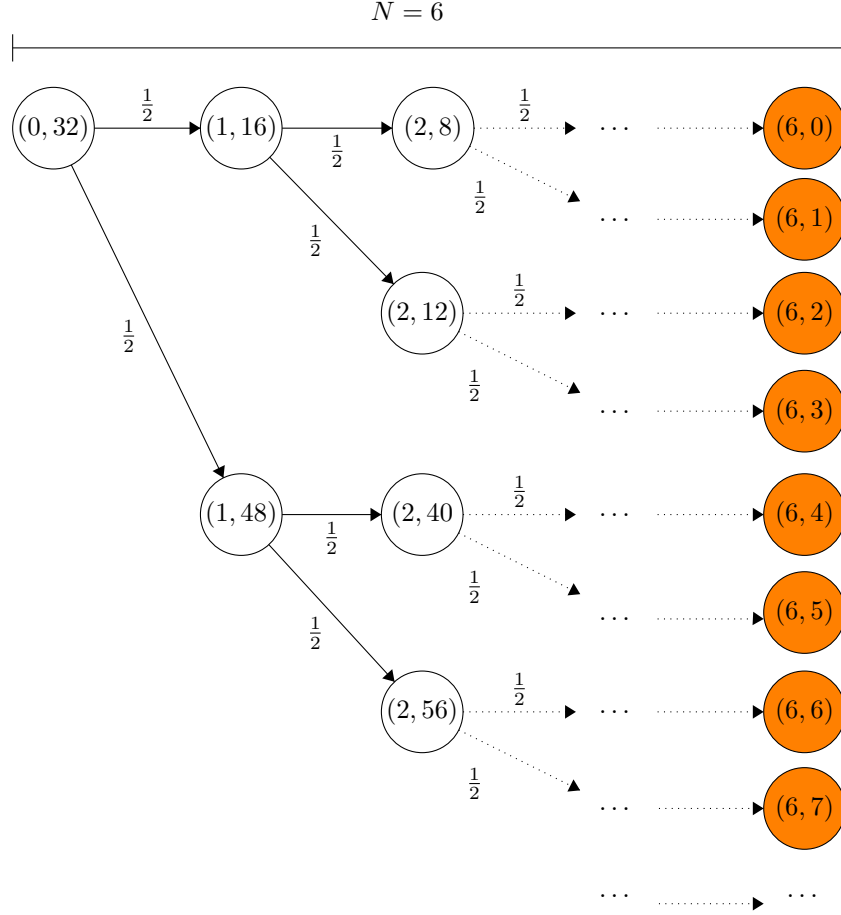


Figure 7.2: Markov chain with $\text{Vars} = \{x, y\}$ with $\text{dom}(x) = \{0 \dots N\}$, $\text{dom}(y) = \{0 \dots 2^N\}$. $\eta := (x = N)$, with $N = 6$.

Example 45. We now run $\text{AdjointPDR}^\downarrow$ with hIn , hSe and hRs on the Markov chain of Figure 7.1, with a similar starting configuration as in Example 44

F	$\mathcal{G}$		F	$\mathcal{G}$		F	$\mathcal{G}$	
$\perp$	—		$\perp$	—		$\perp$	—	
$[0, 0, 0]$	—	$\xRightarrow{\text{In}}$	$[0, 0, 0]$	—	$\xRightarrow{\text{Se}}$	$[0, 0, 0]$	—	$\xRightarrow{\text{Re}}$
$[\frac{1}{2}, 1, 1]$	—		$[\frac{1}{2}, 1, 1]$	—		$[\frac{1}{2}, 1, 1]$	$\{v : [0, \frac{1}{2}, \frac{1}{2}] \leq \frac{1}{2}\}$	
$[1, 1, 1]$	—		$[1, 1, 1]$	$\{v : [1, 0, 0] \leq \frac{1}{2}\}$		$[1, 1, 1]$	$\{v : [1, 0, 0] \leq \frac{1}{2}\}$	
F	$\mathcal{G}$		F	$\mathcal{G}$		F	$\mathcal{G}$	
$\perp$	—		$\perp$	—		$\perp$	—	
$[0, 0, 0]$	—	$\xRightarrow{\text{Re}}$	$[0, 0, 0]$	—		$[0, 0, 0]$	—	
$[0, 0, 1]$	—		$[0, 0, 1]$	—		$[0, 0, 1]$	—	
$[1, 1, 1]$	$\{v : [1, 0, 0] \leq \frac{1}{2}\}$		$[\frac{1}{2}, 0, 1]$	—		$[\frac{1}{2}, 0, 1]$	—	

And $\text{AdjointPDR}^\downarrow$ returns positively conclusive.

As we can see, $\mathcal{G}_2 = \{v : [0, \frac{1}{2}, \frac{1}{2}] \leq \frac{1}{2}\}$ takes on a similar role as the obligation queue $\{(1, s_1, \frac{1}{2}), (1, s_2, \frac{1}{2})\}$ or $\{(1, s_1, 0), (1, s_2, 1)\}$ from Example 44. Unlike in PrIC3 , it does not need to be decided during the “Search” step to which value each state needs to be lowered, this is handled in the **Refine** step.

Example 46. Consider the Markov chain $\mathcal{M}$ from Figure 7.2 with $\lambda < 1$. This problem can very simply be solved by observing that $\Phi^N(\mathbf{0}) \not\leq P$. However, if we run **PrIC3** on $\mathcal{M}$ with $\lambda = \frac{1}{2}$, we will eventually end up with an obligation queue of exponential size, that contains every state which has bad states as direct successors, $Q \supseteq \{(5, y) : y \bmod 2 = 0\}$, with $|Q| \geq 2^{N-1}$. This is problematic if N is large enough. On the other hand, **AdjointPDR**[↓] with **hIn**, **hSe** and **hRs** is able to solve this problem in at $O(N)$ steps, with no data structure that grows exponentially. Let $N = 6$, and let $\xRightarrow{3}$ be short for $\xRightarrow{Ex} \xRightarrow{In} \xRightarrow{Re}$. Furthermore, let the following boolean expressions (here $[\varphi] = (\lambda s.1 \text{ if } \varphi(s) \text{ and } 0 \text{ otherwise})$).

$$\begin{aligned}\varphi_0 &= [x = 0 \wedge y = 32] \\ \varphi_1 &= [x = 1 \wedge y \bmod 16 = 0 \wedge y \bmod 32 \neq 0] \\ \varphi_2 &= [x = 2 \wedge y \bmod 8 = 0 \wedge y \bmod 16 \neq 0] \\ \varphi_3 &= [x = 3 \wedge y \bmod 4 = 0 \wedge y \bmod 8 \neq 0] \\ \varphi_4 &= [x = 4 \wedge y \bmod 2 = 0 \wedge y \bmod 4 \neq 0] \\ \varphi_5 &= [x = 5 \wedge y \bmod 2 \neq 0] \\ \varphi_6 &= [x = 6]\end{aligned}$$

F	G		F	G		F	G		F	G		
$\perp$	-	$\xRightarrow{In}$	$\perp$	-	$\xRightarrow{Re}$	$\perp$	-	$\xRightarrow{3}$	$\perp$	-		
$\mathbf{0}$	-		$\mathbf{0}$	-		$\mathbf{0}$	-		$[\varphi_6]$	-	$[\varphi_6]$	-
$\mathbf{1}$	-		$\mathbf{1}$	$\{v : [\varphi_0] \cdot v \leq \frac{1}{2}\}$		$[\varphi_6]$	-		$[\varphi_5 \vee \varphi_6]$	-	$[\varphi_5 \vee \varphi_6]$	-
					$\xRightarrow{3} \dots \xRightarrow{3}$							

After $N = 6$ **Refine** steps, we get

F	G
$\perp$	-
$\mathbf{0}$	-
$[\varphi_6]$	-
$[\varphi_5 \vee \varphi_6]$	-
$\dots$	$\dots$
$[\varphi_0 \vee \varphi_1 \vee \varphi_2 \vee \varphi_3 \vee \varphi_4 \vee \varphi_5 \vee \varphi_6]$	-

Example 47. We compute Example 46 using our symbolic representation from Chapter 4. Our definition of Φ is not able to handle representation using the modulo operator well. For example, the frame $[\varphi_0 \vee \varphi_1 \vee \varphi_2 \vee \varphi_3 \vee \varphi_4 \vee \varphi_5 \vee \varphi_6]$ is represented as

$$\{([1], x = 0 \wedge y = 32), ([1], x = 1 \wedge y = 16), ([1], x = 1 \wedge y = 48), ([1], x = 2 \wedge y = 8), ([1], x = 2 \wedge y = 12), \dots, ([0], x = 0 \wedge y \neq 32), ([0], x = 1 \wedge y \neq 16), ([0], x = 1 \wedge y \neq 48), ([0], x = 2 \wedge y \neq 8), ([0], x = 2 \wedge y \neq 12), \dots\}$$

and the size of this representation grows exponentially in N . Of course, it is possible to detect and apply simplification in cases where the modulo operator can be used to simplify the representation.

7.3 Sparse frames

As discussed in Chapter 4, we use a *symbolic* representation of frames that supports piecewise affine functions. In Kori *et al.*'s implementation[23], they use Haskell's **IntMap** data structure, which can be described as *sparse*. That is, they have a default value 0, and explicitly store all the states where the value is not 0.

The main advantage to our approach as compared to Kori's approach is that our symbolic frames may be much more information-dense (in the sense that the representation of a frame may take much less memory) than **IntMap**, see Example 48. The main disadvantage to our approach is that our Φ and Ψ functions are much more complex (and costly) to compute, and in order to efficiently define Ψ , we even impose additional restrictions on the programs that are accepted by our implementation, see Chapter 4. With Kori's sparse frames, they can almost directly use the definitions of operators Φ and Ψ_σ (from Section 2.3.1 and Section 3.4.3).

Example 48. Let $\text{Vars} = \{x\}$ with $\text{dom}(x) = \{1 \dots 2000\}$, and let $F, G \in [0, 1]^S$, such that

$$F(s) = \begin{cases} 0 & \text{if } 1 \leq s(x) \leq 1000 \\ 1 & \text{if } 1001 \leq s(x) \leq 2000 \end{cases} \quad \text{and} \quad G(s) = \begin{cases} 0 & \text{if } 1 \leq s(x) \leq 1000 \\ \frac{s(x)}{2000} & \text{if } 1001 \leq s(x) \leq 2000 \end{cases}.$$

Then we can have symbolic frames F_{sym} and G_{sym} with $|F_{\text{sym}}| = 2$ and $|G_{\text{sym}}| = 2$, such that $\text{sem}(F_{\text{sym}}) = F$ and $\text{sem}(G_{\text{sym}}) = G$. Their symbolic representation looks very similar to F and G as they are written in this example.

However, if we use Kori's sparse approach, we need at least a 1000 spaces in memory to store these frames. Their representation would look similar to the following,

$$F(s) = \begin{cases} 1 & \text{if } s(x) = 1001 \\ 1 & \text{if } s(x) = 1002 \\ \dots & \dots \\ 1 & \text{if } s(x) = 2000 \\ 0 & \text{otherwise} \end{cases} \quad \text{and} \quad G(s) = \begin{cases} \frac{1001}{2000} & \text{if } s(x) = 1001 \\ \frac{1002}{2000} & \text{if } s(x) = 1002 \\ \dots & \dots \\ \frac{2000}{2000} & \text{if } s(x) = 2000 \\ 0 & \text{otherwise} \end{cases}.$$

Chapter 8

Conclusions & Future work

In summary, we provided a direct presentation of the $\text{AdjointPDR}^\downarrow$ algorithm for MDPs, proved its soundness and negative termination, and provided a possible extension for positive termination. We then discussed how the relevant data structures and operations can be symbolically represented. Subsequently, we demonstrated that it is possible to introduce generalization into $\text{AdjointPDR}^\downarrow$ as a **Refine** heuristic. Finally, our benchmark results showed that generalization can allow $\text{AdjointPDR}^\downarrow$ to succeed on very large Markov chains, where even Storm fails.

Heuristics

Regarding $\text{AdjointPDR}^\downarrow$'s heuristics, we observed that $\text{AdjointPDR}^\downarrow$ with hRs (and hIn) can be interpreted as a variant of value iteration, and that on some models, the other **Refine** heuristics from Kori *et al.*[23] may also result in runs of $\text{AdjointPDR}^\downarrow$ that are very similar to value iteration in practice. Our benchmark results (and Kori's results) do confirm however, that there are practical cases where hRb and hR₀₁ perform much better than hRs.

Generalization

Using our generalization framework, the previously proposed generalizations from PrIC3 fit well into $\text{AdjointPDR}^\downarrow$ as part of a **Refine** heuristic. We adopted PrIC3's linear generalization, and introduced binary and exponential generalization. An important advantage to our generalization framework as opposed to PrIC3, is that it does not require an external oracle to find a value for δ . Instead, finding this δ is part of the generalization procedure itself.

Symbolic representation

We observed that the representation of data structures and the way that basic operations are computed (our *symbolic* representation versus Kori's *sparse* representation) makes a big impact on performance in $\text{AdjointPDR}^\downarrow$. This is the reason that our *implementation without generalization* generally performed much worse than Kori's implementation of the exact same algorithm on small benchmarks. On very large benchmarks however, our symbolic implementation is sometimes able to provide a result where Kori's sparse implementation and Storm exceed the memory limit.

PrIC3

It appears that $\text{AdjointPDR}^\downarrow$ (with generalization) has a number of important advantages over PrIC3. First, $\text{AdjointPDR}^\downarrow$ does not require an oracle, nor an extra outer loop for the soundness property. Second, the negative chain of $\text{AdjointPDR}^\downarrow$ is more flexible than the obligation queue in PrIC3, and much smaller in some cases. Third, the nondeterminism in the heuristics of $\text{AdjointPDR}^\downarrow$ allows the construction of variants of the algorithm with relative ease.

Final conclusion

In conclusion, generalization in $\text{AdjointPDR}^\downarrow$ is both possible and practical, and highlights the effectiveness of $\text{AdjointPDR}^\downarrow$ in probabilistic model checking.

Future work

For future work we propose the following.

Symbolic representation. It could be a useful contribution to create a library which can efficiently handle symbolic representations of frames, and that also supports polynomial and exponential pieces. (The latter would negate the need for the affine approximation in the exponential generalization, for example.) Such a library should support at least all the basic operations on frames from Table 4.1. It should also be considered that the efficiency of the symbolic data structures in our current implementation is very sensitive to the specific encoding of the model. A change in encoding may lead to an explosion of the size of the data structures. One could look into sophisticated representations of frames which circumvent this problem.

Accepted models. Our implementation currently only accepts Markov chains with certain restrictions on the types of updates (see Chapter 4). The main reason for these restrictions is that it is not clear how to define Θ for updates that are not completely invertible or constant. One could look into how to define Θ for these kinds of updates. Furthermore, our symbolic representation and implementation could be extended to MDPs.

Generalizations. The performance of $\text{AdjointPDR}^\downarrow$ can likely benefit from better heuristics, and in particular better generalizations. It should be considered that the effectiveness of generalization in its current form, is very much tied to the specific encoding of the input program. We may have two equivalent models where $\text{AdjointPDR}^\downarrow$ terminates much faster for one of them because the encoding is more "fortunate". Furthermore, Batz *et al.*[5] proposed two more methods of generalization that we did not implement, *polynomial generalization* and *hybrid generalization*, with polynomial generalization being the most effective out of all their generalizations. One could adopt these into our implementation of $\text{AdjointPDR}^\downarrow$, although polynomial generalization is not possible with piecewise affine frames. Furthermore, PrIC3 has an optimization routine called *propagation* which could also be introduced into $\text{AdjointPDR}^\downarrow$ relatively easily.

Scaling refine heuristics. We noted in Section 4.4 that it is likely impossible to compute any of the **Refine** heuristics from Section 3.4 in a way that does not potentially scale poorly for large state spaces. We believe that it is not hard to come up with a **Refine** heuristic that does not have this problem.

Negative generalization. The negative sequence has an important downside. Suppose that a model is unsafe ($\lambda < \mathbb{P}_{\mathcal{M}, \max}(\Diamond B)(s_0)$). Then $\text{AdjointPDR}^\downarrow$ terminates (cf. Theorem 4), but the negative sequence may be much longer than the implementation can reasonably handle in practice. For example, in the Chain model with 10^{12} states, the negative sequence needs to have a length of 10^{12} in order to terminate negatively. If we wanted to apply a kind of generalization on the negative sequence to avoid this problem, it would need to be a negative *chain*, which is not really possible using lf-sets. At the very least, we would need a simple and computationally cheap comparison operator (similar to $\leq$) between two elements of the negative chain. One could look into different data structures that take on the role of the negative sequence, but do form a chain. The lower bound certificates for maximal reachability from [7, Section 4.3] might serve as a starting point.

Appendix A

Proofs

A.1 Background

Proposition 1. Let $F, G, H \in [0, 1]^S$.

1. Reflexive. For all $s \in S$, $F(s) \leq F(s)$, so $F \leq F$.
2. Transitive. Suppose $F \leq G$ and $G \leq H$ and let $s \in S$. Then $\forall s \in S$, $F(s) \leq G(s)$ and $G(s) \leq H(s)$. Then by transitivity of $\leq$ on real numbers in $[0, 1]$, it follows that $F(s) \leq H(s)$. Therefore, $F \leq H$.
3. Anti-symmetric. Suppose that $F \leq G$ and $G \leq F$. Then $\forall s \in S$, $F(s) \leq G(s) \leq F(s)$. By anti-symmetry of $\leq$ on real numbers in $[0, 1]$, it follows that $F = G$.

□

Proposition 2. Let $\mathcal{F}, \mathcal{G}, \mathcal{H} \in ([0, 1]^S)^\downarrow$.

1. Reflexive. By definition of subset inclusion, $\mathcal{F} \subseteq \mathcal{F}$.
2. Transitive. By transitivity of subset inclusion, if $\mathcal{F} \subseteq \mathcal{G}$ and $\mathcal{G} \subseteq \mathcal{H}$, then $\mathcal{F} \subseteq \mathcal{H}$.
3. Anti-symmetric. Suppose that $\mathcal{F} \subseteq \mathcal{G}$ and $\mathcal{G} \subseteq \mathcal{F}$. Then clearly, $\mathcal{F} = \mathcal{G}$.

□

Proposition 3.

1. For $([0, 1]^S, \leq)$ and $\bigvee$. Let $\mathcal{F} \subseteq [0, 1]^S$.
 - If $\mathcal{F} = \emptyset$, then $\bigvee \mathcal{F} = \mathbf{0}$. Then, (1) $\bigvee \mathcal{F} = \mathbf{0}$ is an upper bound of $\mathcal{F} = \emptyset$ since $\forall F \in \emptyset. F \leq \mathbf{0}$, and (2) for any frame $z \in [0, 1]^S$, $\bigvee \mathcal{F} = \mathbf{0} \leq z$.
 - If $\mathcal{F} \neq \emptyset$, then $(\bigvee \mathcal{F})(s) = \sup_{F \in \mathcal{F}} F(s)$ for $s \in S$. Then (1) $\bigvee \mathcal{F}$ is an upper bound of $\mathcal{F}$, since $\forall F \in \mathcal{F}$, we have that $\forall s \in S$, $F(s) \leq \sup_{F \in \mathcal{F}} F(s) = (\bigvee \mathcal{F})(s) \implies F \leq \bigvee \mathcal{F}$. Furthermore (2), let $z \in [0, 1]^S$, such that $(\forall y \in \mathcal{F}, y \leq z)$. Let $s \in S$, then $\forall y \in \mathcal{F}. y(s) \leq z(s)$. By definition of the supremum on real numbers in $[0, 1]$, it follows that $\sup_{y \in \mathcal{F}} y(s) \leq z(s)$. As a consequence, $\bigvee \mathcal{F} \leq z$.
2. For $(([0, 1]^S)^\downarrow, \subseteq)$ and $\bigcup$. Let $A \subseteq ([0, 1]^S)^\downarrow$.
 - If $A = \emptyset$, then $\bigcup A = \emptyset$. Then, (1) $\bigcup A = \emptyset$ is an upper bound of A since $\forall \mathcal{F} \in \emptyset. \mathcal{F} \subseteq \emptyset$, and (2) for any lf-set $\mathcal{F} \in ([0, 1]^S)^\downarrow$, $\bigcup A = \emptyset \subseteq \mathcal{F}$.
 - If $A \neq \emptyset$, then (1) for all $\mathcal{F} \in A$, $\mathcal{F} \subseteq \bigcup A$ by definition of set union. Furthermore (2), let $z \in A$, s.t. $\forall y \in Z. y \subseteq z$. Then for any frame $F \in [0, 1]^S$ we have the following.
If $\exists y \in A$. s.t. $F \in y$, then $F \in \bigcup A$ by definition of set union, and $F \in z$ (since $F \in y$ and $y \subseteq z$). If there is no y such that $F \in y$, then $F \notin \bigcup A$
(since $\neg \exists y \in A. F \in y \implies \forall y \in A. F \notin y \implies F \notin \bigcup A$). Therefore, $\bigcup A \subseteq z$.

□

Proposition 4. Similar proof to Proposition 3.

□

Proposition 5. By Propositions 1 and 2, $([0, 1]^S, \leq)$ and $(([0, 1]^S)^\downarrow, \subseteq)$ are posets. By Propositions 3 and 4, we know the following.

1. For $([0, 1]^S, \leq)$, the pointwise maximum gives the supremum $\bigvee$ with $\bigvee \emptyset = \mathbf{0}$ and the pointwise minimum $\bigwedge$ gives the infimum with $\bigwedge \emptyset = \mathbf{1}$. Both are defined on any subset of $[0, 1]^S$, hence $([0, 1]^S, \leq)$ is a complete lattice.
2. For $(([0, 1]^S)^\downarrow, \subseteq)$, the set union $\bigcup$ gives the supremum with $\bigcup \emptyset = \emptyset$ and the set intersection $\bigcap$ gives the infimum with $\bigcap \emptyset = [0, 1]^S$. Both are defined on any subset of $([0, 1]^S)^\downarrow$, hence $(([0, 1]^S)^\downarrow, \subseteq)$ is a complete lattice.

□

Theorem 1. We refer to [4, Theorem 2.1].

□

Lemma 1. We refer to [4, Lemma 2.4].

□

Proposition 6. Let $x, y \in X$, s.t. $x \sqsubseteq y$. Then $\Phi(x) \sqsubseteq \bigsqcup \{\Phi(x), \Phi(y)\} = \Phi(\bigsqcup \{x, y\}) = \Phi(y)$. Note that $\{x, y\}$ is directed.

□

Theorem 2. We refer to [4, Theorem 2.2].

□

Proposition 7.

1. We refer to [17].
2. Follows by (1) and Theorem 1.
3. Induction.

- Base case $\Phi^0(\mathbf{0}) = \mathbf{0} \leq \begin{cases} 1, & \text{if } s \in B \\ 0, & \text{otherwise} \end{cases} = \Phi(\mathbf{0})$.
- Inductive case. Suppose that $\Phi^n(\mathbf{0}) \leq \Phi^{n+1}(\mathbf{0})$.
Then by monotonicity $\Phi^{n+1}(\mathbf{0}) = \Phi(\Phi^n(\mathbf{0})) \leq \Phi(\Phi^{n+1}(\mathbf{0})) = \Phi^{n+2}(\mathbf{0})$.

□

Proposition 8.

1. Let a state $s_0 \in S$. This can be any state, not necessarily the fixed initial state. Induction.
 - Base case.

$$\begin{aligned}
\mathbb{P}_{\mathcal{M}, \max}^0(s_0) &= \max_{\sigma \in \Pi^{\text{HD}}} \left\{ \sum_{s_0 \dots s_{m-1} \in \text{Paths}_{\text{fin}}^1(\mathcal{M}, \sigma) \cap (S \setminus B)^* B} \mathbf{P}_\sigma(s_0 \dots s_{m-1}) \right\} \\
&= \sum_{s_0 \in B} \mathbf{P}_\sigma(s_0) && \text{(for any } \sigma \in \Pi^{\text{HD}}) \\
&= \begin{cases} 1 & \text{if } s_0 \in B \\ 0 & \text{otherwise} \end{cases} \\
&= \Phi(\mathbf{0})(s_0).
\end{aligned}$$

- Inductive case, $n > 0$, with IH: $\forall s \in S. \mathbb{P}_{\mathcal{M}, \max}^{\leq n}(\diamond B)(s) = \Phi^{n+1}(\mathbf{0})(s)$. Then $\forall s_0 \in S$,

$$\begin{aligned}
\mathbb{P}_{\mathcal{M}, \max}^{\leq n+1}(s_0) &= \max_{\sigma \in \Pi^{\text{HD}}} \left\{ \sum_{s_0 s_1 \dots s_m \in \text{Paths}_{\text{fin}}^{\leq n+2}(\mathcal{M}, \sigma) \cap (S \setminus B)^* B} \mathbf{P}_{\sigma}(s_0 s_1 \dots s_m) \right\} \\
&= \max_{\sigma \in \Pi^{\text{HD}}} \left\{ \mathcal{B} \left(\sum_{s_1 \dots s_n \in \text{Paths}_{\text{fin}}^{\leq n+1}(\mathcal{M}, \sigma) \cap (S \setminus B)^* B} P(s_0, \sigma(s_0), s_1) \cdot \mathbf{P}_{\sigma}(s_1 \dots s_n) \right) \right\} \\
&= \mathcal{B} \left(\max_{\sigma \in \Pi^{\text{HD}}} \left\{ \sum_{s_1 \dots s_n \in \text{Paths}_{\text{fin}}^{\leq n+1}(\mathcal{M}, \sigma) \cap (S \setminus B)^* B} P(s_0, \sigma(s_0), s_1) \cdot \mathbf{P}_{\sigma}(s_1 \dots s_n) \right\} \right) \\
&= \mathcal{B} \left(\max_{\sigma \in \Pi^{\text{HD}}} \left\{ \sum_{s_1 \in S} P(s_0, \sigma(s_0), s_1) \cdot \left(\sum_{s_1 \dots s_m \in \text{Paths}_{\text{fin}}^{\leq n+1}(\mathcal{M}, \sigma) \cap (S \setminus B)^* B} \mathbf{P}_{\sigma}(s_1 \dots s_m) \right) \right\} \right) \\
&= \mathcal{B} \left(\max_{a \in \text{Act}} \left\{ \sum_{s_1 \in S} P(s_0, a, s_1) \cdot \max_{\sigma \in \Pi^{\text{HD}}} \left\{ \sum_{s_1 \dots s_m \in \text{Paths}_{\text{fin}}^{\leq n+1}(\mathcal{M}, \sigma) \cap (S \setminus B)^* B} \mathbf{P}_{\sigma}(s_1 \dots s_m) \right\} \right\} \right) \quad (*) \\
&= \mathcal{B} \left(\max_{a \in \text{Act}} \left\{ \sum_{s_1 \in S} P(s_0, a, s_1) \cdot \mathbb{P}_{\mathcal{M}, \max}^{\leq n}(\diamond B)(s_1) \right\} \right) \\
&= \mathcal{B} \left(\max_{a \in \text{Act}} \left\{ \sum_{s_1 \in S} P(s_0, a, s_1) \cdot \Phi^{n+1}(\mathbf{0})(s_1) \right\} \right) \quad (\text{IH}) \\
&= \begin{cases} 1 & \text{if } s_0 \in B \\ \max_{a \in \text{Act}} \{ \sum_{s_1 \in S} P(s_0, a, s_1) \cdot (\Phi^{n+1}(\mathbf{0}))(s_1) \} & \text{otherwise} \end{cases} \\
&= \Phi(\Phi^{n+1}(\mathbf{0}))(s_0) \\
&= \Phi^{n+2}(\mathbf{0})(s_0)
\end{aligned}$$

where

$$\mathcal{B}(p) = \begin{cases} 1 & \text{if } s_0 \in B \\ p & \text{otherwise} \end{cases}$$

(*) Note that this step works since we are considering history-dependent policies.

2.

$$\mu \Phi =^{(*)} \bigvee \{ \Phi^n(\mathbf{0}) \}_n = \bigvee \{ \Phi^{n+1}(\mathbf{0}) \}_n = \lim_{n \rightarrow \infty} \Phi^{n+1}(\mathbf{0}) \stackrel{(1)}{=} \lim_{n \rightarrow \infty} \mathbb{P}_{\mathcal{M}, \max}^{\leq n}(\diamond B) = \mathbb{P}_{\mathcal{M}, \max}(\diamond B),$$

where (*) follows from Theorem 2.

□

Proof Principle 1. Follows directly from applying Lemma 1.

□

Proposition 9.

1. Suppose that $\mathcal{F} \subseteq \mathcal{G}$. Then $\Psi(\mathcal{F}) = \{F : \Phi(F) \in \mathcal{F}\} \subseteq \{F : \Phi(F) \in \mathcal{G}\} = \Psi(\mathcal{G})$.

2.

$$\begin{aligned}
\Psi(\mathcal{G}) &= \{F : \Phi(F) \in \mathcal{G}\} \\
&= \{F : (\bigvee_{\sigma \in \Pi^{\text{MD}}} \Phi_\sigma(F)) \in \mathcal{G}\} \tag{*} \\
&= \{F : \Phi_{\sigma_0}(F) \in \mathcal{G} \text{ and } \Phi_{\sigma_1}(F) \in \mathcal{G} \dots \Phi_{\sigma_n}(F) \in \mathcal{G}\} \quad (\text{where } \Pi^{\text{MD}} = \{\sigma_0, \dots, \sigma_n\}, +) \\
&= \bigcap_{\sigma \in \Pi^{\text{MD}}} \{\Phi_\sigma(F) \in \mathcal{G}\} \\
&= \bigcap_{\sigma \in \Pi^{\text{MD}}} \Psi_\sigma(\mathcal{G}).
\end{aligned}$$

(*) Note that it follows from the definition of Φ_σ that there exists a policy $\sigma^* \in \Pi^{\text{MD}}$ (with $\forall s \in S. \sigma^*(s) = \text{argmax}_{a \in \text{Act}} \{\sum_{s' \in S} P(s, \sigma^*(s), s') F(s')\}$) such that $\Phi(F) = \Phi_{\sigma^*}(F) \geq \Phi_\sigma(F)$ for all $\sigma \in \Pi^{\text{MD}}$. Therefore, $\Phi(F) = \Phi_{\sigma^*}(F) = \bigvee_{\sigma \in \Pi^{\text{MD}}} \Phi_\sigma(F)$.

(+) We show that $x = (\bigvee_{\sigma \in \Pi^{\text{MD}}} \Phi_\sigma(F)) \in \mathcal{G} \iff \{F : \Phi_{\sigma_0}(F) \in \mathcal{G} \text{ and } \Phi_{\sigma_1}(F) \in \mathcal{G} \dots \Phi_{\sigma_n}(F) \in \mathcal{G}\}$.

$\Rightarrow$ If $x \in \mathcal{G}$, then for any $i \in \{0 \dots n\}$, it follows by $\Phi_{\sigma_i}(F) \leq x$ and definition of lf-sets, that $\Phi_{\sigma_i}(F) \in \mathcal{G}$.

$\Leftarrow$ Recall the policy $\sigma^* \in \Pi^{\text{MD}}$ from (*). If $x \notin \mathcal{G}$, then $\Phi_{\sigma^*}(F) = x \notin \mathcal{G}$, and therefore there is some $i \in \{0 \dots n\}$ such that $\Phi_{\sigma_i}(F) \notin \mathcal{G}$.

□

Proposition 10.

1. Induction.

- Base case. $\Psi^0(\mathcal{G}) = \mathcal{G} = \{F : F \in \mathcal{G}\} = \{F : \Phi^0(F) \in \mathcal{G}\}$.
- Inductive case with IH: $\Psi^n(\mathcal{G}) = \{F : \Phi^n(F) \in \mathcal{G}\}$, then

$$\begin{aligned}
\Psi^{n+1}(\mathcal{G}) &= \Psi(\Psi^n(\mathcal{G})) \\
&= \Psi(\{F : \Phi^n(F) \in \mathcal{G}\}) \tag{IH} \\
&= \{G : \Phi(G) \in \{F : \Phi^n(F) \in \mathcal{G}\}\} \\
&= \{G : \Phi^{n+1}(G) \in \mathcal{G}\}.
\end{aligned}$$

$$2. \Phi^n(F) \in \mathcal{G} \iff F \in \{G : \Phi^n(G) \in \mathcal{G}\} \iff {}^{(1)}F \in \Psi^n(\mathcal{G}).$$

$$3. \Phi^n(F) \leq G \iff \Phi^n(F) \in \{F : F \leq G\} \iff \Phi^n(F) \in G^\downarrow \iff {}^{(2)}F \in \Psi^n(G^\downarrow).$$

□

Proposition 11.

$$\begin{aligned}
\Psi^{n+1}(P^\downarrow) &= \emptyset \\
&\iff \neg \exists F. F \in \Psi^{n+1}(P^\downarrow) \\
&\iff \neg \exists F. \Phi^{n+1}(F) \leq P \tag{Proposition 10(3)} \\
&\iff \Phi^{n+1}(\mathbf{0}) \not\leq P \tag{*} \\
&\iff \mathbb{P}_{\mathcal{M}, \max}^{\leq n}(\Diamond B) \not\leq P \tag{by Proposition 8(2)}
\end{aligned}$$

(*) $\Leftarrow$ Suppose $\Phi^{n+1}(\mathbf{0}) \not\leq P$. Let $F \in [0, 1]^S$. Since $\mathbf{0} \leq F$, and monotonicity of Φ , $\Phi^{n+1}(\mathbf{0}) \leq \Phi^{n+1}(F)$. Hence, $\Phi^{n+1}(\mathbf{0}) \not\leq P$ implies $\Phi^{n+1}(F) \not\leq P$. Then $\forall F \in [0, 1]^S. \Phi^{n+1}(F) \not\leq P$, or equivalently $\neg \exists F \in [0, 1]^S. \Phi^{n+1}(F) \leq P$.

$\Rightarrow$ $\neg \exists F \in [0, 1]^S. \Phi^{n+1}(F) \leq P$, then clearly $\Phi^{n+1}(\mathbf{0}) \not\leq P$.

□

Proof Principle 2. Suppose that we have a sequence $\mathcal{G}_i$ where $\mathcal{G}_i \supseteq \Psi^n(P^\downarrow)$ for each i , and $\mathcal{G}_j = \emptyset$ for some j . Then $\emptyset = \mathcal{G}_j \supseteq \Psi^j(P^\downarrow) = \emptyset$. Then from Proposition 11, it follows that $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B) \not\leq P$. □

A.2 AdjointPDR[↓]

Lemma 2. Can be proven by checking all the invariants. □

Proposition 12. If $\mathcal{G}_m$ exists, then $F_m \notin \mathcal{G}_m \supseteq \Psi^{n-m-1}(P^\downarrow)$. Then from Proposition 10(3), $\Phi(F_m)^{n-m-1} \not\leq P$. □

We have two technical Lemmas regarding the configuration.

■ **Lemma 4.** If P0 and P4 hold for a configuration c , then so does P5.

Proof. Induction on $i \in \{0 \dots n-2\}$. Base case ($i = 0$) follows from P0, $\Phi^0(\mathbf{0}) = \mathbf{0} \leq F_1$.

For the inductive case $(i+1) \in \{1 \dots n-2\}$, $\Phi^i(\mathbf{0}) \leq F_{i+1}$ (IH), and $\Phi(F_{i+1}) \leq F_{i+2}$ (P4). Then by monotonicity of Φ :

$$\Phi^{i+1}(\mathbf{0}) = \Phi(\Phi^i(\mathbf{0})) \leq \Phi(F_{i+1}) \leq F_{i+2}.$$

□

■ **Lemma 5.** If N1 and N2 hold for a configuration c , then so does N3.

Proof. Proof by induction on $m \in \{k \dots n-1\}$. Base case ($m = n-1$) follows from N1, since $P \in \mathcal{G}_{n-1} \implies P^\downarrow = \Psi^0(P^\downarrow) = \Psi^{n-1-m}(P^\downarrow) \subseteq \mathcal{G}_{n-1}$ by definition of $()^\downarrow$ and lf-sets.

For the inductive case $(m-1) \in \{k \dots n-2\}$, we have that $\Psi^{n-1-m}(P^\downarrow) \subseteq \mathcal{G}_m$ (IH), and $\Psi(\mathcal{G}_m) \subseteq \mathcal{G}_{m-1}$ (N2). Then by monotonicity of Ψ :

$$\Psi^{n-1-(m-1)}(P^\downarrow) = \Psi^{n-m}(P^\downarrow) = \Psi(\Psi^{n-m-1}(P^\downarrow)) \subseteq \Psi(\mathcal{G}_m) \subseteq \mathcal{G}_{m-1}.$$

□

Now we prove that each step preserves the invariants. In these proofs we will refer to $c = (F \parallel \mathcal{G})_{n,k}$, the configuration preceding the application of the step with, and $c' = (F' \parallel \mathcal{G}')_{n',k'}$, the resulting configuration after applying the step.

■ **Lemma 6.** The invariants in Table 3.1 are preserved in the **Extend** step.

Proof. Assume (a) $\mathcal{G} = \epsilon$ and (b) $F_{n-1} \leq P$. We update $n' = n+1$, $k' = k+1$ and set $F'_{n'-1} = \mathbf{1}$. We check every invariant that is not completely trivial.

P1 $k' = k+1 \geq 1$, and $k' = k+1 \leq n+1 = n'$.

P3 Since $F_{n-1} \leq P$, it follows by P2 (on c) that for $0 \leq j \leq n-1 = n'-2$, $F'_j \leq P$.

P4 By P4 (on c), for $1 \leq j \leq n-2 = n'-3$, $\Phi(F'_j) \leq F'_{j+1}$. Clearly, $\Phi(F'_{n'-2}) \leq \mathbf{1}$, so combining this gives for $1 \leq j \leq n'-2$, $\Phi(F'_j) \leq F'_{j+1}$.

P5 Follows by Lemma 4, using P0 and P4 (on c').

□

■ **Lemma 7.** The invariants in Table 3.1 are preserved in the **Investigate** step.

Proof. Assume (a) $\mathcal{G} = \epsilon$ and (b) $F_{n-1} \not\leq P$. We update $n' = n$, $k' = n-1$ and set $\mathcal{G}'_{n-1} = \mathcal{Z}$ subject to (i) $F_{n-1} \notin \mathcal{Z}$ and (ii) $P \in \mathcal{Z}$. We check every invariant that is not completely trivial.

P1 If $n' = n$, $k' = n-1 \geq 1$, and $k' = n-1 \leq n$.

N1 follows By (ii).

PN follows by (i).

□

■ **Lemma 8.** The invariants in Table 3.1 are preserved in the **Refine** step.

Proof. Assume (a) $\mathcal{G} \neq \emptyset$ and (b) $\Phi(F_{k-1}) \in \mathcal{G}_k$. We update $n' = n$ and $k' = k+1$ and set $F'_j = F_j \wedge z$ for $0 \leq j \leq k$, subject to (i) $z \in \mathcal{G}_k$ and (ii) $\Phi(F_{k-1} \wedge z) \leq z$. We check all invariants that are not completely trivial.

P1 Since $\mathcal{G} \neq \epsilon$ (a), it follows that $k < n$. Then $k' = k + 1 \leq n = n'$.

P2 By Lemma 4, using P0 and P4 (on c').

P3 $F'_j = F_j \wedge z \leq F_j \leq P$.

P4 • Let j such that $1 \leq j \leq k$. Clearly, $\Phi(F_{j-1} \wedge z) \leq z \wedge F_j \implies \Phi(F_{k-1} \wedge z) \leq z$ by monotonicity.

$$\begin{aligned} \text{Furthermore, } \Phi(F_{k-1} \wedge z) \leq z &\implies \Phi(F_{j-1} \wedge z) \leq z & (*) \\ &\implies \Phi(F_{j-1} \wedge z) \leq z \text{ and } \Phi(F_{j-1}) \leq F_k & (\text{by P4 on } c) \\ &\implies \Phi(F_{j-1} \wedge z) \leq z \wedge F_k. \end{aligned}$$

This means that we have the equivalence

$$\forall j. 1 \leq j \leq k. \Phi(F_{j-1} \wedge z) \leq z \wedge F_k \iff \Phi(F_{k-1} \wedge z) \leq z,$$

hence by (ii), we know that $\forall j. 1 \leq j \leq k. \Phi(F_{j-1} \wedge z) \leq z \wedge F_k$, and as consequence

$$\Phi(F'_{j-1}) = \Phi(F_{j-1} \wedge z) \leq F_j \wedge z = F_j.$$

- Now let $j = k + 1$. Then by monotonicity and P4 (on c), $\Phi(F'_j) = \Phi(F_{j-1} \wedge z) \leq \Phi(F_{j-1}) \leq F_j = F'_j$.
- Now let j such that $k + 1 < j \leq n$, then by P4 (on c), $\Phi(F'_j) = \Phi(F_j) \leq F_{j+1} = F'_{j+1}$.

(*) Since $j \leq k$, it follows that $F_{j-1} \leq F_{k-1}$ by P2. Then by monotonicity, $\Phi(F_{j-1} \wedge z) \leq \Phi(F_{k-1} \wedge z)$.

P5 By Lemma 4, using P0 and P4 (on c').

□

■ **Lemma 9.** If $\mathcal{G}_k \neq \emptyset$, the invariants in Table 3.1 are preserved in the **Search** step.

Proof. Assume (a) $\mathcal{G} \neq \epsilon$ and (b) $\Phi(F_{k-1}) \notin \mathcal{G}_k$, and (c) $\mathcal{G}_k \neq \emptyset$. We update $n' = n$, $k' = k - 1$ and set $\mathcal{G}'_{k-1} = \mathcal{Z}$, subject to (i) $F_{k-1} \notin \mathcal{Z}$ and (ii) $\mathcal{Z} \supseteq \Psi(\mathcal{G}_k)$. We check every invariant that is not completely trivial.

P1 $n' = n \geq 3$. By P1 (on c), $1 \leq k \leq n$ Furthermore, by contradiction, $k > 1$:

Suppose that $k = 1$. Then by P1, $F_{k-1} = F_0 = \perp$. Then $\Phi(\perp) = \mathbf{0} \notin \mathcal{G}_k$, hence $\mathcal{G}_k = \emptyset$, but this is a contradiction with (c).

Then $k' = k - 1 \geq 1$.

N2 Follows by (ii).

N3 Follows by Lemma 5, using N1 and N2 (on c').

PN Follows by (i).

□

■ **Lemma 10.** For configuration c , if $c_0 \rightarrow^* c$ then c satisfies the invariants from Table 3.1.

Proof. Induction.

Base case ($c = c_0$) is given by Lemma 2.

Inductive case ($c \rightarrow^* c' \rightarrow c$), where c' satisfies the invariants from Table 3.1. If $\mathcal{G}'_{k'} = \emptyset$ (on configuration c'), then $c' \not\rightarrow c$ because $\text{AdjointPDR}^\downarrow$ returns, hence $\mathcal{G}'_{k'} \neq \emptyset$. Then from Lemmas 6, 7, 8, and 9, it follows that c satisfies the invariants from Table 3.1. □

Theorem 3. Suppose the result is true. Then either we returned at line 17 or line 4. In either case, we are in a configuration c , where $c_0 \rightarrow^* c$, so by Lemma 10, the invariants of Table 3.1 hold for c . If we returned at line 17, we found a frame $x = F_k \wedge z$ such that $\Phi(x) \leq x$ and $x \leq P$. If we returned at line 4, we found a j such that $F_j = F_{j+1}$. Then by P4, $\Phi(F_j) \leq F_j$, and by P2, $F_j \leq P$. In either case, by Proof Principle 1, it follows that $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B) \leq P$.

Suppose the result is false. Then we returned at line 5. We are in a configuration c , where $c_0 \rightarrow^* c$, so by Lemma 10, the invariants of Table 3.1 hold for c . Then $\mathcal{G}_k = \emptyset$, and by P3, $\Psi^{n-1-i}(P^\downarrow) \subseteq \mathcal{G}_i$ for $k \leq i \leq n - 1$. Then by Proof Principle 2, it follows that $\mathbb{P}_{\mathcal{M}, \max}(\Diamond B) \not\leq P$. □

Corollary 1. From Proposition 14. □

Definition 56. The order $\sqsubset$ on configurations is given by

$$(F\|\mathcal{G})_{n,k} \sqsubset (F'\|\mathcal{G}')_{n',k'} \iff n < n' \tag{A.1}$$

$$\vee (n = n') \wedge (\forall i. 1 \leq i \leq n-1 \rightarrow F_i \geq F'_i) \wedge (k' < k) \tag{A.2}$$

$$\vee (n = n') \wedge (\forall i. 1 \leq i \leq n-1 \rightarrow F_i \geq F'_i) \wedge (\exists i. 1 \leq i \leq n-1 \wedge F_i > F'_i). \tag{A.3}$$

■ **Lemma 11.** $\sqsubset$ is a strict partial order, i.e. it is irreflexive and transitive.

Proof.

1. Irreflexive. For configuration c compared to itself, none of the equations A.1, A.2 or A.3 are satisfied because $n = n'$, $k = k'$, and $\forall i. 1 \leq i \leq n \rightarrow F_i \not\geq F'_i$ respectively. Then $(F\|\mathcal{G})_{n,k} \not\sqsubset (F'\|\mathcal{G})_{n,k}$.
2. Transitive. Check all cases for $c \sqsubset c'$ and $c' \sqsubset c''$:
 - (a) Suppose $n < n'$. Then, $n < n' \leq n''$ so $c \sqsubset c''$ by A.1.
 - (b) Suppose $n = n'$ and $(\forall i. 1 \leq i \leq n-1 \rightarrow F_i \geq F'_i) \wedge (k' < k)$. Then
 - i. If $n' < n''$, then $c \sqsubset c'$ by A.1
 - ii. If $n' = n''$, then $(\forall i. 1 \leq i \leq n'-1 \rightarrow F'_i \geq F''_i)$ and by transitivity of $\geq$, it follows that $\forall i. 1 \leq i \leq n'-1 \rightarrow F_i \geq F''_i$. Furthermore, $k'' \leq k' < k$, so $c' \sqsubset c''$ by A.2.
 - (c) Suppose $n = n'$ and $(\forall i. 1 \leq i \leq n-1 \rightarrow F_i \geq F'_i) \wedge (\exists i. 1 \leq i \leq n-1 \wedge F_i > F'_i)$. Then
 - i. If $n' < n''$, then $c \sqsubset c'$ by A.1
 - ii. If $n' = n''$, then $(\forall i. 1 \leq i \leq n'-1 \rightarrow F'_i \geq F''_i)$ and by transitivity of $\geq$, it follows that $\forall i. 1 \leq i \leq n'-1 \rightarrow F_i \geq F''_i$. And similarly, by transitivity of $\geq$, it follows that $\exists i. 1 \leq i \leq n-1 \wedge F_i > F'_i \geq F''_i$. Then $c \sqsubset c'$ by A.3.

□

■ **Lemma 12.** For configurations c and c' , if $c_0 \rightarrow^* c \rightarrow c'$, then $c \sqsubset c'$.

Proof. Let $c = (F\|G)_{n,k}$ and $c' = (F'\|G')_{n',k'}$. Then there are four cases:

1. $c \xrightarrow{Ex} c'$. Then $n' = n + 1$ and therefore $c \sqsubset c'$ by A.1.
2. $c \xrightarrow{In} c'$. Then $n = n'$, $(\forall i. 1 \leq i \leq n-1 \rightarrow F_i = F'_i)$, and $k' = k - 1$, therefore $c \sqsubset c'$ by A.2.
3. $c \xrightarrow{Se} c'$. Same proof as for $c \xrightarrow{In} c'$.
4. $c \xrightarrow{Re} c'$. Note that $F_k \notin \mathcal{G}_k$ by PN (we know that PN holds on c since $c_0 \rightarrow^* c$ and Lemma 10). Then since $z \in \mathcal{G}_k$, it follows that $F_k \not\leq z$ by definition of lf-sets. Then $F_k \wedge z \neq F_k$, (the contrapositive of $F_k \wedge z = F_k$ implies that $F_k \leq z$), and $F_k \wedge z \leq F_k$. Then $n = n'$ and $(\forall i. 1 \leq i \leq n-1 \rightarrow F_i \geq F'_i)$, and $(\exists i. 1 \leq i \leq n-1 \wedge F_i > F'_i, \text{ for } i = k)$. Therefore $c \sqsubset c'$ by A.3.

□

Proposition 13. By Lemma 12, and the transitivity of $\sqsubset$, it follows that $c \sqsubset c'$. If $c = c'$, then we contradict the irreflexive property of $\sqsubset$, so $c \neq c'$. □

Given a configuration c_m , we denote the i th element of the positive chain and negative sequence as F_i^m and $\mathcal{G}_i^m$ respectively. Similarly, we denote the values of n and k for c_m as n^m and k^m .

■ **Lemma 13.** For configurations $c_1, c_2, c_3, c_4 \in \mathbf{Conf}$, if $c_0 \rightarrow^* c_1 \xrightarrow{In} c_2 \rightarrow^* c_3 \xrightarrow{In} c_4$, then $n^2 = n^4 \implies \mathcal{G}_k^2 \neq \mathcal{G}_k^4$.

Proof. We prove a slightly stronger property from which the Lemma follows.

For $m \in \mathbb{N}$ and configurations $c_1, c_2, c_3, c_4 \in \mathbf{Conf}$ where $c_0 \rightarrow^* c_1 \xRightarrow{In} c_2 \rightarrow^* c_3 \xRightarrow{In} c_4$, such that $c_3 \rightarrow c_4$ is the m th **Investigate** step, then $n^2 = n^4$ implies that (i) $F_{n-1}^3 \in \mathcal{G}_k^2 \wedge$ (ii) $F_{n-1}^3 \notin \mathcal{G}_k^4$.

- Base case ($m = 0$). If $c_3 \rightarrow c_4$ is the first (0th) **Investigate** step, then there are no c_1, c_2 such that $c_1 \xRightarrow{In} c_2 \rightarrow^* c_3$ and the property vacuously holds.
- Inductive case ($m + 1$). We inductively assume that the property holds up to m , and prove it for $m + 1$. We use a, b, e, f for the configurations where $a \rightarrow b$ ranges over **Investigate** steps $0 \dots m$, and $e \rightarrow f$ is the $m + 1$ th **Investigate** step.

$$c_0 \rightarrow^* \dots \rightarrow^* a \xRightarrow{In} b \rightarrow^* e \xRightarrow{In} f$$

Now it suffices to prove that $F_{n-1}^e \in \mathcal{G}_{n-1}^b$ (i) and $F_{n-1}^e \notin \mathcal{G}_{n-1}^f$ (ii). The other cases are accounted for by the inductive hypothesis.

Since $k^f = k^b = n^f - 1$, there must be at least one configuration d with $k^d = k^f - 1 = k^b - 1$, which triggers a **Refine** step. Let d denote the first such **Refine** step, and let g denote the configuration that results from the step.

$$c_0 \rightarrow^* \dots \rightarrow^* a \xRightarrow{In} b \rightarrow^* d \xRightarrow{Re} g \rightarrow^* e \xRightarrow{In} f.$$

The **Investigate** step at a sets $\mathcal{G}_{n-1}^b$. Since no **Refine** steps with $k = k^d$ occurs between b and d , it follows that $F_{n-1}^b = F_{n-1}^d$ and $\mathcal{G}_{n-1}^b = \mathcal{G}_{n-1}^d$.

Then the **Refine** step at d sets $F_{n-1}^g = F_{n-1}^d \wedge z = F_{n-1}^b \wedge z$, subject to $z \in \mathcal{G}_{n-1}^d$.

Furthermore, since $g \sqsubseteq e$ (cf. Lemma 12) and $n^g = n^e$, it follows that $F_{n-1}^e \leq F_{n-1}^g$.

By definition of lf-sets, $z \in \mathcal{G}_{n-1}^d \implies F_{n-1}^e \leq F_{n-1}^g = (F_{n-1}^b \wedge z) \in \mathcal{G}_{n-1}^d = \mathcal{G}_{n-1}^b$, so $F_{n-1}^e \in \mathcal{G}_{n-1}^b$ (i).

Then the **Investigate** step at e updates $\mathcal{G}_{n-1}^f = \mathcal{Z}$ subject to $F_{n-1}^e \notin \mathcal{Z} = \mathcal{G}_{n-1}^f$ (ii).

□

Lemma 14. For configurations $c_1, c_2, c_3, c_4 \in \mathbf{Conf}$, if $c_0 \rightarrow^* c_1 \xRightarrow{Se} c_2 \rightarrow^* c_3 \xRightarrow{Se} c_4$, then $(n^2 = n^4 \wedge k^2 = k^4) \implies \mathcal{G}_k^2 \neq \mathcal{G}_k^4$.

Proof. This can be proven using a similar induction as Lemma 13.

□

Lemma 15. When $P \not\leq \mathbb{P}_{\mathcal{M}, \max}(\Diamond B)$, the **Extend** step can only be applied a finite amount of times in a run of **AdjointPDR**[↓].

Proof. Since $P \not\leq \mathbb{P}_{\mathcal{M}, \max}(\Diamond B)$, by Proposition 11, it follows that there exists an i such that $\Phi^i(\mathbf{0}) \not\leq P$. From P5, it follows that $\Phi^{n-2}(\mathbf{0}) \leq F_{n-1}$. **Extend** can only be applied if $F_{n-1} \leq P$. Furthermore, **Extend** increments n by 1, and n is never decremented by other rules. Then if $n - 2 \geq i$, **Extend** can no longer be applied. As a consequence, **Extend** can be applied no more than i times. □

Theorem 4. Suppose that $P \not\leq \mathbb{P}_{\mathcal{M}, \max}(\Diamond B)$ and **AdjointPDR**[↓] does not terminate. Because of Lemma 15, it must reach an infinitely repeating sequence $\{c_i\}_i \in \mathbf{Conf}^\omega$ of configurations, that trigger only **Investigate**, **Search** and **Refine** steps. Let l be the constant value of n in $\{c_i\}_i$.

$$c_0, c_1, c_2, \dots \in \mathbf{Conf}^\omega \text{ such that } c_0 \rightarrow c_1 \rightarrow c_2 \rightarrow \dots$$

Furthermore, $\{c_i\}_i$ must contain an infinite amount of **Investigate** and/or **Search** steps, since **Refine** increments k while **Investigate** and **Search** decrement k , and $0 \leq k \leq l - 1$ for finite l .

Let h_1 and h_2 denote the In and Se heuristics. Now we show that for each $Z \in \text{choices}(h_1, h_2)$, there are at most l distinct configurations $c_j \in \{c_i\}_i$ where $h(c_j) = Z$.

Let $c_i \in \{c_i\}_i$.

1. Suppose that $h(c_i) = Z$ for some configuration $c_i \xrightarrow{In}$.
 - (a) Then by Lemma 13, there is no $c_j \in \{c_i\}_i$ with $c_j \neq c_i$ and $c_j \xrightarrow{In}$ and $h(c_j) = Z$.
 - (b) There are at most l distinct $c_j \neq c_i$ with $c_j \xrightarrow{Se}$ where $h(c_j) = Z$, see (2a).
2. Suppose that $h(c_i) = Z$ for some configuration $c_i \xrightarrow{Se}$.
 - (a) Then by Lemma 14, there is no $c_j \in \{c_i\}_i$ with $c_j \neq c_i$ and $c_j \xrightarrow{Se}$ and $h(c_j) = Z$ **and** $k^i = k^j$.
 Since all values of k fall in range $0 \leq k \leq l - 1$, it follows that there are at most l distinct configurations $c_j \in \{c_i\}_i$ with $c_j \neq c_i$ and $c_j \xrightarrow{Se}$ and $h(c_j) = Z$.
 - (b) There is at most one $c_j \in \{c_i\}_i$ with $c_j \neq c_i$ and $c_j \xrightarrow{In}$ and $h(c_j) = Z$, see (1a).

By assumption, the set $\text{choices}(h_1, h_2)$ is finite. Then since for each $Z \in \text{choices}(h_1, h_2)$, there are at most l distinct configurations $c_j \in \{c_i\}_i$ where $h(c_j) = Z$, it follows that $|\{c_i\}_i| \leq l \cdot |\text{choices}(h_1, h_2)|$, and $\{c_i\}_i$ is therefore finite. This is a contradiction since we defined $\{c_i\}_i$ to be infinite.

Assuming that $\text{AdjointPDR}^\downarrow$ does not terminate results in a contradiction, so $\text{AdjointPDR}^\downarrow$ terminates. $\square$

Corollary 2. From Proposition 14 and 16. $\square$

Proposition 14.

- **hIn.** $\mathcal{Z} = P^\downarrow = \{v : v \leq P\}$. Clearly, $P \in P^\downarrow = \mathcal{Z}$ (1). Since $F_{n-1} \not\leq P$, it follows by definition of lf-sets that $F_{n-1} \notin P^\downarrow = \mathcal{Z}$ (2).
- **hSe $_\Psi$.** $\mathcal{Z} = \Psi(\mathcal{G}_k)$. Since $\Phi(F_{k-1}) \notin \mathcal{G}_k$, it follows by Proposition 10(2), that $F_{k-1} \notin \Psi(\mathcal{G}_k) = \mathcal{Z}$ (1). Furthermore, $\Psi(\mathcal{G}_k) \subseteq \mathcal{Z}$ (2).
- **hSe.** $\mathcal{Z} = \Psi_{\sigma^*}(\mathcal{G}_k)$ for $\sigma^* \in \Pi^{\text{MD}}$, such that $\sigma^*(s) = \arg\text{-max}_{a \in \text{Act}} \{\sum_{s' \in S} P(s, a, s') \cdot F_{k-1}(s')\}$. Note that $\Phi(F_{k-1}) = \Phi_{\sigma^*}(F_{k-1})$. Then by Proposition 10(2), $\Phi_{\sigma^*}(F_{k-1}) \notin \mathcal{G}_k$ implies $F_{k-1} \notin \Psi_{\sigma^*}(\mathcal{G}_k) = \mathcal{Z}$ (1). By Proposition 9(2), $\Psi(\mathcal{G}_k) \subseteq \Psi_{\sigma^*}(\mathcal{G}_k) = \mathcal{Z}$ (2).
- **hRs.** $z = \Phi(F_{k-1})$. We already have that $z = \Phi(F_{k-1}) \in \mathcal{G}_k$ (1), and by monotonicity of Φ , from $F_{k-1} \wedge \Phi(F_{k-1}) \leq F_{k-1}$, it follows that $\Phi(F_{k-1} \wedge z) = \Phi(F_{k-1} \wedge \Phi(F_{k-1})) \leq \Phi(F_{k-1})$ (2).
- **hRm.** Note that $\sum_{w_s \neq 0} u_s \cdot w_s \leq r$ implies that $\sum_{w_s \neq 0} z_s \cdot w_s \leq r$, and therefore $z \in \mathcal{G}_k = \{v : v \cdot w \leq r\}$ (1). Let $s \in S$. If $w_s = 0$, then $\Phi(F_{k-1})(s) \leq z(s)$ and if $w_s \neq 0$, then $\Phi(F_{k-1})(s) = z(s)$, hence by monotonicity of Φ , $\Phi(F_{k-1}) \leq z \implies \Phi(F_{k-1} \wedge z) \leq z$ (2).
- **hRb.** If $\mathcal{Z}_k = \emptyset$, then $z = \Phi(F_{k-1})$ (see the proof for hRs). Otherwise, $\mathcal{Z}_k \neq \emptyset$. Recall that $\bigwedge \mathcal{Z}_k = (\bigwedge \{d \in \mathcal{G}_k : d \text{ is almost 0-1-valued} \wedge (\sum_{s \in S} d_s w_s) = r \wedge \Phi(F_{k-1}) \leq d\})$. It follows that $\Phi(F_{k-1}) \leq (\bigwedge \mathcal{Z}_k)$. Furthermore note that for any $d \in \mathcal{G}_k$, $d \leq \bigwedge \mathcal{Z}_k$. Then by definition of lf-sets, $(\bigwedge \mathcal{Z}_k) \in \mathcal{G}_k$.

Now $z(s) = \begin{cases} (\bigwedge \mathcal{Z}_k)(s), & \text{if } w_s \neq 0 \text{ and } \mathcal{Z}_k \neq \emptyset \\ \Phi(F_{k-1})(s), & \text{otherwise} \end{cases}$, and $\Phi(F_{k-1}) \leq (\bigwedge \mathcal{Z}_k)$, so $\Phi(F_{k-1}) \leq z \leq (\bigwedge \mathcal{Z}_k)$.

Then by definition of lf-sets, $z \in \mathcal{G}_k$ (1). And by monotonicity, $\Phi(F_{k-1} \wedge z) \leq z$ (2).

- **hR $_{01}$.** Note that by definition, $z = \text{hR}_{01}((F \parallel \mathcal{G})_{n,k}) \geq \text{hRb}((F \parallel \mathcal{G})) \geq \Phi(F_{k-1})$, so by monotonicity, $\Phi(F_{k-1} \wedge z) \leq z$ (2). Furthermore, z only differs from $\text{hRb}((F \parallel \mathcal{G}))$ on coordinates $s \in S$ where $z_s = 0$, so from $\text{hRb}((F \parallel \mathcal{G})) \in \mathcal{G}_k$ (see hRb) it follows that $z \in \mathcal{G}_k$ (2). $\square$

Lemma 16. Let $\mathcal{G}_k = \{v : v \cdot w \leq r\}$, $\sigma \in \Pi^{\text{MD}}$, $w \in [0, 1]^S$, $t = r - \sum_{s \in B} w(s)$,

$$\Theta_\sigma(F)(s') = \sum_{s \in S} P(s, \sigma(s), s') \cdot F(s), \quad \text{and } u(s) = \begin{cases} w(s) & \text{if } s \notin B \\ 0 & \text{if } s \in B. \end{cases}$$

Then $\Psi_\sigma(\mathcal{G}) = \{v : \Theta_\sigma(u) \cdot v \leq t\}$.

Proof. For the notation in this proof, let $P(s, s') = P(s, \sigma(s), s')$ for all $s, s' \in S$ and let $\Phi = \Phi_\sigma$ and $\Psi = \Psi_\sigma$.

$$\begin{aligned} v \in \Psi(\mathcal{G}_k) &\iff \Phi(v) \in \mathcal{G}_k && \text{(Proposition 10(2))} \\ &\iff w \cdot \Phi(v) \leq r && \text{(def. } \mathcal{G}_k) \\ &\iff \Theta_\sigma(u) \cdot v + \sum_{s \in B} w(s) \leq r && (*) \\ &\iff \Theta_\sigma(u) \cdot v \leq r - \sum_{s \in B} w(s) \\ &\iff \Theta_\sigma(u) \cdot v \leq t \\ &\iff v \in \{v : \Theta_\sigma(u) \cdot v \leq t\} \end{aligned}$$

(*) We show that $w \cdot \Phi(v) = \Theta_\sigma(u) \cdot v + \sum_{s \in B} w(s)$.

$$\begin{aligned} w \cdot \Phi(v) &= \left(\sum_{s \in S \setminus B} w(s) \cdot \Phi(v)(s) \right) + \left(\sum_{s \in B} w(s) \cdot \Phi(v)(s) \right) \\ &= \left(\sum_{s \in S \setminus B} w(s) \cdot \Phi(v)(s) \right) + \sum_{s \in B} w(s) && \text{(because if } s \in B, \text{ then } \Phi(w)(s) = 1) \\ &= \left(\sum_{s \in S} u(s) \cdot \Phi(v)(s) \right) + \sum_{s \in B} w(s) \\ &= \left(\sum_{s \in S} u(s) \cdot \left(\sum_{s' \in S} P(s, s') \cdot v(s') \right) \right) + \sum_{s \in B} w(s) \\ &= \left(\sum_{s' \in S} \left(\sum_{s \in S} P(s, s') \cdot u(s) \right) \cdot v(s') \right) + \sum_{s \in B} w(s) \\ &= \left(\sum_{s' \in S} \Theta_\sigma(u)(s') \cdot v(s') \right) + \sum_{s \in B} w(s) \\ &= \Theta_\sigma(u) \cdot v + \sum_{s \in B} w(s) \end{aligned}$$

□

Proposition 15.

- **hIn.** Recall that $P = [\lambda, 1, \dots, 1]$ for some $\lambda \in [0, 1]$.
Then $P^\downarrow = \{v : v \leq P\} = \{v : v_0 \leq \lambda\} = \{v : [1, 0, \dots, 0] \cdot v \leq \lambda\}$.
- **hSe (and hSe_Ψ).** From Lemma 16.
- **hRb and hR₀₁.** From Proposition 25, $\mathcal{Z}_k = \emptyset \iff b = \text{false}$, and $\mathcal{Z}_k \neq \emptyset \implies c = \bigwedge \mathcal{Z}_k$.
- **hRs and hRm.** We did not specify a computation (using :=).

□

Proposition 16. We need to prove that for any $l \in \mathbb{N}$, the set

$$\text{choices}(\{\text{hIn}, \text{hSe}\}) = \{Z : ((c \xrightarrow{In} \wedge \text{hIn}(c) = Z) \vee (c \xrightarrow{Se} \wedge \text{hSe}(c) = Z)) \wedge c_0 \rightarrow^* c \wedge c^n = l\}$$

is finite.

The length of the negative sequence is bounded by l . By definition of hIn and hSe , it follows that $\mathcal{G}_j = \Psi_{\sigma_0}(\Psi_{\sigma_1}(\dots \Psi_{\sigma_{n-j-1}}(P^\downarrow) \dots))$ for any $j \in \{k \dots n-1\}$. Then since there are finite $|\Pi^{\text{MD}}|$ policies, it follows that $|\text{choices}(\{\text{hIn}, \text{hSe}\})| \leq |\Pi^{\text{MD}}|^l$, which is finite. $\square$

Proposition 17. Simple to prove by looking at the definitions of hIn and hSe . $\square$

Proposition 18. Recall that $z = \text{round}_c(\Phi(F_{k-1}))$.

- Suppose that $\Phi(z) \leq z$ and $z \in \mathcal{G}_k$. Then by definition of round_c , $\Phi(F_{k-1}) \leq z$, and by monotonicity $\Phi(F_{k-1} \wedge z) \leq z$ (1). We already assumed that $z \in \mathcal{G}_k$ (2).
- Otherwise, refer to the proof of hRs in Proposition 14.

$\square$

Theorem 5. With heuristic hRs_c , we end up in a cycle of applying **Investigate**, **Refine** and **Extend** steps. This means that $F_{k-1} = \Phi^l(\mathbf{0})$ for increasing $l \in \mathbb{N}$. At some point, we have $F_{k-1} = \Phi^m(\mathbf{0})$ for high enough m such that $\text{hRs}_c((F \parallel \mathcal{G})_{k,n}) = \text{round}_c(\Phi(F_{k-1})) = \text{round}_c(\Phi^{m+1}(\mathbf{0})) = \text{round}_c(\mu\Phi) = \mu\Psi$. Then $\Phi(\text{round}_c(\mu\Phi)) \leq \text{round}_c(\mu\Phi)$, and line 17 of $\text{AdjointPDR}^\downarrow$ returns true. $\square$

A.3 Symbolic representation

Proposition 19.

1. Suppose that $\exists i. r_i < 0$. Then $\text{sem}(\mathcal{G}) = \dots \cap \{V : \text{sem}(W_i) \cdot V \leq r_i\} \cap \dots = \dots \cap \emptyset \cap \dots = \emptyset$.
Suppose that $\forall i. r_i \geq 0$. Then for all i , $\mathbf{0} \in \{\text{sem}(W_i) \cdot V \leq r_i\}$, so $\text{sem}(\mathcal{G}) \neq \emptyset$.

2.

$$\begin{aligned} \text{sem}(F) \in \text{sem}(\mathcal{G}) &\iff \text{sem}(F) \in \{V \in [0, 1]^S : \text{sem}(W_0) \cdot V \leq r_0\} \cap \dots \cap \{V \in [0, 1]^S : \text{sem}(W_{k-1}) \cdot V \leq r_{k-1}\} \\ &\iff \forall i. \text{sem}(F) \cdot \text{sem}(W_i) \leq r_i \\ &\iff \forall i. F \cdot W_i \leq r_i \end{aligned}$$

$\square$

Recall that we fixed a predicate η , a program

$$\begin{aligned} A &= \{(\varphi_0, \mathbf{br}_0), \dots, (\varphi_{m-1}, \mathbf{br}_{m-1})\} \text{ with} \\ \mathbf{br}_i &= (\varphi_i, \{(p_{i,0}, u_{i,0}), \dots, (p_{i,v-1}, u_{i,v-1})\}) \text{ for } 0 \leq i < m, \end{aligned}$$

a Markov chain $\mathcal{M} = \text{sem}(A) = (S, P)$ and a set of bad states $B = \{s : \eta(s) = 1\}$. Operators Φ , Ψ and Θ are defined w.r.t. this transition function P and set of bad states B .

Proposition 20. Note that we abbreviate $\sum_{0 \leq i < m} \sum_{0 \leq j < b_i}$ to $\sum_{i,j}$.

$$\begin{aligned}
\Phi(\text{sem}(F))(s) &= \begin{cases} 1 & \text{if } s \in B \\ \sum_{s' \in S} \{\text{sem}(F)(s') \cdot P(s, s')\} & \text{otherwise} \end{cases} \\
&= \begin{cases} 1 & \text{if } s \in \{s : \eta(s) = 1\} \\ \sum_{i,j} \{\text{sem}(\phi(i, j))(s) : 0 \leq i < m \wedge 0 \leq j < v\} & \text{otherwise} \end{cases} \quad (*) \\
&= \left(\text{sem}(\text{frame}(\eta)) + \sum_{i,j} \{\text{sem}(\phi(i, j)) : 0 \leq i < m \wedge 0 \leq j < v\} \right) (s) \\
&= \text{sem} \left(\sum \left(\{\text{frame}(\eta)\} \cup \{(\phi(i, j) : 0 \leq i < m \wedge 0 \leq j < v)\} \right) \right) (s) \\
&= \text{sem}(\Phi(F))(s)
\end{aligned}$$

(*) If $\eta(s) = 0$, then $\sum_{s' \in S} \text{sem}(F)(s') \cdot P(s, s') = \sum_{i,j} \{\text{sem}(\phi(F, i, j))(s) : 0 \leq i < m \wedge 0 \leq j < v\}$.

$$\begin{aligned}
\sum_{s' \in S} \text{sem}(F)(s') \cdot P(s, s') &= \sum_{s' \in S} \left(\sum_{i,j} p_{i,j} \cdot [u_{i,j}(s) = s'] \cdot \varphi_i(s) \right) \cdot \text{sem}(F)(s') \quad (\text{def. program semantics}) \\
&= \sum_{i,j} p_{i,j} \cdot \left(\sum_{s' \in S} [(u_{i,j}(s) = s') \cdot \text{sem}(F)(s')] \cdot \varphi_i(s) \right) \\
&= \sum_{i,j} p_{i,j} \cdot (\text{sem}(F)(u_{i,j}(s)) \cdot \varphi_i(s)) \quad (\text{since unique } s' \in S, \text{ s.t. } s' = u_{i,j}(s)) \\
&= \sum_{i,j} p_{i,j} \cdot (\text{sem}(F) \circ u_{i,j})(s) \cdot \varphi_i(s) \cdot (1 - \eta(s)) \quad (\text{since } \eta(s) = 0) \\
&= \sum_{i,j} \text{sem}([p_{i,j}] \odot (F \circ u_{i,j}) \odot \text{frame}(\varphi_i \wedge \neg \eta))(s) \quad (\text{def. basic operations, see Table 4.1}) \\
&= \sum_{i,j} \{\text{sem}(\phi(F, i, j))(s) : 0 \leq i < m \wedge 0 \leq j < v\}
\end{aligned}$$

□

Lemma 3.

$$\begin{aligned}
u(s)(x_i) = s'(x_i) &\iff f_i(s) = s'(x_i) \quad (\text{def. updates}) \\
&\iff c_i \cdot s(x_i) + g_i(s) = s'(x_i) \\
&\iff s(x_i) = \frac{s'(x_i) - g_i(s)}{c_i} \\
&\iff f_i^{-1}(s') = s(x_i) \\
&\iff u^{-1}(s')(x_i) = s(x_i). \quad (\text{def. invertible update})
\end{aligned}$$

□

Proposition 21. Note that we abbreviate $\sum_{0 \leq i < m} \sum_{0 \leq j < b_i}$ to $\sum_{i,j}$.

$$\begin{aligned}
\Theta(\text{sem}(F))(s') &= \sum_{s \in S} P(s, s') \cdot \text{sem}(F)(s) \\
&= \sum_{s \in S} \left(\sum_{i,j} p_{i,j} \cdot [u_{i,j}(s) = s'] \cdot \varphi_i(s) \right) \cdot \text{sem}(F)(s) && (\text{def. program semantics}) \\
&= \sum_{i,j} p_{i,j} \cdot \sum_{s \in S} [u_{i,j}(s) = s'] \cdot \varphi_i(s) \cdot \text{sem}(F)(s) \\
&= \left(\sum_{i,j} \{ \text{sem}(\theta(F, i, j))(s') : 0 \leq i < m \wedge 0 \leq j < v \} \right) && (*) \\
&= \text{sem} \left(\sum \left(\{ \theta(F, i, j) : 0 \leq i < m \wedge 0 \leq j < v \} \right) \right) (s') \\
&= \text{sem}(\Theta(F))(s')
\end{aligned}$$

(*) For i, j , we prove that $p_{i,j} \cdot \sum_{s \in S} [u_{i,j}(s) = s'] \cdot \varphi_i(s) \cdot \text{sem}(F)(s) = \text{sem}(\theta(F, i, j))(s')$ in two cases:

1. Suppose that $u_{i,j}$ is constant, then:

$$\begin{aligned}
p_{i,j} \cdot \sum_{s \in S} [u_{i,j}(s) = s'] \cdot \varphi_i(s) \cdot \text{sem}(F)(s) &= p_{i,j} \cdot \sum_{s \in S} [u_{i,j}(*) = s'] \cdot \varphi_i(s) \cdot \text{sem}(F)(s) && (+) \\
&= p_{i,j} \cdot \sum_{s \in S} \varphi_i(s) \cdot \text{sem}(F)(s) \cdot [u_{i,j}(*) = s'] \\
&= \left(p_{i,j} \cdot \left(\sum_{s \in S} \varphi_i(s) \cdot \text{sem}(F)(s) \right) \odot (\lambda s'. [u_{i,j}(*) = s']) \right) (s') \\
&= \left(\text{sem}([p_{i,j} \cdot \text{sum}(F, \varphi)]) \odot \text{sem}(\text{frame}(\lambda s'. [u_{i,j}(*) = s'])) \right) (s') \\
&= \text{sem}(\theta(F, i, j))(s')
\end{aligned}$$

(+) Since $u_{i,j}$ is a constant update, its result is the same for any input state, i.e. there exists some state $s' \in S$, $\forall s \in S. u_{i,j}(s) = s'$, we denote this as $u_{i,j}(*) = s'$.

2. Suppose that $u_{i,j}$ is invertible, then:

$$\begin{aligned}
p_{i,j} \cdot \sum_{s \in S} [u_{i,j}(s) = s'] \cdot \varphi_i(s) \cdot \text{sem}(F)(s) &= p_{i,j} \cdot [u_{i,j}(s^*) = s'] \cdot \varphi_i(s^*) \cdot \text{sem}(F)(s^*) && (+) \\
&= p_{i,j} \cdot \text{sem}(F)(u_{i,j}^{-1}(s')) \cdot \varphi_i(u_{i,j}^{-1}(s')) && (+) \\
&= \left(\text{sem}([p_{i,j}]) \odot \text{sem}(F \circ u_{i,j}^{-1}) \odot \text{sem}(\text{frame}(\varphi_i \circ u_{i,j}^{-1})) \right) (s') \\
&= \text{sem}(\theta(F, i, j))(s')
\end{aligned}$$

(+) Since $u_{i,j}$ is an invertible update, there is a single $s^* \in S$, such that $u_{i,j}(s^*) = s'$ and $u_{i,j}^{-1}(s') = s^*$. □

Proposition 22. Let $W \in [0, 1]_{\text{sym}}^S$, $r \in [0, 1]$ such that $\mathcal{G} = \{(W, r)\}$.

$$\begin{aligned}
\Psi(\text{sem}(\{(W, r)\})) &= \Psi(\{v : \text{sem}(W) \cdot v \leq r\}) && (\text{semantics of sym. lf-sets}) \\
&= \{v : \Theta(\text{sem}(U)) \cdot v \leq t\} && (\text{Lemma 16}) \\
&= \{v : \text{sem}(\Theta(U)) \cdot v \leq t\} && (\text{Proposition 21}) \\
&= \text{sem}(\{(\Theta(U), t)\}) && (\text{semantics of sym. lf-sets}) \\
&= \text{sem}(\Psi(\{(W, r)\})) && (\text{def. } \Psi)
\end{aligned}$$

□

A.4 Generalization

Proposition 23. By looking at the definitions of each of the generalization methods, we can see that in every case, indeed $z_g(s) \leq \delta$. $\square$

Proposition 24. Simply put, whenever our obtained z is not a legit heuristic, we resort to the fallback. If Algorithm 3 reaches line 7 or 9, then legitness follows from legitness of the fallback heuristic. If Algorithm 3 never reaches line 7 or 9, then the following:

1. Let δ^i and z_g^i , denote their values at the iteration of the for loop (line 2) with state s_i . At line 8,

$$\begin{aligned}
 z \cdot W &= \bigwedge \{z_g^0 \dots z_g^{n-1}\} \cdot W \\
 &= \sum_{s_i \in S, W(s) \neq 0} z_g^i(s) \cdot W(s_i) \\
 &= \sum_{s_i \in S} z_g^i(s_i) \cdot W(s_i) \\
 &\leq \sum_{s \in S} \delta^i \cdot W(s) \quad (\text{since } \mathbf{sg} \text{ is a valid state generalization})
 \end{aligned}$$

And from lines 4-5, we know that if $r' = r - \sum_{s \in S} \delta^i \cdot W(s)$. Furthermore, since line 7 is never reached, $r' \geq 0$. Then

$$r' \geq 0 \implies \sum_{s \in S} \delta^i \cdot W(s) \leq r \implies z \cdot W \leq r \implies z \in \{v : v \cdot W \leq r\} = \mathcal{G}_k.$$

2. Since 9 is never reached, we know from line 8 that, $\Phi(F_k \wedge z) \leq z$.

In conclusion, Algorithm 3 is a legit **Refine** heuristic in either case. $\square$

Appendix B

Computation of $\bigwedge \mathcal{Z}_k$

This algorithm arose out of an email correspondence with Kori, who used a similar algorithm in their paper (although it is not explicitly described there) [23].

Algorithm 11 ComputeMeet

Parameters: $m \in [0, 1]^S, r \in \mathbb{Q}, s_0 \dots s_{n-1} \in S^+, l, k \in \{0 \dots n-1\} \cup \{\perp\}, w \in [0, 1]^S, F \in [0, 1]^S$

Result: If $\mathcal{Z}_k \neq \emptyset$, then true, $\bigwedge \mathcal{Z}_k$. If $\mathcal{Z}_k = \emptyset$, then false, **1**

```

1: if  $l = n - 1$  then
2:   if  $k = \perp$  and  $r = 0$  then
3:     return true,  $F$ 
4:   else if  $k \neq \perp$  and  $m(s_k) \leq \frac{r}{w(s_k)} \leq 1$  then
5:      $F(s) \leftarrow \frac{r}{w(s_k)}$ 
6:     return true,  $F$ 
7:   else
8:     return false, 1
9: else
10:   $c \leftarrow \mathbf{1}$ 
11:   $b \leftarrow \text{false}$ 
12:  if  $m(s_l) = 0$  then
13:     $F(s') \leftarrow 0$ 
14:     $b', c' \leftarrow \text{ComputeMeet}(m, r, s_0 \dots s_{n-1}, l + 1, k, w, F)$ 
15:     $c \leftarrow c \wedge c'$ 
16:     $b \leftarrow b \vee b'$ 
17:  if  $w(s_l) \leq r$  then
18:     $F(s') \leftarrow 1$ 
19:     $b', c' \leftarrow \text{ComputeMeet}(m, r - w(s_l), s_0 \dots s_{n-1}, l + 1, k, w, F)$ 
20:     $c \leftarrow c \wedge c'$ 
21:     $b \leftarrow b \vee b'$ 
22:  if  $s = \perp$  then
23:     $b', c' \leftarrow \text{ComputeMeet}(m, r, s_0 \dots s_{n-1}, l + 1, l, w, F)$ 
24:     $c \leftarrow c \wedge c'$ 
25:     $b \leftarrow b \vee b'$ 
26:  return  $Z, b$ 

```

Definition 57. Let $\text{CM}(F_{k-1}, \mathcal{G}_k) = \text{ComputeMeet}(F_{k-1}, r, \text{coeffs}, s \in S \cup \{\perp\}, F_0)$, where $F_{k-1}, v \in [0, 1]^S, r \in [0, 1], \mathcal{G}_k \in ([0, 1]^S)^\downarrow$, such that $\mathcal{G}_k = \{v : v \cdot w \leq r\}$, $F_0 \in [0, 1]$, such that $F_0(s) = \begin{cases} 0 & \text{if } F_{k-1}(s) = 0 \wedge w(s) = 0 \\ 1 & \text{otherwise} \end{cases}$ and $\text{coeffs} = \{s \in S : w(s) \neq 0\}$.

■ **Proposition 25.** Let $b, c = \text{CM}(F_{k-1}, \mathcal{G}_k)$. $b = \text{true} \iff \mathcal{Z}_k \neq \emptyset$, and if $\mathcal{Z}_k \neq \emptyset$ then $c = \bigwedge \mathcal{Z}_k$.

Proof. During each inductive step of the algorithm, we advance through the sequence $s_0 \dots s_{n-1}$ one by one, and possibly choose one coordinate as the *fractional coordinate*. This is the single coordinate that possibly is not 0 or 1.

We will use k to denote the fractional coordinate ($\perp$ if it was not already chosen), and l (with $k < l$) to note the advancement through the sequence. At each step, the algorithm considers three possibilities for s_l :

- Set s_l to 0
- Set s_l to 1
- Choose s_l as the fractional coordinate, set $k = l$ (if k was previously unset).

A frame F is used to keep track of the coordinates that were already set in previous steps.

Formally, one can define

$$\begin{aligned} Q(k, l, d, r) &= \sum_{l \leq i \leq n-1, i \neq k} d(s_i) \cdot w(s_i) + d(s_k) \cdot w(s_k) = r \wedge d \text{ is almost 0-1-val.}, \text{ with fractional coordinate } s_k. \\ Q(\perp, l, d, r) &= \sum_{l \leq i \leq n-1} d(s_i) \cdot w(s_i) = r \wedge d \text{ is almost 0-1-val.} \\ R(k, l, d, F) &= \forall i \in \{0 \dots l-1\} \setminus \{k\}. d(s_i) = F(s_i) \wedge d(s_k) \in [0, 1] \\ \mathcal{Z}(k, l, r, F) &= \{d : Q(k, l, d, r) \wedge R(k, l, d, F) \wedge m \leq d\} \end{aligned}$$

Now using induction, it can be shown that

for all $m, w \in [0, 1]^S$, 0-1-valued $F \in [0, 1]^S$ where $m \leq F$, $l \in \{0 \dots n-1\}$, $k \in \{\perp\} \cup \{k' : k' < l\}$, $r \in [0, 1]$,
if $b, c, \text{ComputeMeet}(m, r, s_0 \dots s_{n-1}, l, k, w, F)$, then $b = \text{true} \iff \bigwedge \mathcal{Z}(k, l, r, F) \neq \emptyset$ and
 $\mathcal{Z}(k, l, r, F) \neq \emptyset \implies c = \bigwedge \mathcal{Z}(k, l, r, F)$.

This yields the proposition by applying Definition 57.

- Base cases ($l = n-1$).

- If $k = \perp$ and $r = 0$, $\mathcal{Z}(\perp, 0, r, F) = \{d : 0 = r \wedge d \text{ is almost 0-1-val.} \wedge d = F \wedge m \leq d\} = \{F\}$. And indeed $b = \text{true}$ and $c = F$.
- If $k = \perp$ and $r > 0$, then $\mathcal{Z}(\perp, 0, r, F) = \{d : 0 = r \wedge \dots\} = \emptyset$. And indeed, $b = \text{false}$ and $c = \mathbf{1}$.
- If $k \neq \perp$ and $m(s_k) \leq \frac{r}{w(s_k)} \leq 1$, then

$$\begin{aligned} \mathcal{Z}(k, l, r, F) &= \{d : d(s_k) \cdot w(s_k) = r \wedge (d \text{ is almost 0-1-val. with f.c. } s_k) \wedge R(k, l, d, F) \wedge m \leq d\} \\ &= \{d : \forall i \in \{0 \dots l-1\} \setminus \{k\}. d(s_i) = F(s_i) \wedge d(s_k) = \frac{r}{w(s_k)}\} \\ &= \{F'\} \end{aligned}$$

$$\text{where } F'(s_i) = \begin{cases} F(s_i) & \text{if } i \neq k \\ \frac{r}{w(s_k)} & \text{otherwise} \end{cases}, \text{ and indeed, } b = \text{true} \text{ and } c = F'.$$

- If $k \neq \perp$ and $m(s_k) > \frac{r}{w(s_k)}$, then $\mathcal{Z}(k, l, r, F) = \{d : \dots d(s_k) = \frac{r}{w(s_k)} \wedge m(s_k) \leq d(s_k) \dots\} = \emptyset$. And indeed, $b = \text{false}$ and $c = \mathbf{1}$.
- If $k \neq \perp$ and $\frac{r}{w(s_k)} > 1$, then $\mathcal{Z}(k, l, r, F) = \{d : \dots d(s_k) \in [0, 1] \wedge d(s_k) \in [0, 1]\} = \emptyset$. And indeed, $b = \text{false}$ and $c = \mathbf{1}$.

- Inductive cases ($l \leq n$). Let $F_0(s) = \begin{cases} F(s_i) & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$, $F_1(s) = \begin{cases} F(s_i) & \text{if } i = j \\ 1 & \text{otherwise} \end{cases}$.

Furthermore, let

$$\begin{aligned} b_0, F'_0 &\leftarrow \text{ComputeMeet}(m, r, s_0 \dots s_{n-1}, l+1, k, w, F) \\ b_1, F'_1 &\leftarrow \text{ComputeMeet}(m, r - w(s_l), s_0 \dots s_{n-1}, l+1, k, w, F) \\ b_2, F'_2 &\leftarrow \text{ComputeMeet}(m, r, s_0 \dots s_{n-1}, l+1, l, w, F). \end{aligned}$$

By the inductive hypothesis,

$$\begin{aligned}
b_0 = \text{true} &\iff \mathcal{Z}(\perp, l+1, r, F_0) \neq \emptyset & \wedge F'_0 &= \bigwedge \mathcal{Z}(\perp, l+1, r, F_0) \\
b_1 = \text{true} &\iff \mathcal{Z}(\perp, l+1, r - w(s_l), F_1) \neq \emptyset & \wedge F'_1 &= \bigwedge \mathcal{Z}(\perp, l+1, r - w(s_l), F_1) \\
b_2 = \text{true} &\iff \mathcal{Z}(k, l+1, r, F) \neq \emptyset & \wedge F'_2 &= \bigwedge \mathcal{Z}(k, l+1, r, F)
\end{aligned}$$

Finally, note that if $m(s_l) = 0$, then $\mathcal{Z}(\perp, l+1, r, F_0) = \emptyset$,
and if $w(s_l) > r$, then $\mathcal{Z}(\perp, l+1, r - w(s_l), F_1) = \emptyset$. Now we have the following cases.

- If $k = \perp$, then $\mathcal{Z}(\perp, l, r, F) = \mathcal{Z}(\perp, l+1, r, F_0) \cup \mathcal{Z}(\perp, l+1, r - w(s_l), F_1) \cup \mathcal{Z}(l, l+1, r, F)$. Then indeed, $b = \text{true} \iff \bigwedge \mathcal{Z}(k, l, r, F) \neq \emptyset$ and $\mathcal{Z}(k, l, r, F) \neq \emptyset \implies c = \bigwedge \mathcal{Z}(k, l, r, F)$.
- If $k \neq \perp$, then $\mathcal{Z}(k, l, r, F) = \mathcal{Z}(k, l+1, r, F_0) \cup \mathcal{Z}(k, l+1, r - w(s_l), F_1)$. Then indeed, $b = \text{true} \iff \bigwedge \mathcal{Z}(k, l, r, F) \neq \emptyset$ and $\mathcal{Z}(k, l, r, F) \neq \emptyset \implies c = \bigwedge \mathcal{Z}(k, l, r, F)$.

□

Appendix C

Data usage statement

The data that was used in this project best fits data statement “Variant 2 – Use of data without personal data.”

Data collection

- ☒ I generated data myself
- ☐ I reused existing datasets (source cited)
- ☒ I generated software/code

Storage during the research

- ☐ Research institute
- ☐ RIS for Students
- ☒ External facility (explain)

Archiving after completion

- ☐ Data are (also) stored within the research institute
- ☐ Data are stored exclusively by an external party (in case of NDA)
- ☒ Data have been published
- ☐ Data have not been retained (justification provided in the report)

Accessibility

- ☒ Openly accessible
- ☐ Access restricted (provide justification)

This data usage statement concerns the run-time results in Chapter 6. The data was stored on the researcher’s laptop, and on a private repository on GitHub, as well as in this document in Table 6.1. The means of generating the data about our implementation are publicly available on GitHub ¹. For generating the data on Kori *et al.*’s *AdjointPDR*[↓] and Storm, we refer to [23] and [19]. Furthermore, the benchmark models that we used in this thesis are available in the same GitHub repository. The data will be publicly available indefinitely as there is no valid reason to limit its storage.

No other data (including personal data) was created, collected or processed during this thesis.

¹<https://github.com/YouGuessedMyName/master-thesis>

Bibliography

- [1] Samson Abramsky and Achim Jung. “Domain Theory”. In: *Handbook of logic in computer science* (Oct. 1995). DOI: 10.1093/oso/9780198537625.003.0001.
- [2] Christel Baier and Joost-Pieter Katoen. *Principles of model checking*. MIT Press, 2008.
- [3] Christel Baier et al. “Ensuring the Reliability of Your Model Checker: Interval Iteration for Markov Decision Processes”. In: *Computer Aided Verification*. Ed. by Rupak Majumdar and Viktor Kunčák. Cham: Springer International Publishing, 2017, pp. 160–180. ISBN: 978-3-319-63387-9.
- [4] Kevin Batz. “Automated deductive verification of probabilistic programs”. PhD thesis. RWTH Aachen University, 2024.
- [5] Kevin Batz et al. “PrIC3: property directed reachability for MDPs”. In: *International Conference on Computer Aided Verification*. Springer, 2020, pp. 512–538.
- [6] Aaron R Bradley. “SAT-based model checking without unrolling”. In: *International Workshop on Verification, Model Checking, and Abstract Interpretation*. Springer, 2011, pp. 70–87.
- [7] Krishnendu Chatterjee et al. “Fixed Point Certificates for Reachability and Expected Rewards in MDPs”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by Arie Gurfinkel and Marijn Heule. Cham: Springer Nature Switzerland, 2025, pp. 130–151. ISBN: 978-3-031-90653-4.
- [8] S. Cheshire and Bernard Aboba. “Dynamic Configuration of IPv4 Link-Local Addresses”. In: *IETF Internet Draft* (Jan. 2005).
- [9] Alessandro Cimatti and Alberto Griggio. “Software model checking via IC3”. In: *International Conference on Computer Aided Verification*. Springer, 2012, pp. 277–293.
- [10] Pedro D’Argenio et al. “Reachability Analysis of Probabilistic Systems by Successive Refinements”. In: vol. 2165. Aug. 2001, pp. 39–56. ISBN: 978-3-540-42556-4. DOI: 10.1007/3-540-44804-7_3.
- [11] Luca De Alfaro et al. “Symbolic model checking of probabilistic processes using MTBDDs and the Kronecker representation”. In: *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2000, pp. 395–410.
- [12] Niklas Eén, Alan Mishchenko, and Robert Brayton. “Efficient implementation of property directed reachability”. In: *2011 Formal Methods in Computer-Aided Design (FMCAD)*. IEEE. 2011, pp. 125–134.
- [13] Serge Haddad and Benjamin Monmege. “Interval iteration algorithm for MDPs and IMDPs”. In: *Theoretical Computer Science* 735 (2018), pp. 111–131.
- [14] Serge Haddad and Benjamin Monmege. “Reachability in MDPs: Refining convergence of value iteration”. In: *International Workshop on Reachability Problems*. Springer, 2014, pp. 125–137.
- [15] Ernst Moritz Hahn et al. “iscasMc: A web-based probabilistic model checker”. In: *International Symposium on Formal Methods*. Springer, 2014, pp. 312–317.
- [16] Arnd Hartmanns and Holger Hermanns. “The Modest Toolset: An integrated environment for quantitative modelling and verification”. In: *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2014, pp. 593–598.
- [17] Arnd Hartmanns and Benjamin Lucien Kaminski. “Optimistic value iteration”. In: *International Conference on Computer Aided Verification*. Springer, 2020, pp. 488–511.

- [18] Leen Helmink, Martin Paul Alexander Sellink, and Frits W Vaandrager. “Proof-checking a data link protocol”. In: *International Workshop on Types for Proofs and Programs*. Springer, 1994, pp. 127–165.
- [19] Christian Hensel et al. “The probabilistic model checker Storm”. In: *International Journal on Software Tools for Technology Transfer* 24.4 (2022), pp. 589–610.
- [20] Kryštof Hoder and Nikolaj Bjørner. “Generalized Property Directed Reachability”. In: *Theory and Applications of Satisfiability Testing — SAT 2012*. Ed. by Alessandro Cimatti and Roberto Sebastiani. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 157–171. ISBN: 978-3-642-31612-8.
- [21] Alexander Ivrii and Arie Gurfinkel. “Pushing to the top”. In: *Proceedings of the 15th Conference on Formal Methods in Computer-Aided Design*. FMCAD ’15. Austin, Texas: FMCAD Inc, 2015, pp. 65–72. ISBN: 9780983567851.
- [22] Matthias Köppe and Sven Verdoolaege. “Computing Parametric Rational Generating Functions with a Primal Barvinok Algorithm”. In: *The Electronic Journal of Combinatorics* 15.1 (Jan. 2008), R16. DOI: 10.37236/740. URL: <https://www.combinatorics.org/ojs/index.php/eljc/article/view/v15i1r16>.
- [23] Mayuko Kori et al. “Exploiting adjoints in property directed reachability analysis”. In: *International Conference on Computer Aided Verification*. Springer, 2023, pp. 41–63.
- [24] Mayuko Kori et al. “The lattice-theoretic essence of property directed reachability analysis”. In: *International Conference on Computer Aided Verification*. Springer, 2022, pp. 235–256.
- [25] Marta Kwiatkowska, Gethin Norman, and David Parker. “PRISM: Probabilistic symbolic model checker”. In: *International Conference on Modelling Techniques and Tools for Computer Performance Evaluation*. Springer, 2002, pp. 200–204.
- [26] Tim Lange et al. “IC3 software model checking”. In: *International Journal on Software Tools for Technology Transfer* 22.2 (2020), pp. 135–161.
- [27] Aaron Meurer et al. “SymPy: symbolic computing in Python”. In: *PeerJ Computer Science* 3 (Jan. 2017), e103. DOI: 10.7717/peerj-cs.103.
- [28] Leonardo de Moura and Nikolaj Bjørner. “Z3: An Efficient SMT Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Vol. 4963. Lecture Notes in Computer Science. Springer, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3_24.
- [29] David Park. “Fixpoint induction and proofs of program properties”. In: *Machine intelligence* 5 (1969).
- [30] Tim Quatmann and Joost-Pieter Katoen. “Sound value iteration”. In: *International Conference on Computer Aided Verification*. Springer, 2018, pp. 643–661.
- [31] Alfred Tarski. “A Lattice-Theoretical Fixpoint Theorem and Its Applications”. In: *Pacific Journal of Mathematics* 5 (1955), pp. 285–309.
- [32] Sven Verdoolaege. “isl: An Integer Set Library for the Polyhedral Model”. In: *Mathematical Software – ICMS 2010*. Ed. by Komei Fukuda et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 299–302. ISBN: 978-3-642-15582-6.
- [33] Glynn Winskel. *The formal semantics of programming languages: an introduction*. MIT Press, 1993.